Human review packet: PG23MonocultureMatching

Generated from byte-pinned source excerpts, the expanded PaperInterface specifications, and hash-bound raw-source-to-Spec screening records.

Paper: PG23MonocultureMatching
Paper id: PG23MonocultureMatching
Source version: arXiv:2312.09841 source archive cached locally
Source record: audit/paper_statement_map.json
Rows: 13
Generated: 2026-09-06
Source URL: official source

How to use this packet

For each source claim, compare the exact source excerpts with the expanded PaperInterface specification. Mark up this PDF or fill the blank reviewer box in the TeX source; return the annotated file for reconciliation into the paper-local human-review log.

Open the interactive dashboard

The interactive dashboard is an optional alternative to using this PDF; it presents the same review surface rather than an additional review requirement. After forking and cloning (or pulling) AppliedModelingLib, run the following from the repository root. The cache command is needed only the first time in a new clone.

```
cd <your-repository-checkout>
lake exe cache get
python3 scripts/review_dashboard.py --paper PG23MonocultureMatching --serve
```

Open http://127.0.0.1:8765/ in a browser and keep that terminal running while reviewing. The dashboard records saved annotations in this paper's local reviewer-owned review record.

Contents

  - PG23MonocultureMatching.pg23RawCutoffInf
  - PG23MonocultureMatching.pg23RawCutoffLe
  - PG23MonocultureMatching.pg23RawCutoffSup
  - PG23MonocultureMatching.pg23SourceChoice
  - PG23MonocultureMatching.pg23SourceAggregateDemand
  - PG23MonocultureMatching.pg23SourceMarketClearing
  - PG23MonocultureMatching.pg23SourceMatchingBlocks
  - PG23MonocultureMatching.pg23SourceMatchingFeasible
  - PG23MonocultureMatching.pg23SourceRank
  - PG23MonocultureMatching.pg23SourceScoreLevelNull
  - PG23MonocultureMatching.pg23SourceStableMatching
  - PG23MonocultureMatching.source_assumption_value_threshold
- Source claims (13)
  - review_proposition_cdfIncreasing_connectedSupportSpec
  - review_proposition_nonzeroMeasure_openIntervalSpec
  - review_proposition_probabilityFormulaSpec
  - review_corollary4_monocultureCutoff_lt_polycultureCutoffSpec
  - review_theorem3_differentialApplicationAccessSpec
  - review_proposition_nash_differentialApplicationAccessSpec
  - review_proposition_latticeSpec
  - review_lemma10_threePartSpec
  - review_lemma1_supplyDemandSpec
  - review_lemma2_equalCutoffsSpec
  - review_theorem1_wisdomSpec
  - review_theorem2_topChoiceSpec
  - review_lemma_equalCutoffs_differentialAccessSpec

Governing Lean declarations

These exact graph-bound declaration bodies are retained by the displayed specifications or reviewed prerequisites. They are supporting context and do not add source-result rows or semantic judgments.

AL16SupplyDemandMatching.AL16Cutoff

Declaration location: reusable library

Lean declaration body

type:
```
Type v → Type v
```

value:
```
fun (College : Type v) => College → ↑(Set.Icc 0 1)
```

AL16SupplyDemandMatching.AL16Ranking

Declaration location: reusable library

Lean declaration body

type:
```
(College : Type v) → [Fintype College] → Type (max 0 v)
```

value:
```
fun (College : Type v) [Fintype College] => College ≃ Fin (Fintype.card College)
```

AL16SupplyDemandMatching.AL16SourceStudent

Declaration location: reusable library

Lean declaration body

type:
```
(College : Type v) → [Fintype College] → Type v
```

value:
```
fun (College : Type v) [Fintype College] => AL16SupplyDemandMatching.AL16Ranking College × (College → ↑(Set.Icc 0 1))
```

Direct retained dependencies

- AL16SupplyDemandMatching.AL16Ranking

AL16SupplyDemandMatching.al16CutoffValue

Declaration location: reusable library

Lean declaration body

type:
```
{College : Type v} → AL16SupplyDemandMatching.AL16Cutoff College → College → ℝ
```

value:
```
fun {College : Type v} (P : AL16SupplyDemandMatching.AL16Cutoff College) (c : College) => ↑(P c)
```

Direct retained dependencies

- AL16SupplyDemandMatching.AL16Cutoff

AL16SupplyDemandMatching.al16RankPrefers

Declaration location: reusable library

Lean declaration body

type:
```
{Student : Type u} → {College : Type v} → (Student → College → ℕ) → Student → Option College → Option College → Prop
```

value:
```
fun {Student : Type u} {College : Type v} (rank : Student → College → ℕ) (theta : Student) (a a_1 : Option College) =>
  match a, a_1 with
  | some c, some d => rank theta c < rank theta d
  | some val, none => True
  | none, x => False
```

AL16SupplyDemandMatching.al16ScoreValue

Declaration location: reusable library

Lean declaration body

type:
```
{Student : Type u} → {College : Type v} → (Student → College → ↑(Set.Icc 0 1)) → Student → College → ℝ
```

value:
```
fun {Student : Type u} {College : Type v} (score : Student → College → ↑(Set.Icc 0 1)) (theta : Student)
    (c : College) =>
  ↑(score theta c)
```

AL16SupplyDemandMatching.al16Affordable

Declaration location: reusable library

Lean declaration body

type:
```
{Student : Type u} →
  {College : Type v} →
    (Student → College → ↑(Set.Icc 0 1)) → AL16SupplyDemandMatching.AL16Cutoff College → Student → College → Prop
```

value:
```
fun {Student : Type u} {College : Type v} (score : Student → College → ↑(Set.Icc 0 1))
    (P : AL16SupplyDemandMatching.AL16Cutoff College) (theta : Student) (c : College) =>
  AL16SupplyDemandMatching.al16CutoffValue P c ≤ AL16SupplyDemandMatching.al16ScoreValue score theta c
```

Direct retained dependencies

- AL16SupplyDemandMatching.AL16Cutoff
- AL16SupplyDemandMatching.al16CutoffValue
- AL16SupplyDemandMatching.al16ScoreValue

AL16SupplyDemandMatching.al16FavoriteAffordableCollege

Declaration location: reusable library

Lean declaration body

type:
```
{Student : Type u} →
  {College : Type v} →
    [Fintype College] →
      (score : Student → College → ↑(Set.Icc 0 1)) →
        (Student → College → ℕ) →
          (P : AL16SupplyDemandMatching.AL16Cutoff College) →
            (theta : Student) → (∃ (c : College), AL16SupplyDemandMatching.al16Affordable score P theta c) → College
```

value:
```
fun {Student : Type u} {College : Type v} [Fintype College] (score : Student → College → ↑(Set.Icc 0 1))
    (rank : Student → College → ℕ) (P : AL16SupplyDemandMatching.AL16Cutoff College) (theta : Student)
    (h : ∃ (c : College), AL16SupplyDemandMatching.al16Affordable score P theta c) =>
  Classical.choose (AL16SupplyDemandMatching.al16FavoriteAffordableCollege._proof_1 score rank P theta h)
```

Direct retained dependencies

- AL16SupplyDemandMatching.AL16Cutoff
- AL16SupplyDemandMatching.al16Affordable

AL16SupplyDemandMatching.al16FavoriteAffordableChoice

Declaration location: reusable library

Lean declaration body

```
type:
{Student : Type u} →
  {College : Type v} →
    [Fintype College] →
      (Student → College → ↑(Set.Icc 0 1)) →
        (Student → College → ℕ) → AL16SupplyDemandMatching.AL16Cutoff College → Student → Option College

value:
fun {Student : Type u} {College : Type v} [Fintype College] (score : Student → College → ↑(Set.Icc 0 1))
    (rank : Student → College → ℕ) (P : AL16SupplyDemandMatching.AL16Cutoff College) (theta : Student) =>
  if h : ∃ (c : College), AL16SupplyDemandMatching.al16Affordable score P theta c then
    some (AL16SupplyDemandMatching.al16FavoriteAffordableCollege score rank P theta h)
  else none
```

Direct retained dependencies

- AL16SupplyDemandMatching.AL16Cutoff
- AL16SupplyDemandMatching.al16Affordable
- AL16SupplyDemandMatching.al16FavoriteAffordableCollege

AL16SupplyDemandMatching.al16SourceRank

Declaration location: reusable library

Lean declaration body

```
type:
{College : Type v} → [inst : Fintype College] → AL16SupplyDemandMatching.AL16SourceStudent College → College → ℕ

value:
fun {College : Type v} [Fintype College] (theta : AL16SupplyDemandMatching.AL16SourceStudent College) (c : College) =>
  ↑(theta.1 c)
```

Direct retained dependencies

- AL16SupplyDemandMatching.AL16Ranking
- AL16SupplyDemandMatching.AL16SourceStudent

AL16SupplyDemandMatching.al16SourceScore

Declaration location: reusable library

Lean declaration body

```
type:
{College : Type v} →
  [inst : Fintype College] → AL16SupplyDemandMatching.AL16SourceStudent College → College → ↑(Set.Icc 0 1)

value:
fun {College : Type v} [Fintype College] (theta : AL16SupplyDemandMatching.AL16SourceStudent College) (c : College) =>
  theta.2 c
```

Direct retained dependencies

- AL16SupplyDemandMatching.AL16Ranking
- AL16SupplyDemandMatching.AL16SourceStudent

AL16SupplyDemandMatching.al16SourceChoice

Declaration location: reusable library

Lean declaration body

```
type:
{College : Type v} →
  [inst : Fintype College] →
    AL16SupplyDemandMatching.AL16Cutoff College → AL16SupplyDemandMatching.AL16SourceStudent College → Option College

value:
fun {College : Type v} [Fintype College] (P : AL16SupplyDemandMatching.AL16Cutoff College)
    (a : AL16SupplyDemandMatching.AL16SourceStudent College) =>
  AL16SupplyDemandMatching.al16FavoriteAffordableChoice AL16SupplyDemandMatching.al16SourceScore
    AL16SupplyDemandMatching.al16SourceRank P a
```

Direct retained dependencies

- AL16SupplyDemandMatching.AL16Cutoff
- AL16SupplyDemandMatching.AL16SourceStudent
- AL16SupplyDemandMatching.al16FavoriteAffordableChoice
- AL16SupplyDemandMatching.al16SourceRank
- AL16SupplyDemandMatching.al16SourceScore

AL16SupplyDemandMatching.instMeasurableSpaceAL16Ranking

Declaration location: reusable library

Lean declaration body

```
type:
{College : Type v} → [inst : Fintype College] → MeasurableSpace (AL16SupplyDemandMatching.AL16Ranking College)

value:
fun {College : Type v} [Fintype College] => ⊤
```

Direct retained dependencies

- AL16SupplyDemandMatching.AL16Ranking

AppliedModelingLib.Matching.IsLowerBoundOn

Declaration location: reusable library

Lean declaration body

```
type:
{α : Type u} → (α → Prop) → (α → α → Prop) → Set α → α → Prop

value:
fun {α : Type u} (valid : α → Prop) (le : α → α → Prop) (S : Set α) (x : α) =>
  valid x ∧ ∀ (y : α), S y → valid y → le x y
```

AppliedModelingLib.Matching.IsGreatestLowerBoundOn

Declaration location: reusable library

Lean declaration body

```
type:
{α : Type u} → (α → Prop) → (α → α → Prop) → Set α → α → Prop

value:
fun {α : Type u} (valid : α → Prop) (le : α → α → Prop) (S : Set α) (x : α) =>
  AppliedModelingLib.Matching.IsLowerBoundOn valid le S x ∧
    ∀ (z : α), AppliedModelingLib.Matching.IsLowerBoundOn valid le S z → le z x
```

Direct retained dependencies

- AppliedModelingLib.Matching.IsLowerBoundOn

AppliedModelingLib.Matching.IsUpperBoundOn

Declaration location: reusable library

Lean declaration body

```
type:
{α : Type u} → (α → Prop) → (α → α → Prop) → Set α → α → Prop

value:
fun {α : Type u} (valid : α → Prop) (le : α → α → Prop) (S : Set α) (x : α) =>
  valid x ∧ ∀ (y : α), S y → valid y → le y x
```

AppliedModelingLib.Matching.IsLeastUpperBoundOn

Declaration location: reusable library

Lean declaration body

type:
{α : Type u} → (α → Prop) → (α → α → Prop) → Set α → α → Prop

value:
fun {α : Type u} (valid : α → Prop) (le : α → α → Prop) (S : Set α) (x : α) =>
  AppliedModelingLib.Matching.IsUpperBoundOn valid le S x ∧
    ∀ (z : α), AppliedModelingLib.Matching.IsUpperBoundOn valid le S z → le x z

Direct retained dependencies

- AppliedModelingLib.Matching.IsUpperBoundOn

AppliedModelingLib.Matching.CompleteLatticeOn

Declaration location: reusable library

Lean declaration body

field exists_valid:
∀ {α : Type u} {valid : α → Prop} {le : α → α → Prop},
  AppliedModelingLib.Matching.CompleteLatticeOn valid le → ∃ (x : α), valid x

field le_refl:
∀ {α : Type u} {valid : α → Prop} {le : α → α → Prop},
  AppliedModelingLib.Matching.CompleteLatticeOn valid le → ∀ {x : α}, valid x → le x x

field le_trans:
∀ {α : Type u} {valid : α → Prop} {le : α → α → Prop},
  AppliedModelingLib.Matching.CompleteLatticeOn valid le →
    ∀ {x y z : α}, valid x → valid y → valid z → le x y → le y z → le x z

field le_antisymm:
∀ {α : Type u} {valid : α → Prop} {le : α → α → Prop},
  AppliedModelingLib.Matching.CompleteLatticeOn valid le → ∀ {x y : α}, valid x → valid y → le x y → le y x → x = y

field sup_exists:
∀ {α : Type u} {valid : α → Prop} {le : α → α → Prop},
  AppliedModelingLib.Matching.CompleteLatticeOn valid le →
    ∀ (S : Set α), (∃ (x : α), S x ∧ valid x) → ∃ (x : α), AppliedModelingLib.Matching.IsLeastUpperBoundOn valid le S x

field inf_exists:
∀ {α : Type u} {valid : α → Prop} {le : α → α → Prop},
  AppliedModelingLib.Matching.CompleteLatticeOn valid le →
    ∀ (S : Set α),
      (∃ (x : α), S x ∧ valid x) → ∃ (x : α), AppliedModelingLib.Matching.IsGreatestLowerBoundOn valid le S x

Direct retained dependencies

- AppliedModelingLib.Matching.IsGreatestLowerBoundOn
- AppliedModelingLib.Matching.IsLeastUpperBoundOn

AppliedModelingLib.Matching.cutoffWeaklyCrossedOn

Declaration location: reusable library

Lean declaration body

type:
{College : Type u} → Finset College → (College → ℝ) → (College → ℝ) → Prop

value:
fun {College : Type u} (active : Finset College) (score cutoff : College → ℝ) => ∃ c ∈ active, cutoff c ≤ score c

AppliedModelingLib.Matching.noisyScore

Declaration location: reusable library

Lean declaration body

type:
{College : Type u} → ℝ → (College → ℝ) → College → ℝ

value:
fun {College : Type u} (v : ℝ) (noise : College → ℝ) (a : College) => v + noise a

AppliedModelingLib.Matching.cutoffWeakCrossingProbability

Declaration location: reusable library

Lean declaration body

```
type:
{College : Type u} →
  [inst : MeasurableSpace (College → ℝ)] → MeasureTheory.Measure (College → ℝ) → Finset College → ℝ → (College → ℝ) → ℝ

value:
fun {College : Type u} [MeasurableSpace (College → ℝ)] (μ : MeasureTheory.Measure (College → ℝ))
    (active : Finset College) (v : ℝ) (cutoff : College → ℝ) =>
  μ.real
    {noise : College → ℝ |
      AppliedModelingLib.Matching.cutoffWeaklyCrossedOn active (AppliedModelingLib.Matching.noisyScore v noise) cutoff}
```

Direct retained dependencies

- AppliedModelingLib.Matching.cutoffWeaklyCrossedOn
- AppliedModelingLib.Matching.noisyScore

AppliedModelingLib.Probability.ascendingOrderStatistic

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} → (Fin n → ℝ) → Fin n → ℝ

value:
fun {n : ℕ} (sample : Fin n → ℝ) (rank : Fin n) => sample ((Tuple.sort sample) rank)
```

AppliedModelingLib.Probability.lowerCDFMass

Declaration location: reusable library

Lean declaration body

```
type:
MeasureTheory.Measure ℝ → ℝ → ℝ

value:
fun (μ : MeasureTheory.Measure ℝ) (x : ℝ) => μ.real (Set.Iic x)
```

AppliedModelingLib.Probability.topSampleRank

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} → [NeZero n] → Fin n

value:
fun {n : ℕ} [NeZero n] => ⟨0, AppliedModelingLib.Probability.topSampleRank._proof_1⟩
```

AppliedModelingLib.Matching.topSampleRank

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} → [NeZero n] → Fin n

value:
fun {n : ℕ} [NeZero n] => AppliedModelingLib.Probability.topSampleRank
```

Direct retained dependencies

- AppliedModelingLib.Probability.topSampleRank

AppliedModelingLib.Probability.upperOrderStatistic

Declaration location: reusable library

Lean declaration body

type:
{n : ℕ} → (Fin n → ℝ) → Fin n → ℝ

value:
fun {n : ℕ} (sample : Fin n → ℝ) (rankFromTop : Fin n) =>
  AppliedModelingLib.Probability.ascendingOrderStatistic sample rankFromTop.rev

Direct retained dependencies

- AppliedModelingLib.Probability.ascendingOrderStatistic

AppliedModelingLib.Probability.expectedUpperOrderStatistic

Declaration location: reusable library

Lean declaration body

type:
{n : ℕ} → MeasureTheory.Measure (Fin n → ℝ) → Fin n → ℝ

value:
fun {n : ℕ} (μ : MeasureTheory.Measure (Fin n → ℝ)) (rankFromTop : Fin n) =>
  ∫ (sample : Fin n → ℝ), AppliedModelingLib.Probability.upperOrderStatistic sample rankFromTop ∂μ

Direct retained dependencies

- AppliedModelingLib.Probability.upperOrderStatistic

AppliedModelingLib.Probability.expectedTopOrderStatistic

Declaration location: reusable library

Lean declaration body

type:
{n : ℕ} → [NeZero n] → MeasureTheory.Measure (Fin n → ℝ) → ℝ

value:
fun {n : ℕ} [NeZero n] (μ : MeasureTheory.Measure (Fin n → ℝ)) =>
  AppliedModelingLib.Probability.expectedUpperOrderStatistic μ AppliedModelingLib.Probability.topSampleRank

Direct retained dependencies

- AppliedModelingLib.Probability.expectedUpperOrderStatistic
- AppliedModelingLib.Probability.topSampleRank

AppliedModelingLib.Probability.expectedTopOrderStatisticSeq

Declaration location: reusable library

Lean declaration body

type:
((n : ℕ) → MeasureTheory.Measure (Fin (n + 1) → ℝ)) → ℕ → ℝ

value:
fun (sampleLaw : (n : ℕ) → MeasureTheory.Measure (Fin (n + 1) → ℝ)) (n : ℕ) =>
  AppliedModelingLib.Probability.expectedTopOrderStatistic (sampleLaw n)

Direct retained dependencies

- AppliedModelingLib.Probability.expectedTopOrderStatistic

AppliedModelingLib.Probability.topOrderDeviationProbability

Declaration location: reusable library

Lean declaration body

type:
{n : ℕ} → [NeZero n] → MeasureTheory.Measure (Fin n → ℝ) → ℝ → ℝ → ℝ

value:
fun {n : ℕ} [NeZero n] (μ : MeasureTheory.Measure (Fin n → ℝ)) (center ε : ℝ) =>
  μ.real
    {noise : Fin n → ℝ |
      ε <
        |AppliedModelingLib.Probability.upperOrderStatistic noise AppliedModelingLib.Probability.topSampleRank -
          center|}

Direct retained dependencies

- AppliedModelingLib.Probability.topSampleRank
- AppliedModelingLib.Probability.upperOrderStatistic

AppliedModelingLib.Probability.topOrderDeviationProbabilitySeq

Declaration location: reusable library

Lean declaration body

type:
((n : ℕ) → MeasureTheory.Measure (Fin (n + 1) → ℝ)) → (ℕ → ℝ) → ℝ → ℕ → ℝ

value:
fun (sampleLaw : (n : ℕ) → MeasureTheory.Measure (Fin (n + 1) → ℝ)) (center : ℕ → ℝ) (ε : ℝ) (n : ℕ) =>
  AppliedModelingLib.Probability.topOrderDeviationProbability (sampleLaw n) (center n) ε

Direct retained dependencies

- AppliedModelingLib.Probability.topOrderDeviationProbability

AppliedModelingLib.Probability.TopOrderDeviationConcentrating

Declaration location: reusable library

Lean declaration body

type:
((n : ℕ) → MeasureTheory.Measure (Fin (n + 1) → ℝ)) → (ℕ → ℝ) → Prop

value:
fun (sampleLaw : (n : ℕ) → MeasureTheory.Measure (Fin (n + 1) → ℝ)) (center : ℕ → ℝ) =>
  ∀ (ε : ℝ),
    0 < ε →
      Filter.Tendsto (AppliedModelingLib.Probability.topOrderDeviationProbabilitySeq sampleLaw center ε) Filter.atTop
        (nhds 0)

Direct retained dependencies

- AppliedModelingLib.Probability.topOrderDeviationProbabilitySeq

AppliedModelingLib.Probability.upperTailMass

Declaration location: reusable library

Lean declaration body

type:
MeasureTheory.Measure ℝ → ℝ → ℝ

value:
fun (μ : MeasureTheory.Measure ℝ) (x : ℝ) => μ.real (Set.Ioi x)

AppliedModelingLib.Probability.UpperTailThresholdCertificate

Declaration location: reusable library

Lean declaration body

```
field tail_eq_capacity:
∀ {μ : MeasureTheory.Measure ℝ} {capacity threshold : ℝ},
  AppliedModelingLib.Probability.UpperTailThresholdCertificate μ capacity threshold →
    AppliedModelingLib.Probability.upperTailMass μ threshold = capacity
```

Direct retained dependencies

- AppliedModelingLib.Probability.upperTailMass

AppliedModelingLib.pmfExp

Declaration location: reusable library

Lean declaration body

```
type:
{α : Type u_1} → [Fintype α] → PMF α → (α → ℝ) → ℝ

value:
fun {α : Type u_1} [Fintype α] (μ : PMF α) (f : α → ℝ) => ∑ a : α, (μ a).toReal * f a
```

AppliedModelingLib.pmfProduct

Declaration location: reusable library

Lean declaration body

```
type:
(ι : Type u_1) → (α : Type u_2) → [Fintype ι] → [DecidableEq ι] → [Fintype α] → PMF α → PMF (ι → α)

value:
fun (ι : Type u_1) (α : Type u_2) [Fintype ι] [DecidableEq ι] [Fintype α] (μ : PMF α) =>
  PMF.ofFintype (fun (f : ι → α) => ∏ i : ι, μ (f i)) (AppliedModelingLib.pmfProduct._proof_1 ι α μ)
```

PG23MonocultureMatching.NoProfitableApplicationDeviation

Declaration location: paper-local

Lean declaration body

```
type:
{College : Type v} → (Finset College → ℝ) → Finset College → (Finset College → Prop) → Prop

value:
fun {College : Type v} (payoff : Finset College → ℝ) (topChoiceSet : Finset College)
    (feasible : Finset College → Prop) =>
  ∀ (deviation : Finset College), feasible deviation → payoff deviation ≤ payoff topChoiceSet
```

PG23MonocultureMatching.PG23MonocultureSample

Declaration location: paper-local

Lean declaration body

```
type:
(College : Type v) → [Fintype College] → Type (max v 0)

value:
fun (College : Type v) [Fintype College] => (ℝ × PG23MonocultureMatching.PG23Ranking College) × ℝ
```

Direct retained dependencies

- PG23MonocultureMatching.PG23Ranking

PG23MonocultureMatching.PG23PolycultureSample

Declaration location: paper-local

Lean declaration body

type:
(College : Type v) → [Fintype College] → Type v

value:
fun (College : Type v) [Fintype College] => (ℝ × PG23MonocultureMatching.PG23Ranking College) × (College → ℝ)

Direct retained dependencies

- PG23MonocultureMatching.PG23Ranking

PG23MonocultureMatching.favoriteSuccessfulApplicationUtility

Declaration location: paper-local

Lean declaration body

type:
{College : Type v} → [DecidableEq College] → ℝ → (College → ℝ) → Finset College → Finset College → ℝ

value:
fun {College : Type v} [DecidableEq College] (outsideUtility : ℝ) (utility : College → ℝ)
    (applicationSet successful : Finset College) =>
  if h : (applicationSet ∩ successful).Nonempty then (applicationSet ∩ successful).sup' h utility else outsideUtility

PG23MonocultureMatching.expectedFavoriteSuccessfulApplicationUtility

Declaration location: paper-local

Lean declaration body

type:
{College : Type v} →
  [Fintype College] → [DecidableEq College] → PMF (Finset College) → ℝ → (College → ℝ) → Finset College → ℝ

value:
fun {College : Type v} [Fintype College] [DecidableEq College] (successLaw : PMF (Finset College)) (outsideUtility : ℝ)
    (utility : College → ℝ) (applicationSet : Finset College) =>
  AppliedModelingLib.pmfExp successLaw fun (successful : Finset College) =>
    PG23MonocultureMatching.favoriteSuccessfulApplicationUtility outsideUtility utility applicationSet successful

Direct retained dependencies

- AppliedModelingLib.pmfExp
- PG23MonocultureMatching.favoriteSuccessfulApplicationUtility

PG23MonocultureMatching.iidEqualCutoffSuccessLaw

Declaration location: paper-local

Lean declaration body

type:
{College : Type v} → [Fintype College] → [DecidableEq College] → PMF Bool → Finset College → PMF (Finset College)

value:
fun {College : Type v} [Fintype College] [DecidableEq College] (successAtom : PMF Bool)
    (_applicationSet : Finset College) =>
  PMF.map (fun (outcome : College → Bool) => {c : College | outcome c = true})
    (AppliedModelingLib.pmfProduct College Bool successAtom)

Direct retained dependencies

- AppliedModelingLib.pmfProduct

PG23MonocultureMatching.maximumOrderStatisticConcentrating

Declaration location: paper-local

Lean declaration body

type:
((n : ℕ) → MeasureTheory.Measure (Fin (n + 1) → ℝ)) → (ℕ → ℝ) → Prop

value:
fun (sampleLaw : (n : ℕ) → MeasureTheory.Measure (Fin (n + 1) → ℝ)) (center : ℕ → ℝ) =>
  AppliedModelingLib.Probability.TopOrderDeviationConcentrating sampleLaw center

Direct retained dependencies

- AppliedModelingLib.Probability.TopOrderDeviationConcentrating

PG23MonocultureMatching.maximumOrderStatisticConcentratingAroundExpected

Declaration location: paper-local

Lean declaration body

type:
((n : ℕ) → MeasureTheory.Measure (Fin (n + 1) → ℝ)) → Prop

value:
fun (sampleLaw : (n : ℕ) → MeasureTheory.Measure (Fin (n + 1) → ℝ)) =>
  PG23MonocultureMatching.maximumOrderStatisticConcentrating sampleLaw
    (AppliedModelingLib.Probability.expectedTopOrderStatisticSeq sampleLaw)

Direct retained dependencies

- AppliedModelingLib.Probability.expectedTopOrderStatisticSeq
- PG23MonocultureMatching.maximumOrderStatisticConcentrating

PG23MonocultureMatching.pg23MonocultureScalarMatchDemand

Declaration location: paper-local

Lean declaration body

type:
MeasureTheory.Measure ℝ → MeasureTheory.Measure ℝ → ℝ → ℝ

value:
fun (valueLaw noiseLaw : MeasureTheory.Measure ℝ) (cutoff : ℝ) =>
  ∫ (v : ℝ), AppliedModelingLib.Probability.upperTailMass noiseLaw (cutoff - v) ∂valueLaw

Direct retained dependencies

- AppliedModelingLib.Probability.upperTailMass

PG23MonocultureMatching.pg23NoiseInteriorCutoffRegion

Declaration location: paper-local

Lean declaration body

type:
MeasureTheory.Measure ℝ → ℝ → ℝ → Prop

value:
fun (noiseLaw : MeasureTheory.Measure ℝ) (p a : ℝ) => {v : ℝ | p - v ∈ interior noiseLaw.support} a

PG23MonocultureMatching.pg23PolycultureMaxNoise

Declaration location: paper-local

Lean declaration body

type:
{College : Type v} → [Fintype College] → [Nonempty College] → (College → ℝ) → ℝ

value:
fun {College : Type v} [Fintype College] [Nonempty College] (noise : College → ℝ) =>
  Finset.univ.sup' Finset.univ_nonempty noise

PG23MonocultureMatching.pg23PolycultureMaxNoiseLaw

Declaration location: paper-local

Lean declaration body

type:
{College : Type v} → [Fintype College] → [Nonempty College] → MeasureTheory.Measure ℝ → MeasureTheory.Measure ℝ

value:
fun {College : Type v} [Fintype College] [Nonempty College] (noiseLaw : MeasureTheory.Measure ℝ) =>
  MeasureTheory.Measure.map PG23MonocultureMatching.pg23PolycultureMaxNoise
    (MeasureTheory.Measure.pi fun (x : College) => noiseLaw)

Direct retained dependencies

- PG23MonocultureMatching.pg23PolycultureMaxNoise

PG23MonocultureMatching.pg23ScoreNormalize

Declaration location: paper-local

Lean declaration body

type:
ℝ → ↑(Set.Icc 0 1)

value:
fun (x : ℝ) => ((Real.arctan x + Real.pi / 2) / Real.pi, PG23MonocultureMatching.pg23ScoreNormalize._proof_2 x)

PG23MonocultureMatching.pg23NormalizedCutoff

Declaration location: paper-local

Lean declaration body

type:
{College : Type v} → (College → ℝ) → College → ↑(Set.Icc 0 1)

value:
fun {College : Type v} (P : College → ℝ) (a : College) => PG23MonocultureMatching.pg23ScoreNormalize (P a)

Direct retained dependencies

- PG23MonocultureMatching.pg23ScoreNormalize

PG23MonocultureMatching.pg23SourcePrefers

Declaration location: paper-local

Lean declaration body

type:
{College : Type v} →
  [inst : Fintype College] → PG23MonocultureMatching.PG23ApplicantType College → Option College → Option College → Prop

value:
fun {College : Type v} [Fintype College] (a : PG23MonocultureMatching.PG23ApplicantType College)
    (a_1 a_2 : Option College) =>
  AL16SupplyDemandMatching.al16RankPrefers PG23MonocultureMatching.pg23SourceRank a a_1 a_2

Direct retained dependencies

- AL16SupplyDemandMatching.al16RankPrefers
- PG23MonocultureMatching.PG23ApplicantType
- PG23MonocultureMatching.pg23SourceRank

PG23MonocultureMatching.pg23ActiveAffordableSet

Declaration location: paper-local

Lean declaration body

```
type:
{College : Type v} →
  [inst : Fintype College] →
    Finset College → (College → ℝ) → PG23MonocultureMatching.PG23ApplicantType College → Finset College

value:
fun {College : Type v} [Fintype College] (active : Finset College) (P : College → ℝ)
    (theta : PG23MonocultureMatching.PG23ApplicantType College) =>
  {c ∈ active | PG23MonocultureMatching.pg23Affordable P theta c}
```

Direct retained dependencies

- PG23MonocultureMatching.PG23ApplicantType
- PG23MonocultureMatching.pg23Affordable

PG23MonocultureMatching.pg23ActiveSourceChoice

Declaration location: paper-local

Lean declaration body

```
type:
{College : Type v} →
  [inst : Fintype College] →
    Finset College → (College → ℝ) → PG23MonocultureMatching.PG23ApplicantType College → Option College

value:
fun {College : Type v} [Fintype College] (active : Finset College) (P : College → ℝ)
    (a : PG23MonocultureMatching.PG23ApplicantType College) =>
  if hfeasible : (PG23MonocultureMatching.pg23ActiveAffordableSet active P a).Nonempty then
    some (Classical.choose (PG23MonocultureMatching.pg23ActiveSourceChoice._proof_1 active P a hfeasible))
  else none
```

Direct retained dependencies

- PG23MonocultureMatching.PG23ApplicantType
- PG23MonocultureMatching.pg23ActiveAffordableSet
- PG23MonocultureMatching.pg23SourceRank

PG23MonocultureMatching.pg23NormalizedScore

Declaration location: paper-local

Lean declaration body

```
type:
{College : Type v} →
  [inst : Fintype College] → PG23MonocultureMatching.PG23ApplicantType College → College → ↑(Set.Icc 0 1)

value:
fun {College : Type v} [Fintype College] (theta : PG23MonocultureMatching.PG23ApplicantType College) (c : College) =>
  PG23MonocultureMatching.pg23ScoreNormalize (PG23MonocultureMatching.pg23SourceScore theta c)
```

Direct retained dependencies

- PG23MonocultureMatching.PG23ApplicantType
- PG23MonocultureMatching.pg23ScoreNormalize
- PG23MonocultureMatching.pg23SourceScore

PG23MonocultureMatching.pg23NormalizedStudent

Declaration location: paper-local

Lean declaration body

```
type:
{College : Type v} →
  [inst : Fintype College] →
    PG23MonocultureMatching.PG23ApplicantType College → AL16SupplyDemandMatching.AL16SourceStudent College

value:
fun {College : Type v} [Fintype College] (theta : PG23MonocultureMatching.PG23ApplicantType College) =>
  (theta.1, PG23MonocultureMatching.pg23NormalizedScore theta)
```

Direct retained dependencies

- AL16SupplyDemandMatching.AL16Ranking
- AL16SupplyDemandMatching.AL16SourceStudent
- PG23MonocultureMatching.PG23ApplicantType
- PG23MonocultureMatching.PG23Ranking
- PG23MonocultureMatching.pg23NormalizedScore

Paper-specific semantic prerequisites

PG23MonocultureMatching.PG23Ranking

Paper-local declaration:
PG23MonocultureMatching.PG23Ranking
Source locator: cited publication, lines 2-34

Verbatim paper-source connection

```
\section{Model and Preliminaries}
Our model builds on the model of \cite{azevedo2016supply}, in which there is a continuum of applicants and a finite number of firms (analogously,
↪ colleges). An applicant is identified by their true \textit{value}, which is distributed on $\RR$ according to a probability measure $\eta$. Let $\colleges
↪ = \{1, 2, \cdots, \numcolleges\}$ be the set of all firms. Firms have total capacity $\totalsupply < 1$ (so there are fewer positions than applicants)
↪ and each firm has the same capacity $\frac{\totalsupply}{\numcolleges}.$

An applicant with value $v$ is associated with an independent random variable $\theta(v)$, characterizing their preferences over firms and firms'
↪ estimated values for that applicant. The realization of $\theta(v)$ is their \textit{type}, which lies in $\Theta$. $\Theta := \mathcal{R} \times
↪ \RR^\numcolleges$ is the set of applicant types, where $\mathcal{R}$ is the set of preference orderings over the firms. If $\theta(v) = \theta =
↪ (\succ^\theta, e^\theta),$ then $\succ^\theta$ is the preference ordering of $v$ over firms and $e^\theta_\college$ is the \textit{estimated value} of
↪ the applicant at firm $\college$.

In the model, $\theta(v)$ plays a central role. First, it is used to model the process of preference approximation for firms. Firms all prefer applicants with
↪ higher true values, but in practice, rank applicants based on the estimated values given by $\theta(v)$. Second, it connects our model with that of
↪ \cite{azevedo2016supply}. In particular, the distribution $\eta$ over values in $\RR$ together with an appropriate choice of $\theta(v)$ induces a
↪ distribution over applicant types, which, alongside specifications of firm capacities, fully determines the model in
↪ \cite{azevedo2016supply}.\footnote{Formally, consider the probability distribution $\eta'$ over $\Theta$ such that for $A\subseteq \Theta$,
\begin{equation}
    \eta'(A) = \int_\RR \Pr[\theta(v)\in A]\,d\eta(v).
\end{equation}}

\paragraph{}
A \textit{matching} is a function $\mu: \Theta \rightarrow \colleges\cup \{\emptyset\}$ that satisfies the capacity constraints
\begin{equation}
    \int_\RR \Pr[\mu(\theta(v)) = \college]\,d\eta(v) = \frac{\totalsupply}{\numcolleges},\qquad \forall \college\in \colleges,
\end{equation}
as well as two additional technical assumptions: that $\mu^{-1}(\college)$ is measurable and that the set $\{\theta:\mu(\theta)\prec^\theta \college\}$ is
↪ open.\footnote{We remark that matchings are not typically required to fill capacities (i.e., the equality above is typically an inequality), but in our
↪ setting, since there is limited total capacity and applicants all prefer being matched to being unmatched, we are only concerned with matchings that
↪ fill capacity.}

Then $\mu(\theta(v))$ is a random variable indicating where an applicant with value $v$ is matched, with $\mu(\theta(v)) = \emptyset$ indicating that the
↪ applicant is not matched. When it is clear what $\theta$ is, we will abuse notation to write $\mu(v)$ instead of $\mu(\theta(v))$ to denote the random
↪ variable representing where an applicant with value $v$ is matched. Notice that the matching of an applicant depends only on their random type,
↪ reflecting the fact that their matching outcome depends directly on the scores they receive at each firm rather than their true value.

A matching is \textit{stable} if there does not exist a pair $\theta\in \Theta, \college\in \colleges$ such that $\college\succ^\theta \mu(\theta)$, and
↪ $e^\theta_\college > e^{\theta'}_{\college}$ for some $\theta'\in \mu^{-1}(\college)$. In other words, there is no applicant-firm pair that would
↪ jointly prefer to defect from the current matching to be matched with each other.

\subsection{A cutoff characterization of stable matchings}
A benefit of this continuum model is that stable matchings are characterized by a tractable cutoff structure \citep{azevedo2016supply}. Given a vector of
↪ \textit{cutoffs} $P = (P_1, \cdots, P_\numcolleges),$ an applicant with type $\theta$ can \textit{afford} firm $\college$ if $e^\theta_\college \ge
↪ P_\college$. The applicant's \textit{demand} at $P$ is $D^\theta(P)$, the applicant's most preferred firm (according to $\succ^\theta$) among those
↪ they can afford. If they cannot afford any firm, $D^\theta(P)=\emptyset$.
The \textit{aggregate demand} of a firm $\college$ at $P$ is
\begin{equation}
    D^\college(P) := \int_\RR \Pr[D^{\theta(v)}(P) = \college]\,d\eta(v),
\end{equation}
the measure of applicants who demand it.

A vector of cutoffs $P$ is \textit{market clearing} if and only if
\begin{equation}
    D^\college(P) = \frac{\totalsupply}{\numcolleges},\qquad \forall \college\in \numcolleges.\footnote{Again, as in the above footnotes, the standard
    ↪ definition in \cite{azevedo2016supply} would allow for an inequality here if no applicant would prefer to take the empty spots of a firm. This is not
    ↪ possible in our model, so we simplify our definition.}
\end{equation}

[Next verbatim source excerpt]


\subsection{Instantiating monoculture and polyculture}
We instantiate our model by specifying $\theta(v)$ for monoculture and polyculture. In what follows, we consider a fixed \textit{noise distribution} $\DD$.

\paragraph{Monoculture.} In monoculture, we take the random variable
\begin{equation}
    \theta_{\mono}(v) := (\succ, (v + X, v + X, \cdots, v + X)),
\end{equation}
where $\succ$ is drawn uniformly at random from $\mathcal{R}$ and $X\sim \DD.$ In other words, applicants have preferences distributed uniformly at
↪ random, and each applicant has a single estimated value $v+X$ that is shared across firms. This noisy estimated value equals the applicant's value
↪ perturbed by noise drawn from $\DD$.

\paragraph{Polyculture.} In polyculture, we take the random variable
\begin{equation}
    \theta_{\poly}(v) := (\succ, (v + X_1, v + X_2, \cdots, v + X_\numcolleges)),
\end{equation}
where $\succ$ is drawn uniformly at random from $\mathcal{R}$ and $X_1,X_2,\cdots, X_\numcolleges \sim \DD$ are i.i.d.. Again, applicants have
↪ preferences distributed uniformly at random, but now each applicant has a distinct, independently drawn estimated value at each firm.

\paragraph{} For expository purposes, we instantiate our model with a fixed noise distribution $\DD$ shared across both monoculture and polyculture.
↪ However, as will often be obvious, this assumption is not necessary for many of our results. In some cases, (primarily, in Theorem 2) it does allow us
↪ to compare monoculture and polyculture \textit{ceteris paribus}.
```

Lean-expanded paper semantic target

type:
(College : Type v) → [Fintype College] → Type v

value:
fun (College : Type v) [Fintype College] => AL16SupplyDemandMatching.AL16Ranking College

Direct retained dependencies

- AL16SupplyDemandMatching.AL16Ranking

Recorded source-to-declaration screening **Verdict:** matches

Reason: A bijection from colleges to Fin(cardinality) encodes a strict complete ranking, matching the source ranking coordinate and every displayed preference and application use.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

PG23MonocultureMatching.PG23ApplicantType

Paper-local declaration:
PG23MonocultureMatching.PG23ApplicantType
Source locator: cited publication, lines 2-34

Verbatim paper-source connection

```
\section{Model and Preliminaries}
Our model builds on the model of \cite{azevedo2016supply}, in which there is a continuum of applicants and a finite number of firms (analogously,
↪ colleges). An applicant is identified by their true \textit{value}, which is distributed on $\RR$ according to a probability measure $\eta$. Let $\colleges
↪ = \{1, 2, \cdots, \numcolleges\}$ be the set of all firms. Firms have total capacity $\totalsupply < 1$ (so there are fewer positions than applicants)
↪ and each firm has the same capacity $\frac{\totalsupply}{\numcolleges}.$

An applicant with value $v$ is associated with an independent random variable $\theta(v)$, characterizing their preferences over firms and firms'
↪ estimated values for that applicant. The realization of $\theta(v)$ is their \textit{type}, which lies in $\Theta$. $\Theta := \mathcal{R} \times
↪ \RR^\numcolleges$ is the set of applicant types, where $\mathcal{R}$ is the set of preference orderings over the firms. If $\theta(v) = \theta =
↪ (\succ^\theta, e^\theta),$ then $\succ^\theta$ is the preference ordering of $v$ over firms and $e^\theta_\college$ is the \textit{estimated value} of
↪ the applicant at firm $\college$.

In the model, $\theta(v)$ plays a central role. First, it is used to model the process of preference approximation for firms. Firms all prefer applicants with
↪ higher true values, but in practice, rank applicants based on the estimated values given by $\theta(v)$. Second, it connects our model with that of
↪ \cite{azevedo2016supply}. In particular, the distribution $\eta$ over values in $\RR$ together with an appropriate choice of $\theta(v)$ induces a
↪ distribution over applicant types, which, alongside specifications of firm capacities, fully determines the model in
↪ \cite{azevedo2016supply}.\footnote{Formally, consider the probability distribution $\eta'$ over $\Theta$ such that for $A\subseteq \Theta$,
\begin{equation}
    \eta'(A) = \int_\RR \Pr[\theta(v)\in A]\,d\eta(v).
\end{equation}}

\paragraph{}
A \textit{matching} is a function $\mu: \Theta \rightarrow \colleges\cup \{\emptyset\}$ that satisfies the capacity constraints
\begin{equation}
    \int_\RR \Pr[\mu(\theta(v)) = \college]\,d\eta(v) = \frac{\totalsupply}{\numcolleges},\qquad \forall \college\in \colleges,
\end{equation}
as well as two additional technical assumptions: that $\mu^{-1}(\college)$ is measurable and that the set $\{\theta:\mu(\theta)\prec^\theta \college\}$ is
↪ open.\footnote{We remark that matchings are not typically required to fill capacities (i.e., the equality above is typically an inequality), but in our
↪ setting, since there is limited total capacity and applicants all prefer being matched to being unmatched, we are only concerned with matchings that
↪ fill capacity.}

Then $\mu(\theta(v))$ is a random variable indicating where an applicant with value $v$ is matched, with $\mu(\theta(v)) = \emptyset$ indicating that the
↪ applicant is not matched. When it is clear what $\theta$ is, we will abuse notation to write $\mu(v)$ instead of $\mu(\theta(v))$ to denote the random
↪ variable representing where an applicant with value $v$ is matched. Notice that the matching of an applicant depends only on their random type,
↪ reflecting the fact that their matching outcome depends directly on the scores they receive at each firm rather than their true value.

A matching is \textit{stable} if there does not exist a pair $\theta\in \Theta, \college\in \colleges$ such that $\college\succ^\theta \mu(\theta)$, and
↪ $e^\theta_\college > e^{\theta'}_{\college}$ for some $\theta'\in \mu^{-1}(\college)$. In other words, there is no applicant-firm pair that would
↪ jointly prefer to defect from the current matching to be matched with each other.

\subsection{A cutoff characterization of stable matchings}
A benefit of this continuum model is that stable matchings are characterized by a tractable cutoff structure \citep{azevedo2016supply}. Given a vector of
↪ \textit{cutoffs} $P = (P_1, \cdots, P_\numcolleges),$ an applicant with type $\theta$ can \textit{afford} firm $\college$ if $e^\theta_\college \ge
↪ P_\college$. The applicant's \textit{demand} at $P$ is $D^\theta(P)$, the applicant's most preferred firm (according to $\succ^\theta$) among those
↪ they can afford. If they cannot afford any firm, $D^\theta(P)=\emptyset$.
The \textit{aggregate demand} of a firm $\college$ at $P$ is
\begin{equation}
    D^\college(P) := \int_\RR \Pr[D^{\theta(v)}(P) = \college]\,d\eta(v),
\end{equation}
the measure of applicants who demand it.

A vector of cutoffs $P$ is \textit{market clearing} if and only if
\begin{equation}
    D^\college(P) = \frac{\totalsupply}{\numcolleges},\qquad \forall \college\in \numcolleges.\footnote{Again, as in the above footnotes, the standard
    ↪ definition in \cite{azevedo2016supply} would allow for an inequality here if no applicant would prefer to take the empty spots of a firm. This is not
    ↪ possible in our model, so we simplify our definition.}
\end{equation}

[Next verbatim source excerpt]


\subsection{Instantiating monoculture and polyculture}
We instantiate our model by specifying $\theta(v)$ for monoculture and polyculture. In what follows, we consider a fixed \textit{noise distribution} $\DD$.

\paragraph{Monoculture.} In monoculture, we take the random variable
\begin{equation}
    \theta_{\mono}(v) := (\succ, (v + X, v + X, \cdots, v + X)),
\end{equation}
where $\succ$ is drawn uniformly at random from $\mathcal{R}$ and $X\sim \DD.$ In other words, applicants have preferences distributed uniformly at
↪ random, and each applicant has a single estimated value $v+X$ that is shared across firms. This noisy estimated value equals the applicant's value
↪ perturbed by noise drawn from $\DD$.

\paragraph{Polyculture.} In polyculture, we take the random variable
\begin{equation}
    \theta_{\poly}(v) := (\succ, (v + X_1, v + X_2, \cdots, v + X_\numcolleges)),
\end{equation}
where $\succ$ is drawn uniformly at random from $\mathcal{R}$ and $X_1,X_2,\cdots, X_\numcolleges \sim \DD$ are i.i.d.. Again, applicants have
↪ preferences distributed uniformly at random, but now each applicant has a distinct, independently drawn estimated value at each firm.

\paragraph{} For expository purposes, we instantiate our model with a fixed noise distribution $\DD$ shared across both monoculture and polyculture.
↪ However, as will often be obvious, this assumption is not necessary for many of our results. In some cases, (primarily, in Theorem 2) it does allow us
↪ to compare monoculture and polyculture \textit{ceteris paribus}.
```

Lean-expanded paper semantic target

type:
(College : Type v) → [Fintype College] → Type v

value:
fun (College : Type v) [Fintype College] => PG23MonocultureMatching.PG23Ranking College × (College → ℝ)

Direct retained dependencies

- PG23MonocultureMatching.PG23Ranking

Recorded source-to-declaration screening **Verdict:** matches

Reason: The product of a strict college ranking and a real-valued score vector is exactly the applicant type in the source model, and the displayed market and result uses preserve both coordinates.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

PG23MonocultureMatching.PG23DifferentialApplicantType

Paper-local declaration:
PG23MonocultureMatching.PG23DifferentialApplicantType
Source locator: cited publication, lines 641-641

Verbatim paper-source connection

We now consider when different applicants can apply to different numbers of firms (analogously, colleges)---which we refer to as \textit{differential application access}. In particular, we assume some discrete probability distribution κ over $\{1, 2, \cdots, \numcolleges\}$ that determines the number of firms an applicant can apply to. In particular, an applicant with value v can apply to $k(v)$ firms, where $k(v)$ is drawn independently from κ. All other aspects of our model remain the same.

Lean-expanded paper semantic target

type:
(College : Type v) → [Fintype College] → ℕ → Type (max 0 v)

value:
fun (College : Type v) [Fintype College] (n : ℕ) => Fin n × PG23MonocultureMatching.PG23ApplicantType College

Direct retained dependencies

- PG23MonocultureMatching.PG23ApplicantType

Recorded source-to-declaration screening **Verdict:** matches

Reason: Pairing a base applicant type with a Fin n position represents the source access count by kappa = position + 1; the displayed differential-access definitions and lemma use this translation consistently.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

PG23MonocultureMatching.PG23Matching

Paper-local declaration:
PG23MonocultureMatching.PG23Matching
Source locator: cited publication, lines 2-34

Verbatim paper-source connection

```
\section{Model and Preliminaries}
Our model builds on the model of \cite{azevedo2016supply}, in which there is a continuum of applicants and a finite number of firms (analogously,
↪ colleges). An applicant is identified by their true \textit{value}, which is distributed on $\RR$ according to a probability measure $\eta$. Let $\colleges
↪ = \{1, 2, \cdots, \numcolleges\}$ be the set of all firms. Firms have total capacity $\totalsupply < 1$ (so there are fewer positions than applicants)
↪ and each firm has the same capacity $\frac{\totalsupply}{\numcolleges}.$

An applicant with value $v$ is associated with an independent random variable $\theta(v)$, characterizing their preferences over firms and firms'
↪ estimated values for that applicant. The realization of $\theta(v)$ is their \textit{type}, which lies in $\Theta$. $\Theta := \mathcal{R} \times
↪ \RR^\numcolleges$ is the set of applicant types, where $\mathcal{R}$ is the set of preference orderings over the firms. If $\theta(v) = \theta =
↪ (\succ^\theta, e^\theta),$ then $\succ^\theta$ is the preference ordering of $v$ over firms and $e^\theta_\college$ is the \textit{estimated value} of
↪ the applicant at firm $\college$.

In the model, $\theta(v)$ plays a central role. First, it is used to model the process of preference approximation for firms. Firms all prefer applicants with
↪ higher true values, but in practice, rank applicants based on the estimated values given by $\theta(v)$. Second, it connects our model with that of
↪ \cite{azevedo2016supply}. In particular, the distribution $\eta$ over values in $\RR$ together with an appropriate choice of $\theta(v)$ induces a
↪ distribution over applicant types, which, alongside specifications of firm capacities, fully determines the model in
↪ \cite{azevedo2016supply}.\footnote{Formally, consider the probability distribution $\eta'$ over $\Theta$ such that for $A\subseteq \Theta$,
\begin{equation}
    \eta'(A) = \int_\RR \Pr[\theta(v)\in A]\,d\eta(v).
\end{equation}}

\paragraph{}
A \textit{matching} is a function $\mu: \Theta \rightarrow \colleges\cup \{\emptyset\}$ that satisfies the capacity constraints
\begin{equation}
    \int_\RR \Pr[\mu(\theta(v)) = \college]\,d\eta(v) = \frac{\totalsupply}{\numcolleges},\qquad \forall \college\in \colleges,
\end{equation}
as well as two additional technical assumptions: that $\mu^{-1}(\college)$ is measurable and that the set $\{\theta:\mu(\theta)\prec^\theta \college\}$ is
↪ open.\footnote{We remark that matchings are not typically required to fill capacities (i.e., the equality above is typically an inequality), but in our
↪ setting, since there is limited total capacity and applicants all prefer being matched to being unmatched, we are only concerned with matchings that
↪ fill capacity.}

Then $\mu(\theta(v))$ is a random variable indicating where an applicant with value $v$ is matched, with $\mu(\theta(v)) = \emptyset$ indicating that the
↪ applicant is not matched. When it is clear what $\theta$ is, we will abuse notation to write $\mu(v)$ instead of $\mu(\theta(v))$ to denote the random
↪ variable representing where an applicant with value $v$ is matched. Notice that the matching of an applicant depends only on their random type,
↪ reflecting the fact that their matching outcome depends directly on the scores they receive at each firm rather than their true value.

A matching is \textit{stable} if there does not exist a pair $\theta\in \Theta, \college\in \colleges$ such that $\college\succ^\theta \mu(\theta)$, and
↪ $e^\theta_\college > e^{\theta'}_{\college}$ for some $\theta'\in \mu^{-1}(\college)$. In other words, there is no applicant-firm pair that would
↪ jointly prefer to defect from the current matching to be matched with each other.

\subsection{A cutoff characterization of stable matchings}
A benefit of this continuum model is that stable matchings are characterized by a tractable cutoff structure \citep{azevedo2016supply}. Given a vector of
↪ \textit{cutoffs} $P = (P_1, \cdots, P_\numcolleges),$ an applicant with type $\theta$ can \textit{afford} firm $\college$ if $e^\theta_\college \ge
↪ P_\college$. The applicant's \textit{demand} at $P$ is $D^\theta(P)$, the applicant's most preferred firm (according to $\succ^\theta$) among those
↪ they can afford. If they cannot afford any firm, $D^\theta(P)=\emptyset$.
The \textit{aggregate demand} of a firm $\college$ at $P$ is
\begin{equation}
    D^\college(P) := \int_\RR \Pr[D^{\theta(v)}(P) = \college]\,d\eta(v),
\end{equation}
the measure of applicants who demand it.

A vector of cutoffs $P$ is \textit{market clearing} if and only if
\begin{equation}
    D^\college(P) = \frac{\totalsupply}{\numcolleges},\qquad \forall \college\in \numcolleges.\footnote{Again, as in the above footnotes, the standard
    ↪ definition in \cite{azevedo2016supply} would allow for an inequality here if no applicant would prefer to take the empty spots of a firm. This is not
    ↪ possible in our model, so we simplify our definition.}
\end{equation}

[Next verbatim source excerpt]


\subsection{Instantiating monoculture and polyculture}
We instantiate our model by specifying $\theta(v)$ for monoculture and polyculture. In what follows, we consider a fixed \textit{noise distribution} $\DD$.

\paragraph{Monoculture.} In monoculture, we take the random variable
\begin{equation}
    \theta_{\mono}(v) := (\succ, (v + X, v + X, \cdots, v + X)),
\end{equation}
where $\succ$ is drawn uniformly at random from $\mathcal{R}$ and $X\sim \DD.$ In other words, applicants have preferences distributed uniformly at
↪ random, and each applicant has a single estimated value $v+X$ that is shared across firms. This noisy estimated value equals the applicant's value
↪ perturbed by noise drawn from $\DD$.

\paragraph{Polyculture.} In polyculture, we take the random variable
\begin{equation}
    \theta_{\poly}(v) := (\succ, (v + X_1, v + X_2, \cdots, v + X_\numcolleges)),
\end{equation}
where $\succ$ is drawn uniformly at random from $\mathcal{R}$ and $X_1,X_2,\cdots, X_\numcolleges \sim \DD$ are i.i.d.. Again, applicants have
↪ preferences distributed uniformly at random, but now each applicant has a distinct, independently drawn estimated value at each firm.

\paragraph{} For expository purposes, we instantiate our model with a fixed noise distribution $\DD$ shared across both monoculture and polyculture.
↪ However, as will often be obvious, this assumption is not necessary for many of our results. In some cases, (primarily, in Theorem 2) it does allow us
↪ to compare monoculture and polyculture \textit{ceteris paribus}.
```

Lean-expanded paper semantic target

type:
(College : Type v) → [Fintype College] → Type v

value:
fun (College : Type v) [Fintype College] => PG23MonocultureMatching.PG23ApplicantType College → Option College

Direct retained dependencies

- PG23MonocultureMatching.PG23ApplicantType

Recorded source-to-declaration screening **Verdict:** matches

Reason: An optional college exactly represents the source matching outcome of a college assignment or being unmatched, and the displayed feasibility, blocking, and stability uses keep that meaning.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

PG23MonocultureMatching.definition1_maximumOrderStatistic

Paper-local declaration:
PG23MonocultureMatching.definition1_maximumOrderStatistic
Source locator: cited publication, lines 112-118

Verbatim paper-source connection

```
\begin{definition}[Maximum Order Statistic]
   The \textbf{maximum order statistic} of $n$ draws from a distribution $\DD$ is defined as the random variable
   \begin{equation}
      X^{(n)} := \max \{X_1, X_2, \cdots, X_n\}
   \end{equation}
   where $X_1, \cdots, X_n\simiid \DD.$
\end{definition}
```

Lean-expanded paper semantic target

```
type:
{m : ℕ} → [NeZero m] → (Fin m → ℝ) → ℝ

value:
fun {m : ℕ} [NeZero m] (noise : Fin m → ℝ) =>
  AppliedModelingLib.Probability.upperOrderStatistic noise AppliedModelingLib.Matching.topSampleRank
```

Direct retained dependencies

- AppliedModelingLib.Matching.topSampleRank
- AppliedModelingLib.Probability.upperOrderStatistic

Recorded source-to-declaration screening **Verdict:** matches

Reason: Selecting the top sample rank in the upper-order-statistic convention returns the maximum of the finite sample, exactly the source maximum order statistic.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

PG23MonocultureMatching.pg23SourceScore

Paper-local declaration:
PG23MonocultureMatching.pg23SourceScore
Source locator: cited publication, lines 2-34

Verbatim paper-source connection

```
\section{Model and Preliminaries}
Our model builds on the model of \cite{azevedo2016supply}, in which there is a continuum of applicants and a finite number of firms (analogously,
↪ colleges). An applicant is identified by their true \textit{value}, which is distributed on $\RR$ according to a probability measure $\eta$. Let $\colleges
↪ = \{1, 2, \cdots, \numcolleges\}$ be the set of all firms. Firms have total capacity $\totalsupply < 1$ (so there are fewer positions than applicants)
↪ and each firm has the same capacity $\frac{\totalsupply}{\numcolleges}.$

An applicant with value $v$ is associated with an independent random variable $\theta(v)$, characterizing their preferences over firms and firms'
↪ estimated values for that applicant. The realization of $\theta(v)$ is their \textit{type}, which lies in $\Theta$. $\Theta := \mathcal{R} \times
↪ \RR^\numcolleges$ is the set of applicant types, where $\mathcal{R}$ is the set of preference orderings over the firms. If $\theta(v) = \theta =
↪ (\succ^\theta, e^\theta),$ then $\succ^\theta$ is the preference ordering of $v$ over firms and $e^\theta_\college$ is the \textit{estimated value} of
↪ the applicant at firm $\college$.

In the model, $\theta(v)$ plays a central role. First, it is used to model the process of preference approximation for firms. Firms all prefer applicants with
↪ higher true values, but in practice, rank applicants based on the estimated values given by $\theta(v)$. Second, it connects our model with that of
↪ \cite{azevedo2016supply}. In particular, the distribution $\eta$ over values in $\RR$ together with an appropriate choice of $\theta(v)$ induces a
↪ distribution over applicant types, which, alongside specifications of firm capacities, fully determines the model in
↪ \cite{azevedo2016supply}.\footnote{Formally, consider the probability distribution $\eta'$ over $\Theta$ such that for $A\subseteq \Theta$,
\begin{equation}
    \eta'(A) = \int_\RR \Pr[\theta(v)\in A]\,d\eta(v).
\end{equation}}

\paragraph{}
A \textit{matching} is a function $\mu: \Theta \rightarrow \colleges\cup \{\emptyset\}$ that satisfies the capacity constraints
\begin{equation}
    \int_\RR \Pr[\mu(\theta(v)) = \college]\,d\eta(v) = \frac{\totalsupply}{\numcolleges},\qquad \forall \college\in \colleges,
\end{equation}
as well as two additional technical assumptions: that $\mu^{-1}(\college)$ is measurable and that the set $\{\theta:\mu(\theta)\prec^\theta \college\}$ is
↪ open.\footnote{We remark that matchings are not typically required to fill capacities (i.e., the equality above is typically an inequality), but in our
↪ setting, since there is limited total capacity and applicants all prefer being matched to being unmatched, we are only concerned with matchings that
↪ fill capacity.}

Then $\mu(\theta(v))$ is a random variable indicating where an applicant with value $v$ is matched, with $\mu(\theta(v)) = \emptyset$ indicating that the
↪ applicant is not matched. When it is clear what $\theta$ is, we will abuse notation to write $\mu(v)$ instead of $\mu(\theta(v))$ to denote the random
↪ variable representing where an applicant with value $v$ is matched. Notice that the matching of an applicant depends only on their random type,
↪ reflecting the fact that their matching outcome depends directly on the scores they receive at each firm rather than their true value.

A matching is \textit{stable} if there does not exist a pair $\theta\in \Theta, \college\in \colleges$ such that $\college\succ^\theta \mu(\theta)$, and
↪ $e^\theta_\college > e^{\theta'}_{\college}$ for some $\theta'\in \mu^{-1}(\college)$. In other words, there is no applicant-firm pair that would
↪ jointly prefer to defect from the current matching to be matched with each other.

\subsection{A cutoff characterization of stable matchings}
A benefit of this continuum model is that stable matchings are characterized by a tractable cutoff structure \citep{azevedo2016supply}. Given a vector of
↪ \textit{cutoffs} $P = (P_1, \cdots, P_\numcolleges),$ an applicant with type $\theta$ can \textit{afford} firm $\college$ if $e^\theta_\college \ge
↪ P_\college$. The applicant's \textit{demand} at $P$ is $D^\theta(P)$, the applicant's most preferred firm (according to $\succ^\theta$) among those
↪ they can afford. If they cannot afford any firm, $D^\theta(P)=\emptyset$.
The \textit{aggregate demand} of a firm $\college$ at $P$ is
\begin{equation}
    D^\college(P) := \int_\RR \Pr[D^{\theta(v)}(P) = \college]\,d\eta(v),
\end{equation}
the measure of applicants who demand it.

A vector of cutoffs $P$ is \textit{market clearing} if and only if
\begin{equation}
    D^\college(P) = \frac{\totalsupply}{\numcolleges},\qquad \forall \college\in \numcolleges.\footnote{Again, as in the above footnotes, the standard
    ↪ definition in \cite{azevedo2016supply} would allow for an inequality here if no applicant would prefer to take the empty spots of a firm. This is not
    ↪ possible in our model, so we simplify our definition.}
\end{equation}

[Next verbatim source excerpt]


\subsection{Instantiating monoculture and polyculture}
We instantiate our model by specifying $\theta(v)$ for monoculture and polyculture. In what follows, we consider a fixed \textit{noise distribution} $\DD$.

\paragraph{Monoculture.} In monoculture, we take the random variable
\begin{equation}
    \theta_{\mono}(v) := (\succ, (v + X, v + X, \cdots, v + X)),
\end{equation}
where $\succ$ is drawn uniformly at random from $\mathcal{R}$ and $X\sim \DD.$ In other words, applicants have preferences distributed uniformly at
↪ random, and each applicant has a single estimated value $v+X$ that is shared across firms. This noisy estimated value equals the applicant's value
↪ perturbed by noise drawn from $\DD$.

\paragraph{Polyculture.} In polyculture, we take the random variable
\begin{equation}
    \theta_{\poly}(v) := (\succ, (v + X_1, v + X_2, \cdots, v + X_\numcolleges)),
\end{equation}
where $\succ$ is drawn uniformly at random from $\mathcal{R}$ and $X_1,X_2,\cdots, X_\numcolleges \sim \DD$ are i.i.d.. Again, applicants have
↪ preferences distributed uniformly at random, but now each applicant has a distinct, independently drawn estimated value at each firm.

\paragraph{} For expository purposes, we instantiate our model with a fixed noise distribution $\DD$ shared across both monoculture and polyculture.
↪ However, as will often be obvious, this assumption is not necessary for many of our results. In some cases, (primarily, in Theorem 2) it does allow us
↪ to compare monoculture and polyculture \textit{ceteris paribus}.
```

Lean-expanded paper semantic target

```
type:
{College : Type v} → [inst : Fintype College] → PG23MonocultureMatching.PG23ApplicantType College → College → ℝ

value:
fun {College : Type v} [Fintype College] (theta : PG23MonocultureMatching.PG23ApplicantType College) (c : College) =>
  theta.2 c
```

Direct retained dependencies

- PG23MonocultureMatching.PG23ApplicantType
- PG23MonocultureMatching.PG23Ranking

Recorded source-to-declaration screening **Verdict:** matches

Reason: Projecting the college coordinate from the applicant's score vector is exactly the source score function.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

PG23MonocultureMatching.pg23Affordable

Paper-local declaration:
PG23MonocultureMatching.pg23Affordable
Source locator: cited publication, lines 24-34

Verbatim paper-source connection

```
A benefit of this continuum model is that stable matchings are characterized by a tractable cutoff structure \citep{azevedo2016supply}. Given a vector of
↪  \textit{cutoffs} $P = (P_1, \cdots, P_\numcolleges),$ an applicant with type $\theta$ can \textit{afford} firm $\college$ if $e^\theta_\college \ge
↪  P_\college$. The applicant's \textit{demand} at $P$ is $D^\theta(P)$, the applicant's most preferred firm (according to $\succ^\theta$) among those
↪  they can afford. If they cannot afford any firm, $D^\theta(P)=\emptyset$.
The \textit{aggregate demand} of a firm $\college$ at $P$ is
\begin{equation}
   D^\college(P) := \int_\RR \Pr[D^{\theta(v)}(P) = \college]\,d\eta(v),
\end{equation}
the measure of applicants who demand it.

A vector of cutoffs $P$ is \textit{market clearing} if and only if
\begin{equation}
   D^\college(P) = \frac{\totalsupply}{\numcolleges},\qquad \forall \college\in \numcolleges.\footnote{Again, as in the above footnotes, the standard
   ↪  definition in \cite{azevedo2016supply} would allow for an inequality here if no applicant would prefer to take the empty spots of a firm. This is not
   ↪  possible in our model, so we simplify our definition.}
\end{equation}
```

Lean-expanded paper semantic target

```
type:
{College : Type v} →
  [inst : Fintype College] → (College → ℝ) → PG23MonocultureMatching.PG23ApplicantType College → College → Prop

value:
fun {College : Type v} [Fintype College] (P : College → ℝ) (theta : PG23MonocultureMatching.PG23ApplicantType College)
    (c : College) =>
  P c ≤ PG23MonocultureMatching.pg23SourceScore theta c
```

Direct retained dependencies

- PG23MonocultureMatching.PG23ApplicantType
- PG23MonocultureMatching.pg23SourceScore

Recorded source-to-declaration screening **Verdict:** matches

Reason: The predicate cutoff(c) <= score(theta,c) is exactly the source weak affordability condition and is used with that inequality in the displayed choice events.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

PG23MonocultureMatching.pg23ConnectedSupport

Paper-local declaration:
PG23MonocultureMatching.pg23ConnectedSupport
Source locator: cited publication, lines 66-66

Verbatim paper-source connection

\paragraph{A technical assumption on η and $\DD$.} We will make a basic assumption on the probability measure η and the probability distribution $\DD$. Let π be the probability measure associated with $\DD$. Then we assume that both η and π have \textit{connected support}, where we mean that the smallest closed set of measure 1 is an interval. Let these respective intervals be $[V_-, V_+]$ and $[X_-, X_+]$. The assumption of connected support means that there are no ``gaps'' in either distribution. This is theoretically convenient, since it ensures that there is a unique stable matching.

Lean-expanded paper semantic target

type:
MeasureTheory.Measure ℝ → Prop

value:
fun (law : MeasureTheory.Measure ℝ) => IsPreconnected law.support

Recorded source-to-declaration screening **Verdict:** matches

Reason: IsPreconnected on the measure support is the standard real-line formulation that the source support is connected, and all displayed applications use it on the relevant value or noise law.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

PG23MonocultureMatching.pg23TopKApplicationSet

Paper-local declaration:
PG23MonocultureMatching.pg23TopKApplicationSet
Source locator: cited publication, lines 649-649

Verbatim paper-source connection

We will focus on the strategy profile S such that $S_k(v, \succ)$ gives the k most preferred firms according to $\succ$. Therefore, strategies do not depend at all on v. We will later show that this intuitive set of strategies forms a Nash equilibrium. Fixing this strategy profile S, we can now define $\theta(v)$ for monoculture and polyculture in the presence of differential application access.

Lean-expanded paper semantic target

type:
```
{College : Type v} → [inst : Fintype College] → ℕ → PG23MonocultureMatching.PG23Ranking College → Finset College
```

value:
```
fun {College : Type v} [Fintype College] (k : ℕ) (ranking : PG23MonocultureMatching.PG23Ranking College) =>
  {c : College | ↑(ranking c) < k}
```

Direct retained dependencies

- PG23MonocultureMatching.PG23Ranking

Recorded source-to-declaration screening **Verdict:** matches

Reason: The set of colleges with zero-based position less than k is exactly the source set of the applicant’s k highest-ranked colleges.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

PG23MonocultureMatching.pg23DifferentialActiveSet

Paper-local declaration:
PG23MonocultureMatching.pg23DifferentialActiveSet
Source locator: cited publication, lines 651-663

Verbatim paper-source connection

```
\paragraph{Monoculture with differential application access.} Define
\begin{equation}
   \theta_{\mono, \kappa}(v) = (\succ, e)
\end{equation}
where
\begin{equation}
   e_c =
   \begin{cases}
      v + X &\quad \college\in S_{k(v)}(v, \succ)\\
      -\infty &\quad \college\notin S_{k(v)}(v, \succ)
   \end{cases},
\end{equation}
for $\succ$ drawn uniformly at random from $\mathcal{R}$, $k(v)$ drawn from $\kappa,$ and $X$ drawn from $\DD$. As before, an applicant with value
↪  $v$ is given a random preference ordering over firms. However, now the applicant's estimated values across firms depends also on a random draw
↪  from $\kappa$, which determines how many firms they can apply to. Recall that $e$ gives the vector of estimated values of an applicant across firms.
↪  Therefore, under monoculture, this estimated value is shared across the firms the applicant applies to (which are the applicant's top-$k(v)$ most
↪  preferred firms), but is $-\infty$ at other firms (meaning that the applicant will never be matched to firms they do not apply to).
```

Lean-expanded paper semantic target

```
type:
{College : Type v} →
  [inst : Fintype College] → {n : ℕ} → PG23MonocultureMatching.PG23DifferentialApplicantType College n → Finset College

value:
fun {College : Type v} [Fintype College] {n : ℕ}
    (theta : PG23MonocultureMatching.PG23DifferentialApplicantType College n) =>
  PG23MonocultureMatching.pg23TopKApplicationSet (↑theta.1 + 1) theta.2.1
```

Direct retained dependencies

- PG23MonocultureMatching.PG23ApplicantType
- PG23MonocultureMatching.PG23DifferentialApplicantType
- PG23MonocultureMatching.PG23Ranking
- PG23MonocultureMatching.pg23TopKApplicationSet

Recorded source-to-declaration screening **Verdict:** matches

Reason: The active set is the top-(position + 1) application set, exactly implementing access to the source applicant's kappa highest-ranked colleges under the zero-based Fin representation.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

PG23MonocultureMatching.pg23DifferentialSourceChoice

Paper-local declaration:
PG23MonocultureMatching.pg23DifferentialSourceChoice
Source locator: cited publication, lines 651-663

Verbatim paper-source connection

```
\paragraph{Monoculture with differential application access.} Define
\begin{equation}
   \theta_{\mono, \kappa}(v) = (\succ, e)
\end{equation}
where
\begin{equation}
   e_c =
   \begin{cases}
      v + X &\quad \college\in S_{k(v)}(v, \succ)\\
      -\infty &\quad \college\notin S_{k(v)}(v, \succ)
   \end{cases},
\end{equation}
for $\succ$ drawn uniformly at random from $\mathcal{R}$, $k(v)$ drawn from $\kappa,$ and $X$ drawn from $\DD$. As before, an applicant with value
↪  $v$ is given a random preference ordering over firms. However, now the applicant's estimated values across firms depends also on a random draw
↪  from $\kappa$, which determines how many firms they can apply to. Recall that $e$ gives the vector of estimated values of an applicant across firms.
↪  Therefore, under monoculture, this estimated value is shared across the firms the applicant applies to (which are the applicant's top-$k(v)$ most
↪  preferred firms), but is $-\infty$ at other firms (meaning that the applicant will never be matched to firms they do not apply to).
```

Lean-expanded paper semantic target

```
type:
{College : Type v} →
  [inst : Fintype College] →
    {n : ℕ} → (College → ℝ) → PG23MonocultureMatching.PG23DifferentialApplicantType College n → Option College

value:
fun {College : Type v} [Fintype College] {n : ℕ} (P : College → ℝ)
    (a : PG23MonocultureMatching.PG23DifferentialApplicantType College n) =>
  PG23MonocultureMatching.pg23ActiveSourceChoice (PG23MonocultureMatching.pg23DifferentialActiveSet a) P a.2
```

Direct retained dependencies

- PG23MonocultureMatching.PG23ApplicantType
- PG23MonocultureMatching.PG23DifferentialApplicantType
- PG23MonocultureMatching.pg23ActiveSourceChoice
- PG23MonocultureMatching.pg23DifferentialActiveSet

Recorded source-to-declaration screening **Verdict:** matches

Reason: Choosing the favorite affordable college inside the applicant’s active top-kappa set implements the source rule in which non-applied-to colleges have score minus infinity.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

PG23MonocultureMatching.pg23DifferentialSourceAggregateDemand

Paper-local declaration:
PG23MonocultureMatching.pg23DifferentialSourceAggregateDemand
Source locator: cited publication, lines 651-663

Verbatim paper-source connection

```
\paragraph{Monoculture with differential application access.} Define
\begin{equation}
   \theta_{\mono, \kappa}(v) = (\succ, e)
\end{equation}
where
\begin{equation}
   e_c =
   \begin{cases}
      v + X &\quad \college\in S_{k(v)}(v, \succ)\\
      -\infty &\quad \college\notin S_{k(v)}(v, \succ)
   \end{cases},
\end{equation}
for $\succ$ drawn uniformly at random from $\mathcal{R}$, $k(v)$ drawn from $\kappa,$ and $X$ drawn from $\DD$. As before, an applicant with value
↪  $v$ is given a random preference ordering over firms. However, now the applicant's estimated values across firms depends also on a random draw
↪  from $\kappa$, which determines how many firms they can apply to. Recall that $e$ gives the vector of estimated values of an applicant across firms.
↪  Therefore, under monoculture, this estimated value is shared across the firms the applicant applies to (which are the applicant's top-$k(v)$ most
↪  preferred firms), but is $-\infty$ at other firms (meaning that the applicant will never be matched to firms they do not apply to).
```

Lean-expanded paper semantic target

```
type:
{College : Type v} →
  [inst : Fintype College] →
    {n : ℕ} →
      MeasureTheory.Measure (PG23MonocultureMatching.PG23DifferentialApplicantType College n) →
        (College → ℝ) → College → ℝ

value:
fun {College : Type v} [Fintype College] {n : ℕ}
    (typeLaw : MeasureTheory.Measure (PG23MonocultureMatching.PG23DifferentialApplicantType College n))
    (P : College → ℝ) (c : College) =>
  typeLaw.real
    {theta : PG23MonocultureMatching.PG23DifferentialApplicantType College n |
      PG23MonocultureMatching.pg23DifferentialSourceChoice P theta = some c}
```

Direct retained dependencies

- AL16SupplyDemandMatching.instMeasurableSpaceAL16Ranking
- PG23MonocultureMatching.PG23ApplicantType
- PG23MonocultureMatching.PG23DifferentialApplicantType
- PG23MonocultureMatching.PG23Ranking
- PG23MonocultureMatching.pg23DifferentialSourceChoice

Recorded source-to-declaration screening **Verdict:** matches

Reason: The measure of the fiber where differential source choice equals a college is exactly aggregate demand at that college in the source differential-access market.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

PG23MonocultureMatching.pg23DifferentialSourceMarketClearing

Paper-local declaration:
PG23MonocultureMatching.pg23DifferentialSourceMarketClearing
Source locator: cited publication, lines 651-663

Verbatim paper-source connection

```
\paragraph{Monoculture with differential application access.} Define
\begin{equation}
   \theta_{\mono, \kappa}(v) = (\succ, e)
\end{equation}
where
\begin{equation}
   e_c =
   \begin{cases}
      v + X &\quad \college\in S_{k(v)}(v, \succ)\\
      -\infty &\quad \college\notin S_{k(v)}(v, \succ)
   \end{cases},
\end{equation}
for $\succ$ drawn uniformly at random from $\mathcal{R}$, $k(v)$ drawn from $\kappa,$ and $X$ drawn from $\DD$. As before, an applicant with value
↪  $v$ is given a random preference ordering over firms. However, now the applicant's estimated values across firms depends also on a random draw
↪  from $\kappa$, which determines how many firms they can apply to. Recall that $e$ gives the vector of estimated values of an applicant across firms.
↪  Therefore, under monoculture, this estimated value is shared across the firms the applicant applies to (which are the applicant's top-$k(v)$ most
↪  preferred firms), but is $-\infty$ at other firms (meaning that the applicant will never be matched to firms they do not apply to).
```

Lean-expanded paper semantic target

type:
```
{College : Type v} → ((College → ℝ) → College → ℝ) → (College → ℝ) → (College → ℝ) → Prop
```

value:
```
fun {College : Type v} (demand : (College → ℝ) → College → ℝ) (capacity P : College → ℝ) =>
  ∀ (c : College), demand P c = capacity c
```

Recorded source-to-declaration screening **Verdict:** matches

Reason: Equating differential aggregate demand with each college’s capacity is exactly the source market-clearing definition under differential application access.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

PG23MonocultureMatching.pg23DifferentialTypeLaw

Paper-local declaration:
PG23MonocultureMatching.pg23DifferentialTypeLaw
Source locator: cited publication, lines 641-641

Verbatim paper-source connection

We now consider when different applicants can apply to different numbers of firms (analogously, colleges)---which we refer to as \textit{differential application access}. In particular, we assume some discrete probability distribution κ over $\{1, 2, \cdots, \numcolleges\}$ that determines the number of firms an applicant can apply to. In particular, an applicant with value v can apply to $k(v)$ firms, where $k(v)$ is drawn independently from κ. All other aspects of our model remain the same.

Lean-expanded paper semantic target

type:
```
{College : Type v} →
  [inst : Fintype College] →
    {n : ℕ} →
      MeasureTheory.Measure (Fin n) →
        MeasureTheory.Measure (PG23MonocultureMatching.PG23ApplicantType College) →
          MeasureTheory.Measure (PG23MonocultureMatching.PG23DifferentialApplicantType College n)
```

value:
```
fun {College : Type v} [Fintype College] {n : ℕ} (accessLaw : MeasureTheory.Measure (Fin n))
    (baseLaw : MeasureTheory.Measure (PG23MonocultureMatching.PG23ApplicantType College)) =>
  accessLaw.prod baseLaw
```

Direct retained dependencies

- AL16SupplyDemandMatching.instMeasurableSpaceAL16Ranking
- PG23MonocultureMatching.PG23ApplicantType
- PG23MonocultureMatching.PG23DifferentialApplicantType
- PG23MonocultureMatching.PG23Ranking

Recorded source-to-declaration screening **Verdict:** matches

Reason: The product measure of the access-position law and base type law formalizes an independently drawn positive access count and applicant type, with kappa represented as position + 1.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

PG23MonocultureMatching.pg23MatchingCollegeSet

Paper-local declaration:
PG23MonocultureMatching.pg23MatchingCollegeSet
Source locator: cited publication, lines 2-34

Verbatim paper-source connection

```
\section{Model and Preliminaries}
Our model builds on the model of \cite{azevedo2016supply}, in which there is a continuum of applicants and a finite number of firms (analogously,
↪ colleges). An applicant is identified by their true \textit{value}, which is distributed on $\RR$ according to a probability measure $\eta$. Let $\colleges
↪ = \{1, 2, \cdots, \numcolleges\}$ be the set of all firms. Firms have total capacity $\totalsupply < 1$ (so there are fewer positions than applicants)
↪ and each firm has the same capacity $\frac{\totalsupply}{\numcolleges}.$

An applicant with value $v$ is associated with an independent random variable $\theta(v)$, characterizing their preferences over firms and firms'
↪ estimated values for that applicant. The realization of $\theta(v)$ is their \textit{type}, which lies in $\Theta$. $\Theta := \mathcal{R} \times
↪ \RR^\numcolleges$ is the set of applicant types, where $\mathcal{R}$ is the set of preference orderings over the firms. If $\theta(v) = \theta =
↪ (\succ^\theta, e^\theta),$ then $\succ^\theta$ is the preference ordering of $v$ over firms and $e^\theta_\college$ is the \textit{estimated value} of
↪ the applicant at firm $\college$.

In the model, $\theta(v)$ plays a central role. First, it is used to model the process of preference approximation for firms. Firms all prefer applicants with
↪ higher true values, but in practice, rank applicants based on the estimated values given by $\theta(v)$. Second, it connects our model with that of
↪ \cite{azevedo2016supply}. In particular, the distribution $\eta$ over values in $\RR$ together with an appropriate choice of $\theta(v)$ induces a
↪ distribution over applicant types, which, alongside specifications of firm capacities, fully determines the model in
↪ \cite{azevedo2016supply}.\footnote{Formally, consider the probability distribution $\eta'$ over $\Theta$ such that for $A\subseteq \Theta$,
\begin{equation}
    \eta'(A) = \int_\RR \Pr[\theta(v)\in A]\,d\eta(v).
\end{equation}}

\paragraph{}
A \textit{matching} is a function $\mu: \Theta \rightarrow \colleges\cup \{\emptyset\}$ that satisfies the capacity constraints
\begin{equation}
    \int_\RR \Pr[\mu(\theta(v)) = \college]\,d\eta(v) = \frac{\totalsupply}{\numcolleges},\qquad \forall \college\in \colleges,
\end{equation}
as well as two additional technical assumptions: that $\mu^{-1}(\college)$ is measurable and that the set $\{\theta:\mu(\theta)\prec^\theta \college\}$ is
↪ open.\footnote{We remark that matchings are not typically required to fill capacities (i.e., the equality above is typically an inequality), but in our
↪ setting, since there is limited total capacity and applicants all prefer being matched to being unmatched, we are only concerned with matchings that
↪ fill capacity.}

Then $\mu(\theta(v))$ is a random variable indicating where an applicant with value $v$ is matched, with $\mu(\theta(v)) = \emptyset$ indicating that the
↪ applicant is not matched. When it is clear what $\theta$ is, we will abuse notation to write $\mu(v)$ instead of $\mu(\theta(v))$ to denote the random
↪ variable representing where an applicant with value $v$ is matched. Notice that the matching of an applicant depends only on their random type,
↪ reflecting the fact that their matching outcome depends directly on the scores they receive at each firm rather than their true value.

A matching is \textit{stable} if there does not exist a pair $\theta\in \Theta, \college\in \colleges$ such that $\college\succ^\theta \mu(\theta)$, and
↪ $e^\theta_\college > e^{\theta'}_{\college}$ for some $\theta'\in \mu^{-1}(\college)$. In other words, there is no applicant-firm pair that would
↪ jointly prefer to defect from the current matching to be matched with each other.

\subsection{A cutoff characterization of stable matchings}
A benefit of this continuum model is that stable matchings are characterized by a tractable cutoff structure \citep{azevedo2016supply}. Given a vector of
↪ \textit{cutoffs} $P = (P_1, \cdots, P_\numcolleges),$ an applicant with type $\theta$ can \textit{afford} firm $\college$ if $e^\theta_\college \ge
↪ P_\college$. The applicant's \textit{demand} at $P$ is $D^\theta(P)$, the applicant's most preferred firm (according to $\succ^\theta$) among those
↪ they can afford. If they cannot afford any firm, $D^\theta(P)=\emptyset$.
The \textit{aggregate demand} of a firm $\college$ at $P$ is
\begin{equation}
    D^\college(P) := \int_\RR \Pr[D^{\theta(v)}(P) = \college]\,d\eta(v),
\end{equation}
the measure of applicants who demand it.

A vector of cutoffs $P$ is \textit{market clearing} if and only if
\begin{equation}
    D^\college(P) = \frac{\totalsupply}{\numcolleges},\qquad \forall \college\in \numcolleges.\footnote{Again, as in the above footnotes, the standard
    ↪ definition in \cite{azevedo2016supply} would allow for an inequality here if no applicant would prefer to take the empty spots of a firm. This is not
    ↪ possible in our model, so we simplify our definition.}
\end{equation}

[Next verbatim source excerpt]


\subsection{Instantiating monoculture and polyculture}
We instantiate our model by specifying $\theta(v)$ for monoculture and polyculture. In what follows, we consider a fixed \textit{noise distribution} $\DD$.

\paragraph{Monoculture.} In monoculture, we take the random variable
\begin{equation}
    \theta_{\mono}(v) := (\succ, (v + X, v + X, \cdots, v + X)),
\end{equation}
where $\succ$ is drawn uniformly at random from $\mathcal{R}$ and $X\sim \DD.$ In other words, applicants have preferences distributed uniformly at
↪ random, and each applicant has a single estimated value $v+X$ that is shared across firms. This noisy estimated value equals the applicant's value
↪ perturbed by noise drawn from $\DD$.

\paragraph{Polyculture.} In polyculture, we take the random variable
\begin{equation}
    \theta_{\poly}(v) := (\succ, (v + X_1, v + X_2, \cdots, v + X_\numcolleges)),
\end{equation}
where $\succ$ is drawn uniformly at random from $\mathcal{R}$ and $X_1,X_2,\cdots, X_\numcolleges \sim \DD$ are i.i.d.. Again, applicants have
↪ preferences distributed uniformly at random, but now each applicant has a distinct, independently drawn estimated value at each firm.

\paragraph{} For expository purposes, we instantiate our model with a fixed noise distribution $\DD$ shared across both monoculture and polyculture.
↪ However, as will often be obvious, this assumption is not necessary for many of our results. In some cases, (primarily, in Theorem 2) it does allow us
↪ to compare monoculture and polyculture \textit{ceteris paribus}.
```

Lean-expanded paper semantic target

```
type:
{College : Type v} →
  [inst : Fintype College] →
    PG23MonocultureMatching.PG23Matching College → College → PG23MonocultureMatching.PG23ApplicantType College → Prop

value:
fun {College : Type v} [Fintype College] (matching : PG23MonocultureMatching.PG23Matching College) (c : College)
    (a : PG23MonocultureMatching.PG23ApplicantType College) =>
  {theta : PG23MonocultureMatching.PG23ApplicantType College | matching theta = some c} a
```

Direct retained dependencies

- PG23MonocultureMatching.PG23ApplicantType
- PG23MonocultureMatching.PG23Matching

Recorded source-to-declaration screening **Verdict:** matches

Reason: The fiber of applicants mapped to some college c is exactly the set of applicants matched to c in the source model.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

PG23MonocultureMatching.pg23MaximumConcentratingNoiseLaw

Paper-local declaration:
PG23MonocultureMatching.pg23MaximumConcentratingNoiseLaw
Source locator: cited publication, lines 125-130

Verbatim paper-source connection

```
\begin{definition}[Maximum-Concentrating Distribution]\label{def:maximum-concentrating}
   A distribution $\DD$ is \textbf{maximum concentrating} if for all $\epsilon > 0$,
   \begin{equation}
      \lim_{n\rightarrow \infty} \Pr\left[|X^{(n)} - \EE[X^{(n)}]| > \epsilon\right] = 0.
   \end{equation}
\end{definition}
```

Lean-expanded paper semantic target

```
type:
(noiseLaw : MeasureTheory.Measure ℝ) → [MeasureTheory.IsProbabilityMeasure noiseLaw] → Prop

value:
fun (noiseLaw : MeasureTheory.Measure ℝ) [MeasureTheory.IsProbabilityMeasure noiseLaw] =>
  (∀ (n : ℕ),
     MeasureTheory.Integrable
       (fun (sample : Fin (n + 1) → ℝ) =>
         AppliedModelingLib.Probability.upperOrderStatistic sample AppliedModelingLib.Probability.topSampleRank)
       (MeasureTheory.Measure.pi fun (x : Fin (n + 1)) => noiseLaw)) ∧
    PG23MonocultureMatching.maximumOrderStatisticConcentratingAroundExpected fun (n : ℕ) =>
     MeasureTheory.Measure.pi fun (x : Fin (n + 1)) => noiseLaw
```

Direct retained dependencies

- AppliedModelingLib.Probability.topSampleRank
- AppliedModelingLib.Probability.upperOrderStatistic
- PG23MonocultureMatching.maximumOrderStatisticConcentratingAroundExpected

Recorded source-to-declaration screening **Verdict:** matches

Reason: The predicate requires integrability of each finite-sample maximum and concentration of that maximum around its expectation, exactly recording the finite-expectation domain and convergence condition in the source definition.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

PG23MonocultureMatching.pg23MonocultureType

Paper-local declaration:
PG23MonocultureMatching.pg23MonocultureType
Source locator: cited publication, lines 52-56

Verbatim paper-source connection

```
\paragraph{Monoculture.} In monoculture, we take the random variable
\begin{equation}
   \theta_{\mono}(v) := (\succ, (v + X, v + X, \cdots, v + X)),
\end{equation}
where $\succ$ is drawn uniformly at random from $\mathcal{R}$ and $X\sim \DD.$ In other words, applicants have preferences distributed uniformly at
↪  random, and each applicant has a single estimated value $v+X$ that is shared across firms. This noisy estimated value equals the applicant's value
↪  perturbed by noise drawn from $\DD$.
```

Lean-expanded paper semantic target

```
type:
{College : Type v} →
  [inst : Fintype College] →
    ℝ → ℝ → PG23MonocultureMatching.PG23Ranking College → PG23MonocultureMatching.PG23ApplicantType College

value:
fun {College : Type v} [Fintype College] (value noise : ℝ) (ranking : PG23MonocultureMatching.PG23Ranking College) =>
  (ranking, fun (x : College) => value + noise)
```

Direct retained dependencies

- PG23MonocultureMatching.PG23ApplicantType
- PG23MonocultureMatching.PG23Ranking

Recorded source-to-declaration screening **Verdict:** matches

Reason: Assigning score v + epsilon to every college with the supplied ranking is exactly the source monoculture applicant type.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

PG23MonocultureMatching.pg23MonocultureSampleToType

Paper-local declaration:
PG23MonocultureMatching.pg23MonocultureSampleToType
Source locator: cited publication, lines 2-34

Verbatim paper-source connection

```
\section{Model and Preliminaries}
Our model builds on the model of \cite{azevedo2016supply}, in which there is a continuum of applicants and a finite number of firms (analogously,
↪ colleges). An applicant is identified by their true \textit{value}, which is distributed on $\RR$ according to a probability measure $\eta$. Let $\colleges
↪ = \{1, 2, \cdots, \numcolleges\}$ be the set of all firms. Firms have total capacity $\totalsupply < 1$ (so there are fewer positions than applicants)
↪ and each firm has the same capacity $\frac{\totalsupply}{\numcolleges}.$

An applicant with value $v$ is associated with an independent random variable $\theta(v)$, characterizing their preferences over firms and firms'
↪ estimated values for that applicant. The realization of $\theta(v)$ is their \textit{type}, which lies in $\Theta$. $\Theta := \mathcal{R} \times
↪ \RR^\numcolleges$ is the set of applicant types, where $\mathcal{R}$ is the set of preference orderings over the firms. If $\theta(v) = \theta =
↪ (\succ^\theta, e^\theta),$ then $\succ^\theta$ is the preference ordering of $v$ over firms and $e^\theta_\college$ is the \textit{estimated value} of
↪ the applicant at firm $\college$.

In the model, $\theta(v)$ plays a central role. First, it is used to model the process of preference approximation for firms. Firms all prefer applicants with
↪ higher true values, but in practice, rank applicants based on the estimated values given by $\theta(v)$. Second, it connects our model with that of
↪ \cite{azevedo2016supply}. In particular, the distribution $\eta$ over values in $\RR$ together with an appropriate choice of $\theta(v)$ induces a
↪ distribution over applicant types, which, alongside specifications of firm capacities, fully determines the model in
↪ \cite{azevedo2016supply}.\footnote{Formally, consider the probability distribution $\eta'$ over $\Theta$ such that for $A\subseteq \Theta$,
\begin{equation}
    \eta'(A) = \int_\RR \Pr[\theta(v)\in A]\,d\eta(v).
\end{equation}}

\paragraph{}
A \textit{matching} is a function $\mu: \Theta \rightarrow \colleges\cup \{\emptyset\}$ that satisfies the capacity constraints
\begin{equation}
    \int_\RR \Pr[\mu(\theta(v)) = \college]\,d\eta(v) = \frac{\totalsupply}{\numcolleges},\qquad \forall \college\in \colleges,
\end{equation}
as well as two additional technical assumptions: that $\mu^{-1}(\college)$ is measurable and that the set $\{\theta:\mu(\theta)\prec^\theta \college\}$ is
↪ open.\footnote{We remark that matchings are not typically required to fill capacities (i.e., the equality above is typically an inequality), but in our
↪ setting, since there is limited total capacity and applicants all prefer being matched to being unmatched, we are only concerned with matchings that
↪ fill capacity.}

Then $\mu(\theta(v))$ is a random variable indicating where an applicant with value $v$ is matched, with $\mu(\theta(v)) = \emptyset$ indicating that the
↪ applicant is not matched. When it is clear what $\theta$ is, we will abuse notation to write $\mu(v)$ instead of $\mu(\theta(v))$ to denote the random
↪ variable representing where an applicant with value $v$ is matched. Notice that the matching of an applicant depends only on their random type,
↪ reflecting the fact that their matching outcome depends directly on the scores they receive at each firm rather than their true value.

A matching is \textit{stable} if there does not exist a pair $\theta\in \Theta, \college\in \colleges$ such that $\college\succ^\theta \mu(\theta)$, and
↪ $e^\theta_\college > e^{\theta'}_{\college}$ for some $\theta'\in \mu^{-1}(\college)$. In other words, there is no applicant-firm pair that would
↪ jointly prefer to defect from the current matching to be matched with each other.

\subsection{A cutoff characterization of stable matchings}
A benefit of this continuum model is that stable matchings are characterized by a tractable cutoff structure \citep{azevedo2016supply}. Given a vector of
↪ \textit{cutoffs} $P = (P_1, \cdots, P_\numcolleges),$ an applicant with type $\theta$ can \textit{afford} firm $\college$ if $e^\theta_\college \ge
↪ P_\college$. The applicant's \textit{demand} at $P$ is $D^\theta(P)$, the applicant's most preferred firm (according to $\succ^\theta$) among those
↪ they can afford. If they cannot afford any firm, $D^\theta(P)=\emptyset$.
The \textit{aggregate demand} of a firm $\college$ at $P$ is
\begin{equation}
    D^\college(P) := \int_\RR \Pr[D^{\theta(v)}(P) = \college]\,d\eta(v),
\end{equation}
the measure of applicants who demand it.

A vector of cutoffs $P$ is \textit{market clearing} if and only if
\begin{equation}
    D^\college(P) = \frac{\totalsupply}{\numcolleges},\qquad \forall \college\in \numcolleges.\footnote{Again, as in the above footnotes, the standard
    ↪ definition in \cite{azevedo2016supply} would allow for an inequality here if no applicant would prefer to take the empty spots of a firm. This is not
    ↪ possible in our model, so we simplify our definition.}
\end{equation}

[Next verbatim source excerpt]


\subsection{Instantiating monoculture and polyculture}
We instantiate our model by specifying $\theta(v)$ for monoculture and polyculture. In what follows, we consider a fixed \textit{noise distribution} $\DD$.

\paragraph{Monoculture.} In monoculture, we take the random variable
\begin{equation}
    \theta_{\mono}(v) := (\succ, (v + X, v + X, \cdots, v + X)),
\end{equation}
where $\succ$ is drawn uniformly at random from $\mathcal{R}$ and $X\sim \DD.$ In other words, applicants have preferences distributed uniformly at
↪ random, and each applicant has a single estimated value $v+X$ that is shared across firms. This noisy estimated value equals the applicant's value
↪ perturbed by noise drawn from $\DD$.

\paragraph{Polyculture.} In polyculture, we take the random variable
\begin{equation}
    \theta_{\poly}(v) := (\succ, (v + X_1, v + X_2, \cdots, v + X_\numcolleges)),
\end{equation}
where $\succ$ is drawn uniformly at random from $\mathcal{R}$ and $X_1,X_2,\cdots, X_\numcolleges \sim \DD$ are i.i.d.. Again, applicants have
↪ preferences distributed uniformly at random, but now each applicant has a distinct, independently drawn estimated value at each firm.

\paragraph{} For expository purposes, we instantiate our model with a fixed noise distribution $\DD$ shared across both monoculture and polyculture.
↪ However, as will often be obvious, this assumption is not necessary for many of our results. In some cases, (primarily, in Theorem 2) it does allow us
↪ to compare monoculture and polyculture \textit{ceteris paribus}.
```

Lean-expanded paper semantic target

```
type:
{College : Type v} →
  [inst : Fintype College] →
    PG23MonocultureMatching.PG23MonocultureSample College → PG23MonocultureMatching.PG23ApplicantType College

value:
fun {College : Type v} [Fintype College] (a : PG23MonocultureMatching.PG23MonocultureSample College) =>
  PG23MonocultureMatching.pg23MonocultureType a.1.1 a.2 a.1.2
```

Direct retained dependencies

- PG23MonocultureMatching.PG23ApplicantType
- PG23MonocultureMatching.PG23MonocultureSample
- PG23MonocultureMatching.PG23Ranking
- PG23MonocultureMatching.pg23MonocultureType

Recorded source-to-declaration screening **Verdict:** matches

Reason: The sample-to-type map sends one value, one common noise draw, and a ranking to the corresponding monoculture applicant, matching the source construction.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

PG23MonocultureMatching.pg23UniformRankingLaw

Paper-local declaration:
PG23MonocultureMatching.pg23UniformRankingLaw
Source locator: cited publication, lines 52-56

Verbatim paper-source connection

```
\paragraph{Monoculture.} In monoculture, we take the random variable
\begin{equation}
   \theta_{\mono}(v) := (\succ, (v + X, v + X, \cdots, v + X)),
\end{equation}
where $\succ$ is drawn uniformly at random from $\mathcal{R}$ and $X\sim \DD.$ In other words, applicants have preferences distributed uniformly at
↪  random, and each applicant has a single estimated value $v+X$ that is shared across firms. This noisy estimated value equals the applicant's value
↪  perturbed by noise drawn from $\DD$.
```

Lean-expanded paper semantic target

type:
```
{College : Type v} → [inst : Fintype College] → MeasureTheory.Measure (PG23MonocultureMatching.PG23Ranking College)
```

value:
```
fun {College : Type v} [Fintype College] => ProbabilityTheory.uniformOn Set.univ
```

Direct retained dependencies

- AL16SupplyDemandMatching.instMeasurableSpaceAL16Ranking
- PG23MonocultureMatching.PG23Ranking

Recorded source-to-declaration screening **Verdict:** matches

Reason: The uniform probability measure on the finite set of all rankings is exactly the source uniform-ranking law.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

PG23MonocultureMatching.pg23MonocultureTypeLaw

Paper-local declaration:
PG23MonocultureMatching.pg23MonocultureTypeLaw
Source locator: cited publication, lines 52-56

Verbatim paper-source connection

```
\paragraph{Monoculture.} In monoculture, we take the random variable
\begin{equation}
   \theta_{\mono}(v) := (\succ, (v + X, v + X, \cdots, v + X)),
\end{equation}
where $\succ$ is drawn uniformly at random from $\mathcal{R}$ and $X\sim \DD.$ In other words, applicants have preferences distributed uniformly at
↪  random, and each applicant has a single estimated value $v+X$ that is shared across firms. This noisy estimated value equals the applicant's value
↪  perturbed by noise drawn from $\DD$.
```

Lean-expanded paper semantic target

```
type:
{College : Type v} →
  [inst : Fintype College] →
    (valueLaw noiseLaw : MeasureTheory.Measure ℝ) →
      [MeasureTheory.IsProbabilityMeasure valueLaw] →
        [MeasureTheory.IsProbabilityMeasure noiseLaw] →
          MeasureTheory.Measure (PG23MonocultureMatching.PG23ApplicantType College)

value:
fun {College : Type v} [Fintype College] (valueLaw noiseLaw : MeasureTheory.Measure ℝ)
    [MeasureTheory.IsProbabilityMeasure valueLaw] [MeasureTheory.IsProbabilityMeasure noiseLaw] =>
  MeasureTheory.Measure.map PG23MonocultureMatching.pg23MonocultureSampleToType
    ((valueLaw.prod PG23MonocultureMatching.pg23UniformRankingLaw).prod noiseLaw)
```

Direct retained dependencies

- AL16SupplyDemandMatching.instMeasurableSpaceAL16Ranking
- PG23MonocultureMatching.PG23ApplicantType
- PG23MonocultureMatching.PG23MonocultureSample
- PG23MonocultureMatching.PG23Ranking
- PG23MonocultureMatching.pg23MonocultureSampleToType
- PG23MonocultureMatching.pg23UniformRankingLaw

Recorded source-to-declaration screening **Verdict:** matches

Reason: The pushforward of independent uniform ranking, value, and common-noise draws through the monoculture type map is exactly the source monoculture type law.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

PG23MonocultureMatching.pg23MonocultureDifferentialTypeLaw

Paper-local declaration:
PG23MonocultureMatching.pg23MonocultureDifferentialTypeLaw
Source locator: cited publication, lines 651-663

Verbatim paper-source connection

```
\paragraph{Monoculture with differential application access.} Define
\begin{equation}
   \theta_{\mono, \kappa}(v) = (\succ, e)
\end{equation}
where
\begin{equation}
   e_c =
   \begin{cases}
      v + X &\quad \college\in S_{k(v)}(v, \succ)\\
      -\infty &\quad \college\notin S_{k(v)}(v, \succ)
   \end{cases},
\end{equation}
for $\succ$ drawn uniformly at random from $\mathcal{R}$, $k(v)$ drawn from $\kappa,$ and $X$ drawn from $\DD$. As before, an applicant with value
↪  $v$ is given a random preference ordering over firms. However, now the applicant's estimated values across firms depends also on a random draw
↪  from $\kappa$, which determines how many firms they can apply to. Recall that $e$ gives the vector of estimated values of an applicant across firms.
↪  Therefore, under monoculture, this estimated value is shared across the firms the applicant applies to (which are the applicant's top-$k(v)$ most
↪  preferred firms), but is $-\infty$ at other firms (meaning that the applicant will never be matched to firms they do not apply to).
```

Lean-expanded paper semantic target

```
type:
{College : Type v} →
  [inst : Fintype College] →
    {n : ℕ} →
      MeasureTheory.Measure (Fin n) →
        (valueLaw noiseLaw : MeasureTheory.Measure ℝ) →
          [MeasureTheory.IsProbabilityMeasure valueLaw] →
            [MeasureTheory.IsProbabilityMeasure noiseLaw] →
              MeasureTheory.Measure (PG23MonocultureMatching.PG23DifferentialApplicantType College n)

value:
fun {College : Type v} [Fintype College] {n : ℕ} (accessLaw : MeasureTheory.Measure (Fin n))
    (valueLaw noiseLaw : MeasureTheory.Measure ℝ) [MeasureTheory.IsProbabilityMeasure valueLaw]
    [MeasureTheory.IsProbabilityMeasure noiseLaw] =>
  PG23MonocultureMatching.pg23DifferentialTypeLaw accessLaw
    (PG23MonocultureMatching.pg23MonocultureTypeLaw valueLaw noiseLaw)
```

Direct retained dependencies

- AL16SupplyDemandMatching.instMeasurableSpaceAL16Ranking
- PG23MonocultureMatching.PG23ApplicantType
- PG23MonocultureMatching.PG23DifferentialApplicantType
- PG23MonocultureMatching.PG23Ranking
- PG23MonocultureMatching.pg23DifferentialTypeLaw
- PG23MonocultureMatching.pg23MonocultureTypeLaw

Recorded source-to-declaration screening **Verdict:** matches

Reason: Taking the product of the access-position law with the monoculture base type law gives exactly the source monoculture differential-access population with independent kappa.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

PG23MonocultureMatching.pg23PolycultureType

Paper-local declaration:
PG23MonocultureMatching.pg23PolycultureType
Source locator: cited publication, lines 58-62

Verbatim paper-source connection

```
\paragraph{Polyculture.} In polyculture, we take the random variable
\begin{equation}
   \theta_{\poly}(v) := (\succ, (v + X_1, v + X_2, \cdots, v + X_\numcolleges)),
\end{equation}
where $\succ$ is drawn uniformly at random from $\mathcal{R}$ and $X_1,X_2,\cdots, X_\numcolleges \sim \DD$ are i.i.d.. Again, applicants have
↪   preferences distributed uniformly at random, but now each applicant has a distinct, independently drawn estimated value at each firm.
```

Lean-expanded paper semantic target

```
type:
{College : Type v} →
  [inst : Fintype College] →
    ℝ → PG23MonocultureMatching.PG23Ranking College → (College → ℝ) → PG23MonocultureMatching.PG23ApplicantType College

value:
fun {College : Type v} [Fintype College] (value : ℝ) (ranking : PG23MonocultureMatching.PG23Ranking College)
    (noise : College → ℝ) =>
  (ranking, fun (c : College) => value + noise c)
```

Direct retained dependencies

- PG23MonocultureMatching.PG23ApplicantType
- PG23MonocultureMatching.PG23Ranking

Recorded source-to-declaration screening **Verdict:** matches

Reason: Assigning score v + epsilon_c at each college with the supplied ranking is exactly the source polyculture applicant type.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

PG23MonocultureMatching.pg23PolycultureSampleToType

Paper-local declaration:
PG23MonocultureMatching.pg23PolycultureSampleToType
Source locator: cited publication, lines 2-34

Verbatim paper-source connection

```
\section{Model and Preliminaries}
Our model builds on the model of \cite{azevedo2016supply}, in which there is a continuum of applicants and a finite number of firms (analogously,
↪ colleges). An applicant is identified by their true \textit{value}, which is distributed on $\RR$ according to a probability measure $\eta$. Let $\colleges
↪ = \{1, 2, \cdots, \numcolleges\}$ be the set of all firms. Firms have total capacity $\totalsupply < 1$ (so there are fewer positions than applicants)
↪ and each firm has the same capacity $\frac{\totalsupply}{\numcolleges}.$

An applicant with value $v$ is associated with an independent random variable $\theta(v)$, characterizing their preferences over firms and firms'
↪ estimated values for that applicant. The realization of $\theta(v)$ is their \textit{type}, which lies in $\Theta$. $\Theta := \mathcal{R} \times
↪ \RR^\numcolleges$ is the set of applicant types, where $\mathcal{R}$ is the set of preference orderings over the firms. If $\theta(v) = \theta =
↪ (\succ^\theta, e^\theta),$ then $\succ^\theta$ is the preference ordering of $v$ over firms and $e^\theta_\college$ is the \textit{estimated value} of
↪ the applicant at firm $\college$.

In the model, $\theta(v)$ plays a central role. First, it is used to model the process of preference approximation for firms. Firms all prefer applicants with
↪ higher true values, but in practice, rank applicants based on the estimated values given by $\theta(v)$. Second, it connects our model with that of
↪ \cite{azevedo2016supply}. In particular, the distribution $\eta$ over values in $\RR$ together with an appropriate choice of $\theta(v)$ induces a
↪ distribution over applicant types, which, alongside specifications of firm capacities, fully determines the model in
↪ \cite{azevedo2016supply}.\footnote{Formally, consider the probability distribution $\eta'$ over $\Theta$ such that for $A\subseteq \Theta$,
\begin{equation}
    \eta'(A) = \int_\RR \Pr[\theta(v)\in A]\,d\eta(v).
\end{equation}}

\paragraph{}
A \textit{matching} is a function $\mu: \Theta \rightarrow \colleges\cup \{\emptyset\}$ that satisfies the capacity constraints
\begin{equation}
    \int_\RR \Pr[\mu(\theta(v)) = \college]\,d\eta(v) = \frac{\totalsupply}{\numcolleges},\qquad \forall \college\in \colleges,
\end{equation}
as well as two additional technical assumptions: that $\mu^{-1}(\college)$ is measurable and that the set $\{\theta:\mu(\theta)\prec^\theta \college\}$ is
↪ open.\footnote{We remark that matchings are not typically required to fill capacities (i.e., the equality above is typically an inequality), but in our
↪ setting, since there is limited total capacity and applicants all prefer being matched to being unmatched, we are only concerned with matchings that
↪ fill capacity.}

Then $\mu(\theta(v))$ is a random variable indicating where an applicant with value $v$ is matched, with $\mu(\theta(v)) = \emptyset$ indicating that the
↪ applicant is not matched. When it is clear what $\theta$ is, we will abuse notation to write $\mu(v)$ instead of $\mu(\theta(v))$ to denote the random
↪ variable representing where an applicant with value $v$ is matched. Notice that the matching of an applicant depends only on their random type,
↪ reflecting the fact that their matching outcome depends directly on the scores they receive at each firm rather than their true value.

A matching is \textit{stable} if there does not exist a pair $\theta\in \Theta, \college\in \colleges$ such that $\college\succ^\theta \mu(\theta)$, and
↪ $e^\theta_\college > e^{\theta'}_{\college}$ for some $\theta'\in \mu^{-1}(\college)$. In other words, there is no applicant-firm pair that would
↪ jointly prefer to defect from the current matching to be matched with each other.

\subsection{A cutoff characterization of stable matchings}
A benefit of this continuum model is that stable matchings are characterized by a tractable cutoff structure \citep{azevedo2016supply}. Given a vector of
↪ \textit{cutoffs} $P = (P_1, \cdots, P_\numcolleges),$ an applicant with type $\theta$ can \textit{afford} firm $\college$ if $e^\theta_\college \ge
↪ P_\college$. The applicant's \textit{demand} at $P$ is $D^\theta(P)$, the applicant's most preferred firm (according to $\succ^\theta$) among those
↪ they can afford. If they cannot afford any firm, $D^\theta(P)=\emptyset$.
The \textit{aggregate demand} of a firm $\college$ at $P$ is
\begin{equation}
    D^\college(P) := \int_\RR \Pr[D^{\theta(v)}(P) = \college]\,d\eta(v),
\end{equation}
the measure of applicants who demand it.

A vector of cutoffs $P$ is \textit{market clearing} if and only if
\begin{equation}
    D^\college(P) = \frac{\totalsupply}{\numcolleges},\qquad \forall \college\in \numcolleges.\footnote{Again, as in the above footnotes, the standard
    ↪ definition in \cite{azevedo2016supply} would allow for an inequality here if no applicant would prefer to take the empty spots of a firm. This is not
    ↪ possible in our model, so we simplify our definition.}
\end{equation}

[Next verbatim source excerpt]


\subsection{Instantiating monoculture and polyculture}
We instantiate our model by specifying $\theta(v)$ for monoculture and polyculture. In what follows, we consider a fixed \textit{noise distribution} $\DD$.

\paragraph{Monoculture.} In monoculture, we take the random variable
\begin{equation}
    \theta_{\mono}(v) := (\succ, (v + X, v + X, \cdots, v + X)),
\end{equation}
where $\succ$ is drawn uniformly at random from $\mathcal{R}$ and $X\sim \DD.$ In other words, applicants have preferences distributed uniformly at
↪ random, and each applicant has a single estimated value $v+X$ that is shared across firms. This noisy estimated value equals the applicant's value
↪ perturbed by noise drawn from $\DD$.

\paragraph{Polyculture.} In polyculture, we take the random variable
\begin{equation}
    \theta_{\poly}(v) := (\succ, (v + X_1, v + X_2, \cdots, v + X_\numcolleges)),
\end{equation}
where $\succ$ is drawn uniformly at random from $\mathcal{R}$ and $X_1,X_2,\cdots, X_\numcolleges \sim \DD$ are i.i.d.. Again, applicants have
↪ preferences distributed uniformly at random, but now each applicant has a distinct, independently drawn estimated value at each firm.

\paragraph{} For expository purposes, we instantiate our model with a fixed noise distribution $\DD$ shared across both monoculture and polyculture.
↪ However, as will often be obvious, this assumption is not necessary for many of our results. In some cases, (primarily, in Theorem 2) it does allow us
↪ to compare monoculture and polyculture \textit{ceteris paribus}.
```

Lean-expanded paper semantic target

type:
{College : Type v} →
  [inst : Fintype College] →
    PG23MonocultureMatching.PG23PolycultureSample College → PG23MonocultureMatching.PG23ApplicantType College

value:
fun {College : Type v} [Fintype College] (a : PG23MonocultureMatching.PG23PolycultureSample College) =>
  PG23MonocultureMatching.pg23PolycultureType a.1.1 a.1.2 a.2

Direct retained dependencies

- PG23MonocultureMatching.PG23ApplicantType
- PG23MonocultureMatching.PG23PolycultureSample
- PG23MonocultureMatching.PG23Ranking
- PG23MonocultureMatching.pg23PolycultureType

Recorded source-to-declaration screening

Verdict: matches
Reason: The sample-to-type map combines one value, a college-indexed noise vector, and a ranking coordinatewise, matching the source polyculture construction.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

PG23MonocultureMatching.pg23PolycultureTypeLaw

Paper-local declaration:
PG23MonocultureMatching.pg23PolycultureTypeLaw
Source locator: cited publication, lines 58-62

Verbatim paper-source connection

```
\paragraph{Polyculture.} In polyculture, we take the random variable
\begin{equation}
   \theta_{\poly}(v) := (\succ, (v + X_1, v + X_2, \cdots, v + X_\numcolleges)),
\end{equation}
where $\succ$ is drawn uniformly at random from $\mathcal{R}$ and $X_1,X_2,\cdots, X_\numcolleges \sim \DD$ are i.i.d.. Again, applicants have
↪  preferences distributed uniformly at random, but now each applicant has a distinct, independently drawn estimated value at each firm.
```

Lean-expanded paper semantic target

```
type:
{College : Type v} →
  [inst : Fintype College] →
    (valueLaw noiseLaw : MeasureTheory.Measure ℝ) →
      [MeasureTheory.IsProbabilityMeasure valueLaw] →
        [MeasureTheory.IsProbabilityMeasure noiseLaw] →
          MeasureTheory.Measure (PG23MonocultureMatching.PG23ApplicantType College)

value:
fun {College : Type v} [Fintype College] (valueLaw noiseLaw : MeasureTheory.Measure ℝ)
    [MeasureTheory.IsProbabilityMeasure valueLaw] [MeasureTheory.IsProbabilityMeasure noiseLaw] =>
  MeasureTheory.Measure.map PG23MonocultureMatching.pg23PolycultureSampleToType
    ((valueLaw.prod PG23MonocultureMatching.pg23UniformRankingLaw).prod
      (MeasureTheory.Measure.pi fun (x : College) => noiseLaw))
```

Direct retained dependencies

- AL16SupplyDemandMatching.instMeasurableSpaceAL16Ranking
- PG23MonocultureMatching.PG23ApplicantType
- PG23MonocultureMatching.PG23PolycultureSample
- PG23MonocultureMatching.PG23Ranking
- PG23MonocultureMatching.pg23PolycultureSampleToType
- PG23MonocultureMatching.pg23UniformRankingLaw

Recorded source-to-declaration screening **Verdict:** matches

Reason: The pushforward of independent uniform ranking, value, and iid college-noise draws through the polyculture type map is exactly the source polyculture type law.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

PG23MonocultureMatching.pg23PolycultureDifferentialTypeLaw

Paper-local declaration:
PG23MonocultureMatching.pg23PolycultureDifferentialTypeLaw
Source locator: cited publication, lines 665-677

Verbatim paper-source connection

```
\paragraph{Polyculture with differential application access.} Define
\begin{equation}
   \theta_{\poly, \kappa}(v) = (\succ, e)
\end{equation}
where
\begin{equation}
   e_\college =
   \begin{cases}
      v + X_\college &\quad \college\in S_{k(v)}(v, \succ)\\
      -\infty &\quad \college\notin S_{k(v)}(v, \succ)
   \end{cases},
\end{equation}
for $\succ$ drawn uniformly at random from $\mathcal{R}$, $k(v)$ drawn from $\kappa,$ and $X_1,\cdots,X_\numcolleges\simiid \DD.$ Here, an applicant
↪  gets an independently drawn estimated value at each firm they apply to.
```

Lean-expanded paper semantic target

```
type:
{College : Type v} →
  [inst : Fintype College] →
    {n : ℕ} →
      MeasureTheory.Measure (Fin n) →
        (valueLaw noiseLaw : MeasureTheory.Measure ℝ) →
          [MeasureTheory.IsProbabilityMeasure valueLaw] →
            [MeasureTheory.IsProbabilityMeasure noiseLaw] →
              MeasureTheory.Measure (PG23MonocultureMatching.PG23DifferentialApplicantType College n)

value:
fun {College : Type v} [Fintype College] {n : ℕ} (accessLaw : MeasureTheory.Measure (Fin n))
    (valueLaw noiseLaw : MeasureTheory.Measure ℝ) [MeasureTheory.IsProbabilityMeasure valueLaw]
    [MeasureTheory.IsProbabilityMeasure noiseLaw] =>
  PG23MonocultureMatching.pg23DifferentialTypeLaw accessLaw
    (PG23MonocultureMatching.pg23PolycultureTypeLaw valueLaw noiseLaw)
```

Direct retained dependencies

- AL16SupplyDemandMatching.instMeasurableSpaceAL16Ranking
- PG23MonocultureMatching.PG23ApplicantType
- PG23MonocultureMatching.PG23DifferentialApplicantType
- PG23MonocultureMatching.PG23Ranking
- PG23MonocultureMatching.pg23DifferentialTypeLaw
- PG23MonocultureMatching.pg23PolycultureTypeLaw

Recorded source-to-declaration screening **Verdict:** matches

Reason: Taking the product of the access-position law with the polyculture base type law gives exactly the source polyculture differential-access population with independent kappa.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

PG23MonocultureMatching.pg23RawCutoffInf

Paper-local declaration:
PG23MonocultureMatching.pg23RawCutoffInf
Source locator: cited publication, lines 188-196

Verbatim paper-source connection

```
\subsection{The Lattice Theorem for Stable Matchings}
A powerful result in the stable matching literature shows that stable matchings have a complete lattice structure. In this section, we state this result in our
↪  setup (Theorem A.1 in \cite{azevedo2016supply}). Consider the lattice operators $\vee$ and $\wedge$ on cutoffs in $\RR^C$, where for a set $Z$ of
↪  cutoffs,
\begin{equation}
   \left(\bigvee_{P\in Z} P\right)_\college = \sup_{P\in Z} P_\college
\end{equation}
and
\begin{equation}
   \left(\bigwedge_{P\in Z} P\right)_\college = \inf_{P\in Z} P_\college.
\end{equation}
```

Lean-expanded paper semantic target

```
type:
{College : Type v} → Set (College → ℝ) → College → ℝ

value:
fun {College : Type v} (Z : Set (College → ℝ)) (a : College) => sInf ((fun (P : College → ℝ) => P a) '' Z)
```

Recorded source-to-declaration screening **Verdict:** matches

Reason: The finite-coordinate infimum is the source pointwise lowest cutoff used to define the raw cutoff order.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

PG23MonocultureMatching.pg23RawCutoffLe

Paper-local declaration:
PG23MonocultureMatching.pg23RawCutoffLe
Source locator: cited publication, lines 188-196

Verbatim paper-source connection

```
\subsection{The Lattice Theorem for Stable Matchings}
A powerful result in the stable matching literature shows that stable matchings have a complete lattice structure. In this section, we state this result in our
↪  setup (Theorem A.1 in \cite{azevedo2016supply}). Consider the lattice operators $\vee$ and $\wedge$ on cutoffs in $\RR^C$, where for a set $Z$ of
↪  cutoffs,
\begin{equation}
   \left(\bigvee_{P\in Z} P\right)_\college = \sup_{P\in Z} P_\college
\end{equation}
and
\begin{equation}
   \left(\bigwedge_{P\in Z} P\right)_\college = \inf_{P\in Z} P_\college.
\end{equation}
```

Lean-expanded paper semantic target

```
type:
{College : Type v} → (College → ℝ) → (College → ℝ) → Prop

value:
fun {College : Type v} (P Q : College → ℝ) => ∀ (c : College), P c ≤ Q c
```

Recorded source-to-declaration screening **Verdict:** matches

Reason: Pointwise comparison of cutoff vectors is exactly the source coordinatewise cutoff order.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

PG23MonocultureMatching.pg23RawCutoffSup

Paper-local declaration:
PG23MonocultureMatching.pg23RawCutoffSup
Source locator: cited publication, lines 188-196

Verbatim paper-source connection

```
\subsection{The Lattice Theorem for Stable Matchings}
A powerful result in the stable matching literature shows that stable matchings have a complete lattice structure. In this section, we state this result in our
↪  setup (Theorem A.1 in \cite{azevedo2016supply}). Consider the lattice operators $\vee$ and $\wedge$ on cutoffs in $\RR^C$, where for a set $Z$ of
↪  cutoffs,
\begin{equation}
   \left(\bigvee_{P\in Z} P\right)_\college = \sup_{P\in Z} P_\college
\end{equation}
and
\begin{equation}
   \left(\bigwedge_{P\in Z} P\right)_\college = \inf_{P\in Z} P_\college.
\end{equation}
```

Lean-expanded paper semantic target

```
type:
{College : Type v} → Set (College → ℝ) → College → ℝ

value:
fun {College : Type v} (Z : Set (College → ℝ)) (a : College) => sSup ((fun (P : College → ℝ) => P a) '' Z)
```

Recorded source-to-declaration screening **Verdict:** matches

Reason: The finite-coordinate supremum is the source pointwise highest cutoff used to define the raw cutoff order.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

PG23MonocultureMatching.pg23SourceChoice

Paper-local declaration:
PG23MonocultureMatching.pg23SourceChoice
Source locator: cited publication, lines 24-34

Verbatim paper-source connection

```
A benefit of this continuum model is that stable matchings are characterized by a tractable cutoff structure \citep{azevedo2016supply}. Given a vector of
↪  \textit{cutoffs} $P = (P_1, \cdots, P_\numcolleges),$ an applicant with type $\theta$ can \textit{afford} firm $\college$ if $e^\theta_\college \ge
↪  P_\college$. The applicant's \textit{demand} at $P$ is $D^\theta(P)$, the applicant's most preferred firm (according to $\succ^\theta$) among those
↪  they can afford. If they cannot afford any firm, $D^\theta(P)=\emptyset$.
The \textit{aggregate demand} of a firm $\college$ at $P$ is
\begin{equation}
   D^\college(P) := \int_\RR \Pr[D^{\theta(v)}(P) = \college]\,d\eta(v),
\end{equation}
the measure of applicants who demand it.

A vector of cutoffs $P$ is \textit{market clearing} if and only if
\begin{equation}
   D^\college(P) = \frac{\totalsupply}{\numcolleges},\qquad \forall \college\in \numcolleges.\footnote{Again, as in the above footnotes, the standard
   ↪  definition in \cite{azevedo2016supply} would allow for an inequality here if no applicant would prefer to take the empty spots of a firm. This is not
   ↪  possible in our model, so we simplify our definition.}
\end{equation}
```

Lean-expanded paper semantic target

```
type:
{College : Type v} →
  [inst : Fintype College] → (College → ℝ) → PG23MonocultureMatching.PG23ApplicantType College → Option College

value:
fun {College : Type v} [Fintype College] (P : College → ℝ) (a : PG23MonocultureMatching.PG23ApplicantType College) =>
  AL16SupplyDemandMatching.al16SourceChoice (PG23MonocultureMatching.pg23NormalizedCutoff P)
    (PG23MonocultureMatching.pg23NormalizedStudent a)
```

Direct retained dependencies

- AL16SupplyDemandMatching.al16SourceChoice
- PG23MonocultureMatching.PG23ApplicantType
- PG23MonocultureMatching.pg23NormalizedCutoff
- PG23MonocultureMatching.pg23NormalizedStudent

Recorded source-to-declaration screening **Verdict:** matches

Reason: The AL16 favorite-affordable selector is applied after the same strictly increasing normalization to scores and cutoffs, so it preserves the source weak affordability comparisons and strict ranking-based choice in every displayed use.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

PG23MonocultureMatching.pg23SourceAggregateDemand

Paper-local declaration:
PG23MonocultureMatching.pg23SourceAggregateDemand
Source locator: cited publication, lines 24-34

Verbatim paper-source connection

```
A benefit of this continuum model is that stable matchings are characterized by a tractable cutoff structure \citep{azevedo2016supply}. Given a vector of
↪  \textit{cutoffs} $P = (P_1, \cdots, P_\numcolleges),$ an applicant with type $\theta$ can \textit{afford} firm $\college$ if $e^\theta_\college \ge
↪  P_\college$. The applicant's \textit{demand} at $P$ is $D^\theta(P)$, the applicant's most preferred firm (according to $\succ^\theta$) among those
↪  they can afford. If they cannot afford any firm, $D^\theta(P)=\emptyset$.
The \textit{aggregate demand} of a firm $\college$ at $P$ is
\begin{equation}
   D^\college(P) := \int_\RR \Pr[D^{\theta(v)}(P) = \college]\,d\eta(v),
\end{equation}
the measure of applicants who demand it.

A vector of cutoffs $P$ is \textit{market clearing} if and only if
\begin{equation}
   D^\college(P) = \frac{\totalsupply}{\numcolleges},\qquad \forall \college\in \numcolleges.\footnote{Again, as in the above footnotes, the standard
   ↪  definition in \cite{azevedo2016supply} would allow for an inequality here if no applicant would prefer to take the empty spots of a firm. This is not
   ↪  possible in our model, so we simplify our definition.}
\end{equation}
```

Lean-expanded paper semantic target

```
type:
{College : Type v} →
  [inst : Fintype College] →
    MeasureTheory.Measure (PG23MonocultureMatching.PG23ApplicantType College) → (College → ℝ) → College → ℝ

value:
fun {College : Type v} [Fintype College]
    (typeLaw : MeasureTheory.Measure (PG23MonocultureMatching.PG23ApplicantType College)) (P : College → ℝ)
    (c : College) =>
  typeLaw.real (PG23MonocultureMatching.pg23MatchingCollegeSet (PG23MonocultureMatching.pg23SourceChoice P) c)
```

Direct retained dependencies

- AL16SupplyDemandMatching.instMeasurableSpaceAL16Ranking
- PG23MonocultureMatching.PG23ApplicantType
- PG23MonocultureMatching.PG23Ranking
- PG23MonocultureMatching.pg23MatchingCollegeSet
- PG23MonocultureMatching.pg23SourceChoice

Recorded source-to-declaration screening **Verdict:** matches

Reason: The measure of the applicants whose source choice is c is exactly the source aggregate-demand function at c.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

PG23MonocultureMatching.pg23SourceMarketClearing

Paper-local declaration:
PG23MonocultureMatching.pg23SourceMarketClearing
Source locator: cited publication, lines 24-34

Verbatim paper-source connection

```
A benefit of this continuum model is that stable matchings are characterized by a tractable cutoff structure \citep{azevedo2016supply}. Given a vector of
↪  \textit{cutoffs} $P = (P_1, \cdots, P_\numcolleges),$ an applicant with type $\theta$ can \textit{afford} firm $\college$ if $e^\theta_\college \ge
↪  P_\college$. The applicant's \textit{demand} at $P$ is $D^\theta(P)$, the applicant's most preferred firm (according to $\succ^\theta$) among those
↪  they can afford. If they cannot afford any firm, $D^\theta(P)=\emptyset$.
The \textit{aggregate demand} of a firm $\college$ at $P$ is
\begin{equation}
   D^\college(P) := \int_\RR \Pr[D^{\theta(v)}(P) = \college]\,d\eta(v),
\end{equation}
the measure of applicants who demand it.

A vector of cutoffs $P$ is \textit{market clearing} if and only if
\begin{equation}
   D^\college(P) = \frac{\totalsupply}{\numcolleges},\qquad \forall \college\in \numcolleges.\footnote{Again, as in the above footnotes, the standard
   ↪  definition in \cite{azevedo2016supply} would allow for an inequality here if no applicant would prefer to take the empty spots of a firm. This is not
   ↪  possible in our model, so we simplify our definition.}
\end{equation}
```

Lean-expanded paper semantic target

```
type:
{College : Type v} → ((College → ℝ) → College → ℝ) → (College → ℝ) → (College → ℝ) → Prop

value:
fun {College : Type v} (demand : (College → ℝ) → College → ℝ) (capacity P : College → ℝ) =>
  ∀ (c : College), demand P c = capacity c
```

Recorded source-to-declaration screening **Verdict:** matches

Reason: Requiring source aggregate demand to equal each college’s capacity is exactly the source market-clearing condition.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

PG23MonocultureMatching.pg23SourceMatchingBlocks

Paper-local declaration:
PG23MonocultureMatching.pg23SourceMatchingBlocks
Source locator: cited publication, lines 2-34

Verbatim paper-source connection

```
\section{Model and Preliminaries}
Our model builds on the model of \cite{azevedo2016supply}, in which there is a continuum of applicants and a finite number of firms (analogously,
↪ colleges). An applicant is identified by their true \textit{value}, which is distributed on $\RR$ according to a probability measure $\eta$. Let $\colleges
↪ = \{1, 2, \cdots, \numcolleges\}$ be the set of all firms. Firms have total capacity $\totalsupply < 1$ (so there are fewer positions than applicants)
↪ and each firm has the same capacity $\frac{\totalsupply}{\numcolleges}.$

An applicant with value $v$ is associated with an independent random variable $\theta(v)$, characterizing their preferences over firms and firms'
↪ estimated values for that applicant. The realization of $\theta(v)$ is their \textit{type}, which lies in $\Theta$. $\Theta := \mathcal{R} \times
↪ \RR^\numcolleges$ is the set of applicant types, where $\mathcal{R}$ is the set of preference orderings over the firms. If $\theta(v) = \theta =
↪ (\succ^\theta, e^\theta),$ then $\succ^\theta$ is the preference ordering of $v$ over firms and $e^\theta_\college$ is the \textit{estimated value} of
↪ the applicant at firm $\college$.

In the model, $\theta(v)$ plays a central role. First, it is used to model the process of preference approximation for firms. Firms all prefer applicants with
↪ higher true values, but in practice, rank applicants based on the estimated values given by $\theta(v)$. Second, it connects our model with that of
↪ \cite{azevedo2016supply}. In particular, the distribution $\eta$ over values in $\RR$ together with an appropriate choice of $\theta(v)$ induces a
↪ distribution over applicant types, which, alongside specifications of firm capacities, fully determines the model in
↪ \cite{azevedo2016supply}.\footnote{Formally, consider the probability distribution $\eta'$ over $\Theta$ such that for $A\subseteq \Theta$,
\begin{equation}
    \eta'(A) = \int_\RR \Pr[\theta(v)\in A]\,d\eta(v).
\end{equation}}

\paragraph{}
A \textit{matching} is a function $\mu: \Theta \rightarrow \colleges\cup \{\emptyset\}$ that satisfies the capacity constraints
\begin{equation}
    \int_\RR \Pr[\mu(\theta(v)) = \college]\,d\eta(v) = \frac{\totalsupply}{\numcolleges},\qquad \forall \college\in \colleges,
\end{equation}
as well as two additional technical assumptions: that $\mu^{-1}(\college)$ is measurable and that the set $\{\theta:\mu(\theta)\prec^\theta \college\}$ is
↪ open.\footnote{We remark that matchings are not typically required to fill capacities (i.e., the equality above is typically an inequality), but in our
↪ setting, since there is limited total capacity and applicants all prefer being matched to being unmatched, we are only concerned with matchings that
↪ fill capacity.}

Then $\mu(\theta(v))$ is a random variable indicating where an applicant with value $v$ is matched, with $\mu(\theta(v)) = \emptyset$ indicating that the
↪ applicant is not matched. When it is clear what $\theta$ is, we will abuse notation to write $\mu(v)$ instead of $\mu(\theta(v))$ to denote the random
↪ variable representing where an applicant with value $v$ is matched. Notice that the matching of an applicant depends only on their random type,
↪ reflecting the fact that their matching outcome depends directly on the scores they receive at each firm rather than their true value.

A matching is \textit{stable} if there does not exist a pair $\theta\in \Theta, \college\in \colleges$ such that $\college\succ^\theta \mu(\theta)$, and
↪ $e^\theta_\college > e^{\theta'}_{\college}$ for some $\theta'\in \mu^{-1}(\college)$. In other words, there is no applicant-firm pair that would
↪ jointly prefer to defect from the current matching to be matched with each other.

\subsection{A cutoff characterization of stable matchings}
A benefit of this continuum model is that stable matchings are characterized by a tractable cutoff structure \citep{azevedo2016supply}. Given a vector of
↪ \textit{cutoffs} $P = (P_1, \cdots, P_\numcolleges),$ an applicant with type $\theta$ can \textit{afford} firm $\college$ if $e^\theta_\college \ge
↪ P_\college$. The applicant's \textit{demand} at $P$ is $D^\theta(P)$, the applicant's most preferred firm (according to $\succ^\theta$) among those
↪ they can afford. If they cannot afford any firm, $D^\theta(P)=\emptyset$.
The \textit{aggregate demand} of a firm $\college$ at $P$ is
\begin{equation}
    D^\college(P) := \int_\RR \Pr[D^{\theta(v)}(P) = \college]\,d\eta(v),
\end{equation}
the measure of applicants who demand it.

A vector of cutoffs $P$ is \textit{market clearing} if and only if
\begin{equation}
    D^\college(P) = \frac{\totalsupply}{\numcolleges},\qquad \forall \college\in \numcolleges.\footnote{Again, as in the above footnotes, the standard
    ↪ definition in \cite{azevedo2016supply} would allow for an inequality here if no applicant would prefer to take the empty spots of a firm. This is not
    ↪ possible in our model, so we simplify our definition.}
\end{equation}

[Next verbatim source excerpt]


\subsection{Instantiating monoculture and polyculture}
We instantiate our model by specifying $\theta(v)$ for monoculture and polyculture. In what follows, we consider a fixed \textit{noise distribution} $\DD$.

\paragraph{Monoculture.} In monoculture, we take the random variable
\begin{equation}
    \theta_{\mono}(v) := (\succ, (v + X, v + X, \cdots, v + X)),
\end{equation}
where $\succ$ is drawn uniformly at random from $\mathcal{R}$ and $X\sim \DD.$ In other words, applicants have preferences distributed uniformly at
↪ random, and each applicant has a single estimated value $v+X$ that is shared across firms. This noisy estimated value equals the applicant's value
↪ perturbed by noise drawn from $\DD$.

\paragraph{Polyculture.} In polyculture, we take the random variable
\begin{equation}
    \theta_{\poly}(v) := (\succ, (v + X_1, v + X_2, \cdots, v + X_\numcolleges)),
\end{equation}
where $\succ$ is drawn uniformly at random from $\mathcal{R}$ and $X_1,X_2,\cdots, X_\numcolleges \sim \DD$ are i.i.d.. Again, applicants have
↪ preferences distributed uniformly at random, but now each applicant has a distinct, independently drawn estimated value at each firm.

\paragraph{} For expository purposes, we instantiate our model with a fixed noise distribution $\DD$ shared across both monoculture and polyculture.
↪ However, as will often be obvious, this assumption is not necessary for many of our results. In some cases, (primarily, in Theorem 2) it does allow us
↪ to compare monoculture and polyculture \textit{ceteris paribus}.
```

Lean-expanded paper semantic target

```
type:
{College : Type v} →
  [inst : Fintype College] →
    PG23MonocultureMatching.PG23Matching College → PG23MonocultureMatching.PG23ApplicantType College → College → Prop

value:
fun {College : Type v} [Fintype College] (matching : PG23MonocultureMatching.PG23Matching College)
    (theta : PG23MonocultureMatching.PG23ApplicantType College) (c : College) =>
  PG23MonocultureMatching.pg23SourcePrefers theta (some c) (matching theta) ∧
    ∃ (theta' : PG23MonocultureMatching.PG23ApplicantType College),
      matching theta' = some c ∧
        PG23MonocultureMatching.pg23SourceScore theta' c < PG23MonocultureMatching.pg23SourceScore theta c
```

Direct retained dependencies

- PG23MonocultureMatching.PG23ApplicantType
- PG23MonocultureMatching.PG23Matching
- PG23MonocultureMatching.pg23SourcePrefers
- PG23MonocultureMatching.pg23SourceScore

Recorded source-to-declaration screening **Verdict:** matches

Reason: The predicate states that an applicant prefers c to the current assignment and has higher score at c than an applicant currently assigned there, exactly the source blocking-pair condition.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

PG23MonocultureMatching.pg23SourceMatchingFeasible

Paper-local declaration:
PG23MonocultureMatching.pg23SourceMatchingFeasible
Source locator: cited publication, lines 2-34

Verbatim paper-source connection

```
\section{Model and Preliminaries}
Our model builds on the model of \cite{azevedo2016supply}, in which there is a continuum of applicants and a finite number of firms (analogously,
↪ colleges). An applicant is identified by their true \textit{value}, which is distributed on $\RR$ according to a probability measure $\eta$. Let $\colleges
↪ = \{1, 2, \cdots, \numcolleges\}$ be the set of all firms. Firms have total capacity $\totalsupply < 1$ (so there are fewer positions than applicants)
↪ and each firm has the same capacity $\frac{\totalsupply}{\numcolleges}.$

An applicant with value $v$ is associated with an independent random variable $\theta(v)$, characterizing their preferences over firms and firms'
↪ estimated values for that applicant. The realization of $\theta(v)$ is their \textit{type}, which lies in $\Theta$. $\Theta := \mathcal{R} \times
↪ \RR^\numcolleges$ is the set of applicant types, where $\mathcal{R}$ is the set of preference orderings over the firms. If $\theta(v) = \theta =
↪ (\succ^\theta, e^\theta),$ then $\succ^\theta$ is the preference ordering of $v$ over firms and $e^\theta_\college$ is the \textit{estimated value} of
↪ the applicant at firm $\college$.

In the model, $\theta(v)$ plays a central role. First, it is used to model the process of preference approximation for firms. Firms all prefer applicants with
↪ higher true values, but in practice, rank applicants based on the estimated values given by $\theta(v)$. Second, it connects our model with that of
↪ \cite{azevedo2016supply}. In particular, the distribution $\eta$ over values in $\RR$ together with an appropriate choice of $\theta(v)$ induces a
↪ distribution over applicant types, which, alongside specifications of firm capacities, fully determines the model in
↪ \cite{azevedo2016supply}.\footnote{Formally, consider the probability distribution $\eta'$ over $\Theta$ such that for $A\subseteq \Theta$,
\begin{equation}
    \eta'(A) = \int_\RR \Pr[\theta(v)\in A]\,d\eta(v).
\end{equation}}

\paragraph{}
A \textit{matching} is a function $\mu: \Theta \rightarrow \colleges\cup \{\emptyset\}$ that satisfies the capacity constraints
\begin{equation}
    \int_\RR \Pr[\mu(\theta(v)) = \college]\,d\eta(v) = \frac{\totalsupply}{\numcolleges},\qquad \forall \college\in \colleges,
\end{equation}
as well as two additional technical assumptions: that $\mu^{-1}(\college)$ is measurable and that the set $\{\theta:\mu(\theta)\prec^\theta \college\}$ is
↪ open.\footnote{We remark that matchings are not typically required to fill capacities (i.e., the equality above is typically an inequality), but in our
↪ setting, since there is limited total capacity and applicants all prefer being matched to being unmatched, we are only concerned with matchings that
↪ fill capacity.}

Then $\mu(\theta(v))$ is a random variable indicating where an applicant with value $v$ is matched, with $\mu(\theta(v)) = \emptyset$ indicating that the
↪ applicant is not matched. When it is clear what $\theta$ is, we will abuse notation to write $\mu(v)$ instead of $\mu(\theta(v))$ to denote the random
↪ variable representing where an applicant with value $v$ is matched. Notice that the matching of an applicant depends only on their random type,
↪ reflecting the fact that their matching outcome depends directly on the scores they receive at each firm rather than their true value.

A matching is \textit{stable} if there does not exist a pair $\theta\in \Theta, \college\in \colleges$ such that $\college\succ^\theta \mu(\theta)$, and
↪ $e^\theta_\college > e^{\theta'}_{\college}$ for some $\theta'\in \mu^{-1}(\college)$. In other words, there is no applicant-firm pair that would
↪ jointly prefer to defect from the current matching to be matched with each other.

\subsection{A cutoff characterization of stable matchings}
A benefit of this continuum model is that stable matchings are characterized by a tractable cutoff structure \citep{azevedo2016supply}. Given a vector of
↪ \textit{cutoffs} $P = (P_1, \cdots, P_\numcolleges),$ an applicant with type $\theta$ can \textit{afford} firm $\college$ if $e^\theta_\college \ge
↪ P_\college$. The applicant's \textit{demand} at $P$ is $D^\theta(P)$, the applicant's most preferred firm (according to $\succ^\theta$) among those
↪ they can afford. If they cannot afford any firm, $D^\theta(P)=\emptyset$.
The \textit{aggregate demand} of a firm $\college$ at $P$ is
\begin{equation}
    D^\college(P) := \int_\RR \Pr[D^{\theta(v)}(P) = \college]\,d\eta(v),
\end{equation}
the measure of applicants who demand it.

A vector of cutoffs $P$ is \textit{market clearing} if and only if
\begin{equation}
    D^\college(P) = \frac{\totalsupply}{\numcolleges},\qquad \forall \college\in \numcolleges.\footnote{Again, as in the above footnotes, the standard
    ↪ definition in \cite{azevedo2016supply} would allow for an inequality here if no applicant would prefer to take the empty spots of a firm. This is not
    ↪ possible in our model, so we simplify our definition.}
\end{equation}

[Next verbatim source excerpt]


\subsection{Instantiating monoculture and polyculture}
We instantiate our model by specifying $\theta(v)$ for monoculture and polyculture. In what follows, we consider a fixed \textit{noise distribution} $\DD$.

\paragraph{Monoculture.} In monoculture, we take the random variable
\begin{equation}
    \theta_{\mono}(v) := (\succ, (v + X, v + X, \cdots, v + X)),
\end{equation}
where $\succ$ is drawn uniformly at random from $\mathcal{R}$ and $X\sim \DD.$ In other words, applicants have preferences distributed uniformly at
↪ random, and each applicant has a single estimated value $v+X$ that is shared across firms. This noisy estimated value equals the applicant's value
↪ perturbed by noise drawn from $\DD$.

\paragraph{Polyculture.} In polyculture, we take the random variable
\begin{equation}
    \theta_{\poly}(v) := (\succ, (v + X_1, v + X_2, \cdots, v + X_\numcolleges)),
\end{equation}
where $\succ$ is drawn uniformly at random from $\mathcal{R}$ and $X_1,X_2,\cdots, X_\numcolleges \sim \DD$ are i.i.d.. Again, applicants have
↪ preferences distributed uniformly at random, but now each applicant has a distinct, independently drawn estimated value at each firm.

\paragraph{} For expository purposes, we instantiate our model with a fixed noise distribution $\DD$ shared across both monoculture and polyculture.
↪ However, as will often be obvious, this assumption is not necessary for many of our results. In some cases, (primarily, in Theorem 2) it does allow us
↪ to compare monoculture and polyculture \textit{ceteris paribus}.
```

Lean-expanded paper semantic target

type:
```
{College : Type v} →
  [inst : Fintype College] →
    MeasureTheory.Measure (PG23MonocultureMatching.PG23ApplicantType College) →
      (College → ℝ) → PG23MonocultureMatching.PG23Matching College → Prop
```

value:
```
fun {College : Type v} [Fintype College]
    (typeLaw : MeasureTheory.Measure (PG23MonocultureMatching.PG23ApplicantType College)) (capacity : College → ℝ)
    (matching : PG23MonocultureMatching.PG23Matching College) =>
  (∀ (c : College), typeLaw.real (PG23MonocultureMatching.pg23MatchingCollegeSet matching c) = capacity c) ∧
    (∀ (c : College), MeasurableSet (PG23MonocultureMatching.pg23MatchingCollegeSet matching c)) ∧
      ∀ (c : College),
        IsOpen
          {theta : PG23MonocultureMatching.PG23ApplicantType College |
            PG23MonocultureMatching.pg23SourcePrefers theta (some c) (matching theta)}
```

Direct retained dependencies

- AL16SupplyDemandMatching.instMeasurableSpaceAL16Ranking
- PG23MonocultureMatching.PG23ApplicantType
- PG23MonocultureMatching.PG23Matching
- PG23MonocultureMatching.PG23Ranking
- PG23MonocultureMatching.pg23MatchingCollegeSet
- PG23MonocultureMatching.pg23SourcePrefers

Recorded source-to-declaration screening **Verdict:** matches

Reason: The capacity bound, measurability of matching fibers, and openness of strict-improvement sets are precisely the source feasibility requirements for continuum matchings.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

PG23MonocultureMatching.pg23SourceRank

Paper-local declaration:
PG23MonocultureMatching.pg23SourceRank
Source locator: cited publication, lines 600-600

Verbatim paper-source connection

Proceeding to the statement of the result, we define $\Rank_{\theta(v)}(\college) := |\{\college':\college'\succeq^{\theta(v)} \college\}|$ to be the rank of an applicant's matched firm. (We let $\Rank_{\theta(v)}(\emptyset) = 0$.) Then $\Rank_{v}(\mu(v))$ is a random variable giving the rank of the (random) firm an applicant of value v is matched to under μ. Also recall that (X_-, X_+) is the support of the noise distribution $\DD$.

Lean-expanded paper semantic target

type:
{College : Type v} → [inst : Fintype College] → PG23MonocultureMatching.PG23ApplicantType College → College → ℕ

value:
fun {College : Type v} [Fintype College] (theta : PG23MonocultureMatching.PG23ApplicantType College) (c : College) =>
  ↑(theta.1 c)

Direct retained dependencies

- PG23MonocultureMatching.PG23ApplicantType
- PG23MonocultureMatching.PG23Ranking

Recorded source-to-declaration screening **Verdict:** matches

Reason: The natural-valued position is the source one-based college rank minus one; every displayed comparison, top-choice case, and top-k application use preserves that translation, while unmatched outcomes are represented separately by none.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

PG23MonocultureMatching.pg23SourceScoreLevelNull

Paper-local declaration:
PG23MonocultureMatching.pg23SourceScoreLevelNull
Source locator: cited publication, lines 433-435

Verbatim paper-source connection

college. We make the following assumption on η, which corresponds to colleges having
strict preferences over students in the discrete model.
Assumption 1. (Strict Preferences) Every college's indifference curves have ηmeasure 0. That is, for any college c and real number x we have η({θ : eθc =
↪ x}) = 0.

Lean-expanded paper semantic target

```
type:
{College : Type v} →
  [inst : Fintype College] → MeasureTheory.Measure (PG23MonocultureMatching.PG23ApplicantType College) → Prop

value:
fun {College : Type v} [Fintype College]
    (typeLaw : MeasureTheory.Measure (PG23MonocultureMatching.PG23ApplicantType College)) =>
  ∀ (c : College) (x : ℝ),
    typeLaw
        {theta : PG23MonocultureMatching.PG23ApplicantType College |
          PG23MonocultureMatching.pg23SourceScore theta c = x} =
      0
```

Direct retained dependencies

- AL16SupplyDemandMatching.instMeasurableSpaceAL16Ranking
- PG23MonocultureMatching.PG23ApplicantType
- PG23MonocultureMatching.PG23Ranking
- PG23MonocultureMatching.pg23SourceScore

Recorded source-to-declaration screening **Verdict:** matches

Reason: Zero mass for every college-specific score level set is exactly the cited continuum no-atoms condition used by the displayed source-market results.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

PG23MonocultureMatching.pg23SourceStableMatching

Paper-local declaration:
PG23MonocultureMatching.pg23SourceStableMatching
Source locator: cited publication, lines 2-34

Verbatim paper-source connection

```
\section{Model and Preliminaries}
Our model builds on the model of \cite{azevedo2016supply}, in which there is a continuum of applicants and a finite number of firms (analogously,
↪ colleges). An applicant is identified by their true \textit{value}, which is distributed on $\RR$ according to a probability measure $\eta$. Let $\colleges
↪ = \{1, 2, \cdots, \numcolleges\}$ be the set of all firms. Firms have total capacity $\totalsupply < 1$ (so there are fewer positions than applicants)
↪ and each firm has the same capacity $\frac{\totalsupply}{\numcolleges}.$

An applicant with value $v$ is associated with an independent random variable $\theta(v)$, characterizing their preferences over firms and firms'
↪ estimated values for that applicant. The realization of $\theta(v)$ is their \textit{type}, which lies in $\Theta$. $\Theta := \mathcal{R} \times
↪ \RR^\numcolleges$ is the set of applicant types, where $\mathcal{R}$ is the set of preference orderings over the firms. If $\theta(v) = \theta =
↪ (\succ^\theta, e^\theta),$ then $\succ^\theta$ is the preference ordering of $v$ over firms and $e^\theta_\college$ is the \textit{estimated value} of
↪ the applicant at firm $\college$.

In the model, $\theta(v)$ plays a central role. First, it is used to model the process of preference approximation for firms. Firms all prefer applicants with
↪ higher true values, but in practice, rank applicants based on the estimated values given by $\theta(v)$. Second, it connects our model with that of
↪ \cite{azevedo2016supply}. In particular, the distribution $\eta$ over values in $\RR$ together with an appropriate choice of $\theta(v)$ induces a
↪ distribution over applicant types, which, alongside specifications of firm capacities, fully determines the model in
↪ \cite{azevedo2016supply}.\footnote{Formally, consider the probability distribution $\eta'$ over $\Theta$ such that for $A\subseteq \Theta$,
\begin{equation}
    \eta'(A) = \int_\RR \Pr[\theta(v)\in A]\,d\eta(v).
\end{equation}}

\paragraph{}
A \textit{matching} is a function $\mu: \Theta \rightarrow \colleges\cup \{\emptyset\}$ that satisfies the capacity constraints
\begin{equation}
    \int_\RR \Pr[\mu(\theta(v)) = \college]\,d\eta(v) = \frac{\totalsupply}{\numcolleges},\qquad \forall \college\in \colleges,
\end{equation}
as well as two additional technical assumptions: that $\mu^{-1}(\college)$ is measurable and that the set $\{\theta:\mu(\theta)\prec^\theta \college\}$ is
↪ open.\footnote{We remark that matchings are not typically required to fill capacities (i.e., the equality above is typically an inequality), but in our
↪ setting, since there is limited total capacity and applicants all prefer being matched to being unmatched, we are only concerned with matchings that
↪ fill capacity.}

Then $\mu(\theta(v))$ is a random variable indicating where an applicant with value $v$ is matched, with $\mu(\theta(v)) = \emptyset$ indicating that the
↪ applicant is not matched. When it is clear what $\theta$ is, we will abuse notation to write $\mu(v)$ instead of $\mu(\theta(v))$ to denote the random
↪ variable representing where an applicant with value $v$ is matched. Notice that the matching of an applicant depends only on their random type,
↪ reflecting the fact that their matching outcome depends directly on the scores they receive at each firm rather than their true value.

A matching is \textit{stable} if there does not exist a pair $\theta\in \Theta, \college\in \colleges$ such that $\college\succ^\theta \mu(\theta)$, and
↪ $e^\theta_\college > e^{\theta'}_{\college}$ for some $\theta'\in \mu^{-1}(\college)$. In other words, there is no applicant-firm pair that would
↪ jointly prefer to defect from the current matching to be matched with each other.

\subsection{A cutoff characterization of stable matchings}
A benefit of this continuum model is that stable matchings are characterized by a tractable cutoff structure \citep{azevedo2016supply}. Given a vector of
↪ \textit{cutoffs} $P = (P_1, \cdots, P_\numcolleges),$ an applicant with type $\theta$ can \textit{afford} firm $\college$ if $e^\theta_\college \ge
↪ P_\college$. The applicant's \textit{demand} at $P$ is $D^\theta(P)$, the applicant's most preferred firm (according to $\succ^\theta$) among those
↪ they can afford. If they cannot afford any firm, $D^\theta(P)=\emptyset$.
The \textit{aggregate demand} of a firm $\college$ at $P$ is
\begin{equation}
    D^\college(P) := \int_\RR \Pr[D^{\theta(v)}(P) = \college]\,d\eta(v),
\end{equation}
the measure of applicants who demand it.

A vector of cutoffs $P$ is \textit{market clearing} if and only if
\begin{equation}
    D^\college(P) = \frac{\totalsupply}{\numcolleges},\qquad \forall \college\in \numcolleges.\footnote{Again, as in the above footnotes, the standard
    ↪ definition in \cite{azevedo2016supply} would allow for an inequality here if no applicant would prefer to take the empty spots of a firm. This is not
    ↪ possible in our model, so we simplify our definition.}
\end{equation}

[Next verbatim source excerpt]


\subsection{Instantiating monoculture and polyculture}
We instantiate our model by specifying $\theta(v)$ for monoculture and polyculture. In what follows, we consider a fixed \textit{noise distribution} $\DD$.

\paragraph{Monoculture.} In monoculture, we take the random variable
\begin{equation}
    \theta_{\mono}(v) := (\succ, (v + X, v + X, \cdots, v + X)),
\end{equation}
where $\succ$ is drawn uniformly at random from $\mathcal{R}$ and $X\sim \DD.$ In other words, applicants have preferences distributed uniformly at
↪ random, and each applicant has a single estimated value $v+X$ that is shared across firms. This noisy estimated value equals the applicant's value
↪ perturbed by noise drawn from $\DD$.

\paragraph{Polyculture.} In polyculture, we take the random variable
\begin{equation}
    \theta_{\poly}(v) := (\succ, (v + X_1, v + X_2, \cdots, v + X_\numcolleges)),
\end{equation}
where $\succ$ is drawn uniformly at random from $\mathcal{R}$ and $X_1,X_2,\cdots, X_\numcolleges \sim \DD$ are i.i.d.. Again, applicants have
↪ preferences distributed uniformly at random, but now each applicant has a distinct, independently drawn estimated value at each firm.

\paragraph{} For expository purposes, we instantiate our model with a fixed noise distribution $\DD$ shared across both monoculture and polyculture.
↪ However, as will often be obvious, this assumption is not necessary for many of our results. In some cases, (primarily, in Theorem 2) it does allow us
↪ to compare monoculture and polyculture \textit{ceteris paribus}.
```

Lean-expanded paper semantic target

```
type:
{College : Type v} →
  [inst : Fintype College] →
    MeasureTheory.Measure (PG23MonocultureMatching.PG23ApplicantType College) →
      (College → ℝ) → PG23MonocultureMatching.PG23Matching College → Prop

value:
fun {College : Type v} [Fintype College]
    (typeLaw : MeasureTheory.Measure (PG23MonocultureMatching.PG23ApplicantType College)) (capacity : College → ℝ)
    (matching : PG23MonocultureMatching.PG23Matching College) =>
  PG23MonocultureMatching.pg23SourceMatchingFeasible typeLaw capacity matching ∧
    ∀ (theta : PG23MonocultureMatching.PG23ApplicantType College) (c : College),
      ¬PG23MonocultureMatching.pg23SourceMatchingBlocks matching theta c
```

Direct retained dependencies

- AL16SupplyDemandMatching.instMeasurableSpaceAL16Ranking
- PG23MonocultureMatching.PG23ApplicantType
- PG23MonocultureMatching.PG23Matching
- PG23MonocultureMatching.PG23Ranking
- PG23MonocultureMatching.pg23SourceMatchingBlocks
- PG23MonocultureMatching.pg23SourceMatchingFeasible

Recorded source-to-declaration screening **Verdict:** matches

Reason: Conjoining source feasibility with absence of source blocking pairs is exactly the source definition of a stable matching.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

PG23MonocultureMatching.source_assumption_value_threshold

Paper-local declaration:
PG23MonocultureMatching.source_assumption_value_threshold
Source locator: cited publication, lines 537-537

Verbatim paper-source connection

Define v_S to be the unique value such that $\eta((v_S, \infty)) = S$, i.e., the threshold at which the mass of students with a value above v_S equals the overall firm capacity. Then note that in the firm-optimal matching, only applicants with value greater than v_S are matched. We may now state our first main result.

Lean-expanded paper semantic target

type:
```
MeasureTheory.Measure ℝ → ℝ → ℝ → Prop
```

value:
```
fun (η : MeasureTheory.Measure ℝ) (capacity threshold : ℝ) =>
  AppliedModelingLib.Probability.UpperTailThresholdCertificate η capacity threshold
```

Direct retained dependencies

- AppliedModelingLib.Probability.UpperTailThresholdCertificate

Recorded source-to-declaration screening **Verdict:** matches

Reason: The certificate records Pr[V > vS] = S; in every displayed paper use, connected support together with 0 < S < 1 makes a solution to this tail equation unique, so the routed object has the source threshold meaning.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

Source claims

Review row 1

Source locator: cited publication, lines 151-174

Verbatim source input

```
\section{Preliminary Results}

\subsection{Observations About Measures with Connected Support}

To aid subsequent proofs, we recall our assumption on the probability measure $\eta$ (of student values) and the probability distribution $\DD$ (the noise
↪  distribution); namely, that they have connected support. We then make two useful observations.

Let $\pi$ be the probability measure associated with $\DD$. Then we make the assumption that both $\eta$ and $\pi$ have \textit{connected support},
↪  where we mean that the smallest closed set of measure $1$ is an interval. Let these respective intervals be $[V_-, V_+]$ and $[X_-, X_+]$. (Note that
↪  $V_-, X_-$ may be equal to $-\infty$ and $V_+, X_+$ may be equal to $\infty$.) In what follows we let $V$ and $X$ be random variables distributed
↪  according to the probability measures $\eta$ and $\pi$, and let $F_V$ and $F_X$ denote their cdfs.

\begin{proposition}\label{prop:cdf-increasing}
The following hold:
\begin{itemize}
    \item[(i)] $F_V$ is strictly increasing on the interval $(V_-, V_+),$ and $F_V(v)\in (0,1)$ when $v\in (V_-, V_+).$
    \item[(ii)] $F_X$ is strictly increasing on the interval $(X_-, X_+),$ and $F_X(x)\in (0,1)$ when $x\in (X_-, X_+).$
    \item[(iii)] For all integers $\numcolleges\ge 1,$ $F_X^\numcolleges$ is strictly increasing on the interval $(X_-, X_+),$ and $F_X^\numcolleges(x)\in
    ↪  (0,1)$ when $x\in (X_-, X_+).$
\end{itemize}
\end{proposition}

\begin{proof}
We show the result for $F_V$; the result for $F_X$ is (clearly) analogous.

First suppose that $F_V$ were not strictly increasing on $V_-, V_+$, such that there exists $v_1, v_2$ such that $V_- < v_1 < v_2 < V_+$ and $F_V(v_1) =
↪  F_V(v_2)$. Then $\eta((v_1, v_2)) = F_V(v_2) - F_V(v_1) = 0.$ This implies that $\eta([V_-, V_+]\setminus [v_1, v_2]) = 1 - 0 = 1,$ which contradicts
↪  the assumption that $[V_-, V_+]$ is the smallest closed set of $\eta$-measure $1$.

To show that $F_V(v)\in (0,1)$ when $v\in (V_-, V_+)$, it suffices to show that $F_V(V_-)=0$ and $F_V(V_+)=1.$ Assume otherwise, such that $F_V(V_-) >
↪  0$ or $F_V(V_+) < 1.$ This would imply that $\eta([V_-, V_+]) < 1$, which gives a contradiction.
\end{proof}
```

Expanded PaperInterface specification

```
∀ (law : MeasureTheory.Measure ℝ) [MeasureTheory.IsProbabilityMeasure law],
  PG23MonocultureMatching.pg23ConnectedSupport law →
    StrictMonoOn (AppliedModelingLib.Probability.lowerCDFMass law) (interior law.support) ∧
      (∀ x ∈ interior law.support, AppliedModelingLib.Probability.lowerCDFMass law x ∈ Set.Ioo 0 1) ∧
        ∀ (n : ℕ),
          0 < n →
            StrictMonoOn (fun (x : ℝ) => AppliedModelingLib.Probability.lowerCDFMass law x ^ n) (interior law.support) ∧
              ∀ x ∈ interior law.support, AppliedModelingLib.Probability.lowerCDFMass law x ^ n ∈ Set.Ioo 0 1
```

Retained governing declarations

- AppliedModelingLib.Probability.lowerCDFMass
- PG23MonocultureMatching.pg23ConnectedSupport

Lean-elaborated claim roles

```
1. parameter: MeasureTheory.Measure ℝ
2. assumption: MeasureTheory.IsProbabilityMeasure law
3. assumption: PG23MonocultureMatching.pg23ConnectedSupport law
4. conclusion: StrictMonoOn (AppliedModelingLib.Probability.lowerCDFMass law) (interior law.support) ∧
  (∀ x ∈ interior law.support, AppliedModelingLib.Probability.lowerCDFMass law x ∈ Set.Ioo 0 1) ∧
    ∀ (n : ℕ),
      0 < n →
        StrictMonoOn (fun (x : ℝ) => AppliedModelingLib.Probability.lowerCDFMass law x ^ n) (interior law.support) ∧
          ∀ x ∈ interior law.support, AppliedModelingLib.Probability.lowerCDFMass law x ^ n ∈ Set.Ioo 0 1
```

Lean proof endpoint (built separately)

PG23MonocultureMatching.PaperInterface.review_proposition_cdfIncreasing_connectedSupportSpec_proof

Recorded source-to-Spec screening **Verdict:** matches

Reason: The target states strict monotonicity and interior (0,1) range of the lower CDF on the interior of a connected probability support, and repeats both properties for every positive integer power. Instantiating the generic law with the source value or noise distribution gives all three source clauses.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 2

Source locator: cited publication, lines 176-185

Verbatim source input

```
\begin{proposition}\label{prop:nonzero-measure}
   Let $I$ be an interval such that
   \begin{equation}
      I \cap (V_-, V_+) \neq \emptyset.
   \end{equation}
   Then $\eta(I) > 0.$
\end{proposition}

\begin{proof}
   There exists $a,b$ such that $V_-\le a<b\le V_+$ such that $(a,b)\subseteq I \cap (V_-, V_+)$. Therefore, $\eta(I) > \eta((a,b)).$ We have that
   ↪  $\eta((a,b)) = F_V(b) - F_V(a) > 0$, where we used that $F_V$ is strictly increasing on $[V_-, V_+]$, as shown in \Cref{prop:cdf-increasing}. This
   ↪  gives the result.
```

Formalized review target

The archival source above is not asserted equivalent to this formalized target.

Every nonempty open real interval whose intersection with the interior of a probability law's support is nonempty has positive law mass.

Archival source anchor: cited publication, lines 176-185

Expanded PaperInterface specification

```
∀ (law : MeasureTheory.Measure ℝ) [MeasureTheory.IsProbabilityMeasure law] {a b : ℝ},
  (Set.Ioo a b ∩ interior law.support).Nonempty → 0 < law.real (Set.Ioo a b)
```

Lean-elaborated claim roles

1. parameter: MeasureTheory.Measure ℝ
2. assumption: MeasureTheory.IsProbabilityMeasure law
3. parameter: ℝ
4. parameter: ℝ
5. assumption: (Set.Ioo a b ∩ interior law.support).Nonempty
6. conclusion: 0 < law.real (Set.Ioo a b)

Lean proof endpoint (built separately)

PG23MonocultureMatching.PaperInterface.review_proposition_nonzeroMeasure_openIntervalSpec_proof

Recorded source-to-Spec screening

Verdict: matches_approved_corrected_target
Reason: The approved correction restricts the claim to nonempty open intervals meeting the support interior, and the expanded Lean target states exactly that such an Ioo interval has strictly positive probability mass.
Reviewer check: ☐ Matches formalized target
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

Review row 3

Source locator: cited publication, lines 81-95

Verbatim source input

```
\begin{proposition}\label{prop:prob-match}
    For all $v \in \RR$,
    \begin{enumerate}
    \item[(i)] the probability an applicant with value $v$ is matched under monoculture is
\begin{equation}\label{eq:match-mono}
    \Pr[\mu_{\mono}(v) \in \colleges] = \Pr[v + X > P_{\mono}],
\end{equation}
where $X\sim \DD,$ and
\item[(ii)] the probability an applicant with value $v$ is matched under polyculture is
\begin{equation}\label{eq:match-poly}
        \Pr[\mu_{\poly}(v) \in \colleges] = \Pr[v + \max_{\college\in \colleges} X_\college > P_{\poly}],
    \end{equation}
    where $X_1,\cdots, X_\numcolleges\simiid \DD.$
\end{enumerate}
\end{proposition}

[Next verbatim source excerpt]


\paragraph{}
This cutoff structure allows for clean analysis of stable matchings. Before moving to our main results, we make some basic observations that follow. The
↪ next proposition follows directly from the Supply and Demand Lemma (\Cref{lem:supply-demand}) and the Equal Cutoffs Lemma
↪ (\Cref{prop:equal-cutoffs}).
```

Formalized review target

The archival source above is not asserted equivalent to this formalized target.

For an atomless noise law, the literal monoculture and iid polyculture matching probabilities at shared cutoffs equal the corresponding strict upper-tail
↪ probabilities.

Archival source anchor: cited publication, lines 81-95

Expanded PaperInterface specification

```
∀ {n : ℕ} [inst : NeZero n] (noiseLaw : MeasureTheory.Measure ℝ) [MeasureTheory.IsProbabilityMeasure noiseLaw]
  [MeasureTheory.NoAtoms noiseLaw] (ranking : PG23MonocultureMatching.PG23Ranking (Fin n)) (v Pmono Ppoly : ℝ),
  noiseLaw.real
      {noise : ℝ |
        ∃ (c : Fin n),
          PG23MonocultureMatching.pg23ActiveSourceChoice Finset.univ (fun (x : Fin n) => Pmono)
              (PG23MonocultureMatching.pg23MonocultureType v noise ranking) =
            some c} =
    AppliedModelingLib.Probability.upperTailMass noiseLaw (Pmono - v) ∧
  (MeasureTheory.Measure.pi fun (x : Fin n) => noiseLaw).real
      {noise : Fin n → ℝ |
        ∃ (c : Fin n),
          PG23MonocultureMatching.pg23ActiveSourceChoice Finset.univ (fun (x : Fin n) => Ppoly)
              (PG23MonocultureMatching.pg23PolycultureType v ranking noise) =
            some c} =
    (PG23MonocultureMatching.pg23PolycultureMaxNoiseLaw noiseLaw).real (Set.Ioi (Ppoly - v))
```

Retained governing declarations

- AppliedModelingLib.Probability.upperTailMass
- PG23MonocultureMatching.PG23Ranking
- PG23MonocultureMatching.pg23ActiveSourceChoice
- PG23MonocultureMatching.pg23MonocultureType
- PG23MonocultureMatching.pg23PolycultureMaxNoiseLaw
- PG23MonocultureMatching.pg23PolycultureType

Lean-elaborated claim roles

1. parameter: ℕ
2. assumption: NeZero n
3. parameter: MeasureTheory.Measure ℝ
4. assumption: MeasureTheory.IsProbabilityMeasure noiseLaw
5. assumption: MeasureTheory.NoAtoms noiseLaw
6. parameter: PG23MonocultureMatching.PG23Ranking (Fin n)
7. parameter: ℝ
8. parameter: ℝ
9. parameter: ℝ
10. conclusion: noiseLaw.real
      {noise : ℝ |
        ∃ (c : Fin n),

```
      PG23MonocultureMatching.pg23ActiveSourceChoice Finset.univ (fun (x : Fin n) => Pmono)
        (PG23MonocultureMatching.pg23MonocultureType v noise ranking) =
      some c} =
  AppliedModelingLib.Probability.upperTailMass noiseLaw (Pmono - v) ∧
(MeasureTheory.Measure.pi fun (x : Fin n) => noiseLaw).real
  {noise : Fin n → ℝ |
   ∃ (c : Fin n),
    PG23MonocultureMatching.pg23ActiveSourceChoice Finset.univ (fun (x : Fin n) => Ppoly)
      (PG23MonocultureMatching.pg23PolycultureType v ranking noise) =
    some c} =
 (PG23MonocultureMatching.pg23PolycultureMaxNoiseLaw noiseLaw).real (Set.Ioi (Ppoly - v))
```

Lean proof endpoint (built separately)

PG23MonocultureMatching.PaperInterface.review_proposition_probabilityFormulaSpec_proof

Recorded source-to-Spec screening **Verdict:** matches_approved_corrected_target

Reason: For an atomless noise law, the target equates the literal weak-choice match events with the strict upper-tail event for common monoculture noise and the strict upper tail of the iid polyculture maximum. This is precisely the weak-to-strict conversion and both probability formulas authorized by PG23-CONTINUOUS-REGULARITY-01.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 4

Source locator: cited publication, lines 100-105

Verbatim source input

```
\begin{corollary}[Lower Cutoff Under Monoculture]\label{cor:cutoff-inequality}
When $\numcolleges \ge 2,$ the shared cutoff in an economy is lower under monoculture than under polyculture:
   \begin{equation}
      P_{\mono} < P_{\poly}.
   \end{equation}
\end{corollary}

[Next verbatim source excerpt]

As a consequence of the above proof,
\begin{equation}\label{eq:interval-mono}
   \eta((P_{\mono} - X_+, P_{\mono} - X_-)) > 0
\end{equation}
and
\begin{equation}\label{eq:interval-poly}
   \eta((P_{\poly} - X_+, P_{\poly} - X_-)) > 0,
\end{equation}
which will be useful in subsequent proofs.

\begin{proof}[\textbf{Proof of \Cref{cor:cutoff-inequality}}]
   Noting that the total measure of students matched is the same under both monoculture and polyculture (equal to $\totalsupply$), we have, using
   ↪  \eqref{eq:match-mono} and \eqref{eq:match-poly} that
   \begin{equation}
      \int_{\RR} \Pr[v + X > P_{\mono}]\,d\eta(v) = \totalsupply = \int_{\RR} \Pr[v + X^{(\numcolleges)} > P_{\poly}]\,d\eta(v).
   \end{equation}
   Assume for the sake of contradiction that $P_{\mono}\ge P_{\poly}.$ To give the desired contradiction, we will show that this implies
   \begin{equation}
      \int_{\RR} \Pr[v + X > P_{\mono}]\,d\eta(v) < \int_{\RR} \Pr[v + X^{(\numcolleges)} > P_{\poly}]\,d\eta(v).
   \end{equation}
   Clearly,
   \begin{equation}
      \Pr[v + X > P_{\mono}] \le \Pr[v + X^{(\numcolleges)} > P_{\poly}]
   \end{equation}
   for all $v$. We now show that
   \begin{equation}
      \Pr[v + X > P_{\mono}] < \Pr[v + X^{(\numcolleges)} > P_{\poly}]
   \end{equation}
   for $v\in (P_{\poly}-X_+, P_{\poly}-X_-)$, an interval which we will show has positive $\eta$-measure. We have that
   \begin{equation}
      \Pr[v + X > P_{\mono}] \le \Pr[v + X > P_{\poly}],
   \end{equation}
   so it suffices to show that
   \begin{equation}\label{eq:water}
      \Pr[v + X > P_{\poly}] < \Pr[v + X^{(\numcolleges)} > P_{\poly}].
   \end{equation}
   We have that
   \begin{align}
      \Pr[v + X > P_{\poly}] &= 1 - F_X(P_{\poly} - v)\\
      \Pr[v + X^{(\numcolleges)} > P_{\poly}] &= 1 - F_X^\numcolleges(P_{\poly} - v).
   \end{align}
   Therefore, since $\numcolleges\ge 2$, \eqref{eq:water} holds whenever $F_X(P_{\poly} - v)\in (0,1)$, which, by \Cref{prop:cdf-increasing}, is true
   ↪  when $P_{\poly} - v \in (X_-, X_+).$ Equivalently, $v \in (P_{\poly} - X_+, P_{\poly} - X_-).$ This interval has positive measure
   ↪  \eqref{eq:interval-poly}.
\end{proof}
```

Formalized review target

The archival source above is not asserted equivalent to this formalized target.

With at least two colleges, connected-support value and noise laws, nondegenerate noise, score-level-null literal economies, and exact clearing, the
↪ shared monoculture cutoff is strictly lower than the shared polyculture cutoff.

Archival source anchor: cited publication, lines 100-105

Expanded PaperInterface specification

```
∀ {n : ℕ} [NeZero n] (valueLaw noiseLaw : MeasureTheory.Measure ℝ) [inst : MeasureTheory.IsProbabilityMeasure valueLaw]
  [inst_1 : MeasureTheory.IsProbabilityMeasure noiseLaw],
  1 < n →
    PG23MonocultureMatching.pg23ConnectedSupport valueLaw →
      PG23MonocultureMatching.pg23ConnectedSupport noiseLaw →
        (∃ (x : ℝ) (y : ℝ), x ∈ noiseLaw.support ∧ y ∈ noiseLaw.support ∧ x < y) →
          ∀ {Pmono Ppoly S : ℝ},
            0 < S ∧ S < 1 →
              (∀ (c : Fin n) (x : ℝ),
                 (PG23MonocultureMatching.pg23MonocultureTypeLaw valueLaw noiseLaw)
                    {theta : PG23MonocultureMatching.PG23ApplicantType (Fin n) |
                      PG23MonocultureMatching.pg23SourceScore theta c = x} =
                  0) →
                (∀ (c : Fin n) (x : ℝ),
                   (PG23MonocultureMatching.pg23PolycultureTypeLaw valueLaw noiseLaw)
                      {theta : PG23MonocultureMatching.PG23ApplicantType (Fin n) |
                        PG23MonocultureMatching.pg23SourceScore theta c = x} =
                    0) →
                  (PG23MonocultureMatching.pg23SourceMarketClearing
```

```
  (PG23MonocultureMatching.pg23SourceAggregateDemand
    (PG23MonocultureMatching.pg23MonocultureTypeLaw valueLaw noiseLaw))
  (fun (x : Fin n) => S / ↑(Fintype.card (Fin n))) fun (x : Fin n) => Pmono) →
(PG23MonocultureMatching.pg23SourceMarketClearing
    (PG23MonocultureMatching.pg23SourceAggregateDemand
      (PG23MonocultureMatching.pg23PolycultureTypeLaw valueLaw noiseLaw))
    (fun (x : Fin n) => S / ↑(Fintype.card (Fin n))) fun (x : Fin n) => Ppoly) →
  Pmono < Ppoly
```

Retained governing declarations

- AL16SupplyDemandMatching.instMeasurableSpaceAL16Ranking
- PG23MonocultureMatching.PG23ApplicantType
- PG23MonocultureMatching.PG23Ranking
- PG23MonocultureMatching.pg23ConnectedSupport
- PG23MonocultureMatching.pg23MonocultureTypeLaw
- PG23MonocultureMatching.pg23PolycultureTypeLaw
- PG23MonocultureMatching.pg23SourceAggregateDemand
- PG23MonocultureMatching.pg23SourceMarketClearing
- PG23MonocultureMatching.pg23SourceScore

Lean-elaborated claim roles

1. parameter: ℕ
2. assumption: NeZero n
3. parameter: MeasureTheory.Measure ℝ
4. parameter: MeasureTheory.Measure ℝ
5. assumption: MeasureTheory.IsProbabilityMeasure valueLaw
6. assumption: MeasureTheory.IsProbabilityMeasure noiseLaw
7. assumption: 1 < n
8. assumption: PG23MonocultureMatching.pg23ConnectedSupport valueLaw
9. assumption: PG23MonocultureMatching.pg23ConnectedSupport noiseLaw
10. assumption: ∃ (x : ℝ) (y : ℝ), x ∈ noiseLaw.support ∧ y ∈ noiseLaw.support ∧ x < y
11. parameter: ℝ
12. parameter: ℝ
13. parameter: ℝ
14. assumption: 0 < S ∧ S < 1
15. assumption: ∀ (c : Fin n) (x : ℝ),
  (PG23MonocultureMatching.pg23MonocultureTypeLaw valueLaw noiseLaw)
      {theta : PG23MonocultureMatching.PG23ApplicantType (Fin n) |
        PG23MonocultureMatching.pg23SourceScore theta c = x} =
    0
16. assumption: ∀ (c : Fin n) (x : ℝ),
  (PG23MonocultureMatching.pg23PolycultureTypeLaw valueLaw noiseLaw)
      {theta : PG23MonocultureMatching.PG23ApplicantType (Fin n) |
        PG23MonocultureMatching.pg23SourceScore theta c = x} =
    0
17. assumption: PG23MonocultureMatching.pg23SourceMarketClearing
  (PG23MonocultureMatching.pg23SourceAggregateDemand (PG23MonocultureMatching.pg23MonocultureTypeLaw valueLaw noiseLaw))
  (fun (x : Fin n) => S / ↑(Fintype.card (Fin n))) fun (x : Fin n) => Pmono
18. assumption: PG23MonocultureMatching.pg23SourceMarketClearing
  (PG23MonocultureMatching.pg23SourceAggregateDemand (PG23MonocultureMatching.pg23PolycultureTypeLaw valueLaw noiseLaw))
  (fun (x : Fin n) => S / ↑(Fintype.card (Fin n))) fun (x : Fin n) => Ppoly
19. conclusion: Pmono < Ppoly

Lean proof endpoint (built separately)

PG23MonocultureMatching.PaperInterface.review_corollary4_monocultureCutoff_lt_polycultureCutoffSpec_proof

Recorded source-to-Spec screening **Verdict:** matches_approved_corrected_target

Reason: The expanded target concludes Pmono < Ppoly for at least two colleges under exact equal-capacity clearing, connected value and noise supports, nondegenerate noise, interior total supply, and zero mass on every induced score level. These are exactly the materially non-equivalent regularity premises and strict-cutoff conclusion bound by approved correction PG23-CONTINUOUS-REGULARITY-01.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 5

Source locator: cited publication, lines 705-719

Verbatim source input

```
\begin{theorem}[Differential Application Access in Monoculture and Polyculture]\label{thm:diff-app-access}
The following hold:
  \begin{itemize}
    \item[(i)] In monoculture, applicants who apply to more firms do not gain an advantage: For all $v$,
    \begin{equation}
      \Pr[\mu_{\mono, \kappa}(v)\in \colleges\,|\,k(v)=k] = \Pr[\mu_{\mono}(v)\in \colleges]
    \end{equation}
    is constant in $k$.
    \item[(ii)] In polyculture, applicants who apply to more firms gain an advantage: For all $v$,
    \begin{equation}
      \Pr[\mu_{\poly, \kappa}(v)\in \colleges\,|\,k(v)=k]
    \end{equation}
    is increasing in $k$, and strictly increasing in $k$ for $v\in (P_{\poly, \kappa} - X_+, P_{\poly, \kappa} - X_-).$
  \end{itemize}
\end{theorem}

[Next verbatim source excerpt]

\begin{proof}[\textbf{Proof of \Cref{thm:diff-app-access}}]
  To show part (i), note that $P_{\mono} = P_{\mono, \kappa}$.

  To show part (ii), note that
  \begin{align}
    \Pr[\mu_{\poly, \kappa}(v)\in \colleges\,|\,k(v)=k]
    &= \Pr[v + X^{(k)} > P_{\poly, \kappa}]\\
    &= 1 - F_X^k(P_{\poly, \kappa} -v),\label{eq:clock}
  \end{align}
  which is increasing in $k$. Moreover, when $v\in (P_{\poly, \kappa} - X_+, P_{\poly, \kappa} - X_-),$ we have that $P_{\poly, \kappa} - v\in (X_-, X_+),$
  ↪ so by \Cref{prop:cdf-increasing}, $F_X(P_{\poly, \kappa})\in (0,1),$ in which case \eqref{eq:clock} is strictly increasing in $k$.
```

Formalized review target

The archival source above is not asserted equivalent to this formalized target.

With connected-support value and noise laws, atomless values, and baseline and differential monoculture market clearing at the same total supply, each
↪ differential monoculture matching probability equals the unrestricted monoculture matching probability and is constant in the positive application
↪ count. Polyculture matching probability is weakly increasing in that count, and strictly increasing when the cutoff minus applicant value lies in the
↪ interior of the noise support.

Archival source anchor: cited publication, lines 705-719

Expanded PaperInterface specification

```
∀ {College : Type v} [inst : Fintype College] [Nonempty College] (valueLaw noiseLaw : MeasureTheory.Measure ℝ)
  [inst_2 : MeasureTheory.IsProbabilityMeasure valueLaw] [inst_3 : MeasureTheory.IsProbabilityMeasure noiseLaw]
  [MeasureTheory.NoAtoms valueLaw],
  PG23MonocultureMatching.pg23ConnectedSupport valueLaw →
    PG23MonocultureMatching.pg23ConnectedSupport noiseLaw →
      ∀ {n : ℕ} (accessLaw : MeasureTheory.Measure (Fin n)) [MeasureTheory.IsProbabilityMeasure accessLaw]
        {Pmono PmonoDiff PpolyDiff S : ℝ},
        0 < S ∧ S < 1 →
          PG23MonocultureMatching.pg23MonocultureScalarMatchDemand valueLaw noiseLaw Pmono = S →
            (PG23MonocultureMatching.pg23DifferentialSourceMarketClearing
                (PG23MonocultureMatching.pg23DifferentialSourceAggregateDemand
                  (PG23MonocultureMatching.pg23MonocultureDifferentialTypeLaw accessLaw valueLaw noiseLaw))
                (fun (x : College) => S / ↑(Fintype.card College)) fun (x : College) => PmonoDiff) →
              (∀ (k : Fin n) (v : ℝ),
                  noiseLaw.real
                    {noise : ℝ |
                      ∃ (c : Fin (↑k + 1)),
                        PG23MonocultureMatching.pg23ActiveSourceChoice Finset.univ
                          (fun (x : Fin (↑k + 1)) => PmonoDiff)
                          (PG23MonocultureMatching.pg23MonocultureType v noise (Fintype.equivFin (Fin (↑k + 1)))) =
                        some c} =
                noiseLaw.real
                  {noise : ℝ |
                    ∃ (c : Fin 1),
                      PG23MonocultureMatching.pg23ActiveSourceChoice Finset.univ (fun (x : Fin 1) => Pmono)
                        (PG23MonocultureMatching.pg23MonocultureType v noise (Fintype.equivFin (Fin 1))) =
                        some c}) ∧
                (∀ (k₁ k₂ : Fin n) (v : ℝ),
                    noiseLaw.real
                      {noise : ℝ |
                        ∃ (c : Fin (↑k₁ + 1)),
                          PG23MonocultureMatching.pg23ActiveSourceChoice Finset.univ
                            (fun (x : Fin (↑k₁ + 1)) => PmonoDiff)
                            (PG23MonocultureMatching.pg23MonocultureType v noise
                              (Fintype.equivFin (Fin (↑k₁ + 1)))) =
                          some c} =
                  noiseLaw.real
                    {noise : ℝ |
                      ∃ (c : Fin (↑k₂ + 1)),
                        PG23MonocultureMatching.pg23ActiveSourceChoice Finset.univ
                          (fun (x : Fin (↑k₂ + 1)) => PmonoDiff)
                          (PG23MonocultureMatching.pg23MonocultureType v noise
```

```
            (Fintype.equivFin (Fin (↑k₂ + 1)))) =
          some c}) ∧
  (∀ (k₁ k₂ : Fin n) (v : ℝ),
     k₁ ≤ k₂ →
      (MeasureTheory.Measure.pi fun (x : Fin (↑k₁ + 1)) => noiseLaw).real
          {noise : Fin (↑k₁ + 1) → ℝ |
           ∃ (c : Fin (↑k₁ + 1)),
             PG23MonocultureMatching.pg23ActiveSourceChoice Finset.univ
                (fun (x : Fin (↑k₁ + 1)) => PpolyDiff)
                (PG23MonocultureMatching.pg23PolycultureType v (Fintype.equivFin (Fin (↑k₁ + 1)))
                  noise) =
              some c} ≤
        (MeasureTheory.Measure.pi fun (x : Fin (↑k₂ + 1)) => noiseLaw).real
          {noise : Fin (↑k₂ + 1) → ℝ |
           ∃ (c : Fin (↑k₂ + 1)),
             PG23MonocultureMatching.pg23ActiveSourceChoice Finset.univ
                (fun (x : Fin (↑k₂ + 1)) => PpolyDiff)
                (PG23MonocultureMatching.pg23PolycultureType v (Fintype.equivFin (Fin (↑k₂ + 1)))
                  noise) =
              some c}) ∧
   ∀ (v : ℝ),
    PpolyDiff - v ∈ interior noiseLaw.support →
     ∀ {k₁ k₂ : Fin n},
      k₁ < k₂ →
       (MeasureTheory.Measure.pi fun (x : Fin (↑k₁ + 1)) => noiseLaw).real
          {noise : Fin (↑k₁ + 1) → ℝ |
           ∃ (c : Fin (↑k₁ + 1)),
             PG23MonocultureMatching.pg23ActiveSourceChoice Finset.univ
                (fun (x : Fin (↑k₁ + 1)) => PpolyDiff)
                (PG23MonocultureMatching.pg23PolycultureType v
                  (Fintype.equivFin (Fin (↑k₁ + 1))) noise) =
              some c} <
         (MeasureTheory.Measure.pi fun (x : Fin (↑k₂ + 1)) => noiseLaw).real
           {noise : Fin (↑k₂ + 1) → ℝ |
            ∃ (c : Fin (↑k₂ + 1)),
              PG23MonocultureMatching.pg23ActiveSourceChoice Finset.univ
                 (fun (x : Fin (↑k₂ + 1)) => PpolyDiff)
                 (PG23MonocultureMatching.pg23PolycultureType v
                   (Fintype.equivFin (Fin (↑k₂ + 1))) noise) =
               some c}
```

Retained governing declarations

- PG23MonocultureMatching.pg23ActiveSourceChoice
- PG23MonocultureMatching.pg23ConnectedSupport
- PG23MonocultureMatching.pg23DifferentialSourceAggregateDemand
- PG23MonocultureMatching.pg23DifferentialSourceMarketClearing
- PG23MonocultureMatching.pg23MonocultureDifferentialTypeLaw
- PG23MonocultureMatching.pg23MonocultureScalarMatchDemand
- PG23MonocultureMatching.pg23MonocultureType
- PG23MonocultureMatching.pg23PolycultureType

Lean-elaborated claim roles

1. parameter: Type v
2. parameter: Fintype College
3. assumption: Nonempty College
4. parameter: MeasureTheory.Measure ℝ
5. parameter: MeasureTheory.Measure ℝ
6. assumption: MeasureTheory.IsProbabilityMeasure valueLaw
7. assumption: MeasureTheory.IsProbabilityMeasure noiseLaw
8. assumption: MeasureTheory.NoAtoms valueLaw
9. assumption: PG23MonocultureMatching.pg23ConnectedSupport valueLaw
10. assumption: PG23MonocultureMatching.pg23ConnectedSupport noiseLaw
11. parameter: ℕ
12. parameter: MeasureTheory.Measure (Fin n)
13. assumption: MeasureTheory.IsProbabilityMeasure accessLaw
14. parameter: ℝ
15. parameter: ℝ
16. parameter: ℝ
17. parameter: ℝ
18. assumption: 0 < S ∧ S < 1
19. assumption: PG23MonocultureMatching.pg23MonocultureScalarMatchDemand valueLaw noiseLaw Pmono = S
20. assumption: PG23MonocultureMatching.pg23DifferentialSourceMarketClearing
  (PG23MonocultureMatching.pg23DifferentialSourceAggregateDemand
    (PG23MonocultureMatching.pg23MonocultureDifferentialTypeLaw accessLaw valueLaw noiseLaw))
  (fun (x : College) => S / ↑(Fintype.card College)) fun (x : College) => PmonoDiff
21. conclusion: (∀ (k : Fin n) (v : ℝ),
    noiseLaw.real
       {noise : ℝ |
        ∃ (c : Fin (↑k + 1)),
          PG23MonocultureMatching.pg23ActiveSourceChoice Finset.univ (fun (x : Fin (↑k + 1)) => PmonoDiff)
             (PG23MonocultureMatching.pg23MonocultureType v noise (Fintype.equivFin (Fin (↑k + 1)))) =
           some c} =

```
    noiseLaw.real
      {noise : ℝ |
       ∃ (c : Fin 1),
         PG23MonocultureMatching.pg23ActiveSourceChoice Finset.univ (fun (x : Fin 1) => Pmono)
           (PG23MonocultureMatching.pg23MonocultureType v noise (Fintype.equivFin (Fin 1))) =
         some c}) ∧
  (∀ (k₁ k₂ : Fin n) (v : ℝ),
     noiseLaw.real
       {noise : ℝ |
        ∃ (c : Fin (↑k₁ + 1)),
          PG23MonocultureMatching.pg23ActiveSourceChoice Finset.univ (fun (x : Fin (↑k₁ + 1)) => PmonoDiff)
            (PG23MonocultureMatching.pg23MonocultureType v noise (Fintype.equivFin (Fin (↑k₁ + 1)))) =
          some c} =
     noiseLaw.real
       {noise : ℝ |
        ∃ (c : Fin (↑k₂ + 1)),
          PG23MonocultureMatching.pg23ActiveSourceChoice Finset.univ (fun (x : Fin (↑k₂ + 1)) => PmonoDiff)
            (PG23MonocultureMatching.pg23MonocultureType v noise (Fintype.equivFin (Fin (↑k₂ + 1)))) =
          some c}) ∧
   (∀ (k₁ k₂ : Fin n) (v : ℝ),
      k₁ ≤ k₂ →
        (MeasureTheory.Measure.pi fun (x : Fin (↑k₁ + 1)) => noiseLaw).real
          {noise : Fin (↑k₁ + 1) → ℝ |
           ∃ (c : Fin (↑k₁ + 1)),
             PG23MonocultureMatching.pg23ActiveSourceChoice Finset.univ (fun (x : Fin (↑k₁ + 1)) => PpolyDiff)
               (PG23MonocultureMatching.pg23PolycultureType v (Fintype.equivFin (Fin (↑k₁ + 1))) noise) =
             some c} ≤
         (MeasureTheory.Measure.pi fun (x : Fin (↑k₂ + 1)) => noiseLaw).real
           {noise : Fin (↑k₂ + 1) → ℝ |
            ∃ (c : Fin (↑k₂ + 1)),
              PG23MonocultureMatching.pg23ActiveSourceChoice Finset.univ (fun (x : Fin (↑k₂ + 1)) => PpolyDiff)
                (PG23MonocultureMatching.pg23PolycultureType v (Fintype.equivFin (Fin (↑k₂ + 1))) noise) =
              some c}) ∧
    ∀ (v : ℝ),
      PpolyDiff - v ∈ interior noiseLaw.support →
        ∀ {k₁ k₂ : Fin n},
          k₁ < k₂ →
            (MeasureTheory.Measure.pi fun (x : Fin (↑k₁ + 1)) => noiseLaw).real
              {noise : Fin (↑k₁ + 1) → ℝ |
               ∃ (c : Fin (↑k₁ + 1)),
                 PG23MonocultureMatching.pg23ActiveSourceChoice Finset.univ (fun (x : Fin (↑k₁ + 1)) => PpolyDiff)
                   (PG23MonocultureMatching.pg23PolycultureType v (Fintype.equivFin (Fin (↑k₁ + 1))) noise) =
                 some c} <
              (MeasureTheory.Measure.pi fun (x : Fin (↑k₂ + 1)) => noiseLaw).real
                {noise : Fin (↑k₂ + 1) → ℝ |
                 ∃ (c : Fin (↑k₂ + 1)),
                   PG23MonocultureMatching.pg23ActiveSourceChoice Finset.univ (fun (x : Fin (↑k₂ + 1)) => PpolyDiff)
                     (PG23MonocultureMatching.pg23PolycultureType v (Fintype.equivFin (Fin (↑k₂ + 1))) noise) =
                   some c}
```

Lean proof endpoint (built separately)

PG23MonocultureMatching.PaperInterface.review_theorem3_differentialApplicationAccessSpec_proof

Recorded source-to-Spec screening **Verdict:** matches_approved_corrected_target

Reason: Compared the archival theorem at audit/cited publication:705-719 and proof at :517-526 with the complete corrected target, its approval excerpt and PG23-ENDPOINT-AND-INTERVAL-01 scope, the expanded Spec and claim atoms, and all 26 displayed prerequisite bodies. The route explicitly disclaims archival equivalence, so this is a match to the approved corrected target. The value and noise laws are probabilities with connected supports; atomlessness is required of values only. Differential types use independent positive application counts, uniform strict rankings, a common value and a single shared monoculture noise draw. The active set consists of the best k+1 colleges, and favorite-affordable choice returns some college exactly when an active raw score meets the cutoff. Per-college clearing at S/card College sums to total matched mass S. Atomless values make score-equality boundaries null after value integration, so that weak aggregate event agrees with the displayed strict-tail scalar demand integral; connected-support scalar uniqueness at 0<S<1 identifies PmonoDiff with Pmono. For each fixed value and every positive count, the monoculture choice event is therefore the same weak event Pmono<=v+noise, including the one-college baseline event, yielding both baseline equality and count invariance. Under the finite product noise law, the polyculture choice event is existence of a coordinate with PpolyDiff<=v+noise, so for m=k.val+1 its probability is 1-Pr[X<PpolyDiff-v]^m. This is weakly increasing in m for every value and strictly increasing when PpolyDiff-v lies in the support interior, where that lower-open-tail probability lies strictly between zero and one. Thus no atomless-noise premise is needed, and the support-interior strictness has exactly the approved scope. Fixing a canonical ranking does not change existence of an affordable college; the stated polyculture comparison for any common cutoff specializes to the equilibrium cutoff. No selected conclusion or model-domain case of the corrected target is lost.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 6

Source locator: cited publication, lines 689-691

Verbatim source input

```
\begin{proposition}[Nash Equilibrium]\label{prop:nash}
    For any $\kappa$, and under both monoculture and polyculture, no applicant benefits ex-ante by deviating from $S$.
\end{proposition}
```

Formalized review target

The archival source above is not asserted equivalent to this formalized target.

For a finite college set, same-cardinality feasible application deviations, ranking-monotone utility, an outside option, and application-set-invariant iid equal-cutoff success indicators, the prescribed top-k application set has no profitable ex-ante expected favorite-success payoff deviation.

Archival source anchor: cited publication, lines 689-691

Expanded PaperInterface specification

```
∀ {College : Type v} [inst : Fintype College] [Nonempty College] (k : ℕ) (rank : College → ℕ)
  (topChoiceSet : Finset College) (successAtom : PMF Bool) (outsideUtility : ℝ) (utility : College → ℝ)
  (feasible : Finset College → Prop),
  (∀ (c : College), c ∈ topChoiceSet ↔ rank c < k) →
    feasible topChoiceSet →
      (∀ (deviation : Finset College), feasible deviation → deviation.card = topChoiceSet.card) →
        (∀ (c d : College), rank c < rank d → utility d ≤ utility c) →
          PG23MonocultureMatching.NoProfitableApplicationDeviation
            (fun (applicationSet : Finset College) =>
              PG23MonocultureMatching.expectedFavoriteSuccessfulApplicationUtility
                (PG23MonocultureMatching.iidEqualCutoffSuccessLaw successAtom applicationSet) outsideUtility utility
                applicationSet)
            topChoiceSet feasible
```

Retained governing declarations

- PG23MonocultureMatching.NoProfitableApplicationDeviation
- PG23MonocultureMatching.expectedFavoriteSuccessfulApplicationUtility
- PG23MonocultureMatching.iidEqualCutoffSuccessLaw

Lean-elaborated claim roles

```
1. parameter: Type v
2. parameter: Fintype College
3. assumption: Nonempty College
4. parameter: ℕ
5. parameter: College → ℕ
6. parameter: Finset College
7. parameter: PMF Bool
8. parameter: ℝ
9. parameter: College → ℝ
10. parameter: Finset College → Prop
11. assumption: ∀ (c : College), c ∈ topChoiceSet ↔ rank c < k
12. assumption: feasible topChoiceSet
13. assumption: ∀ (deviation : Finset College), feasible deviation → deviation.card = topChoiceSet.card
14. assumption: ∀ (c d : College), rank c < rank d → utility d ≤ utility c
15. conclusion: PG23MonocultureMatching.NoProfitableApplicationDeviation
  (fun (applicationSet : Finset College) =>
    PG23MonocultureMatching.expectedFavoriteSuccessfulApplicationUtility
      (PG23MonocultureMatching.iidEqualCutoffSuccessLaw successAtom applicationSet) outsideUtility utility
      applicationSet)
  topChoiceSet feasible
```

Lean proof endpoint (built separately)

PG23MonocultureMatching.PaperInterface.review_proposition_nash_differentialApplicationAccessSpec_proof

Recorded source-to-Spec screening

Verdict: matches_approved_corrected_target
Reason: The target makes the approved finite ex-ante semantics explicit: the prescribed rank-threshold application set is feasible, every feasible deviation has the same cardinality, utility is ranking-monotone, college successes are iid with an application-set-invariant common law, and payoff is the expectation of the favorite successful application with an outside option. Its no-profitable-deviation conclusion exactly matches correction PG23-NASH-EXANTE-SEMANTICS-01.
Reviewer check: ☐ Matches formalized target
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

Review row 7

Source locator: cited publication, lines 198-200

Verbatim source input

\begin{proposition}[The Lattice Theorem]\label{prop:lattice}
The set of market-clearing cutoffs forms a complete lattice with respect to $\vee$ and $\wedge$ as defined above.
\end{proposition}

[Next verbatim source excerpt]

\section{Model and Preliminaries}
Our model builds on the model of \cite{azevedo2016supply}, in which there is a continuum of applicants and a finite number of firms (analogously, colleges). An applicant is identified by their true \textit{value}, which is distributed on $\RR$ according to a probability measure η. Let $\colleges = \{1, 2, \cdots, \numcolleges\}$ be the set of all firms. Firms have total capacity $\totalsupply < 1$ (so there are fewer positions than applicants) and each firm has the same capacity $\frac{\totalsupply}{\numcolleges}.$

An applicant with value v is associated with an independent random variable $\theta(v)$, characterizing their preferences over firms and firms' estimated values for that applicant. The realization of $\theta(v)$ is their \textit{type}, which lies in Θ. $\Theta := \mathcal{R} \times \RR^\numcolleges$ is the set of applicant types, where $\mathcal{R}$ is the set of preference orderings over the firms. If $\theta(v) = \theta = (\succ^\theta, e^\theta),$ then $\succ^\theta$ is the preference ordering of v over firms and $e^\theta_\college$ is the \textit{estimated value} of the applicant at firm $\college$.

In the model, $\theta(v)$ plays a central role. First, it is used to model the process of preference approximation for firms. Firms all prefer applicants with higher true values, but in practice, rank applicants based on the estimated values given by $\theta(v)$. Second, it connects our model with that of \cite{azevedo2016supply}. In particular, the distribution η over values in $\RR$ together with an appropriate choice of $\theta(v)$ induces a distribution over applicant types, which, alongside specifications of firm capacities, fully determines the model in \cite{azevedo2016supply}.\footnote{Formally, consider the probability distribution η' over Θ such that for $A\subseteq \Theta$,
\begin{equation}
\eta'(A) = \int_\RR \Pr[\theta(v)\in A]\,d\eta(v).
\end{equation}}

\paragraph{}
A \textit{matching} is a function $\mu: \Theta \rightarrow \colleges\cup \{\emptyset\}$ that satisfies the capacity constraints
\begin{equation}
\int_\RR \Pr[\mu(\theta(v)) = \college]\,d\eta(v) = \frac{\totalsupply}{\numcolleges},\qquad \forall \college\in \colleges,
\end{equation}
as well as two additional technical assumptions: that $\mu^{-1}(\college)$ is measurable and that the set $\{\theta:\mu(\theta)\prec^\theta \college\}$ is open.\footnote{We remark that matchings are not typically required to fill capacities (i.e., the equality above is typically an inequality), but in our setting, since there is limited total capacity and applicants all prefer being matched to being unmatched, we are only concerned with matchings that fill capacity.}

Then $\mu(\theta(v))$ is a random variable indicating where an applicant with value v is matched, with $\mu(\theta(v)) = \emptyset$ indicating that the applicant is not matched. When it is clear what θ is, we will abuse notation to write $\mu(v)$ instead of $\mu(\theta(v))$ to denote the random variable representing where an applicant with value v is matched. Notice that the matching of an applicant depends only on their random type, reflecting the fact that their matching outcome depends directly on the scores they receive at each firm rather than their true value.

A matching is \textit{stable} if there does not exist a pair $\theta\in \Theta, \college\in \colleges$ such that $\college\succ^\theta \mu(\theta)$, and $e^\theta_\college > e^{\theta'}_{\college}$ for some $\theta'\in \mu^{-1}(\college)$. In other words, there is no applicant-firm pair that would jointly prefer to defect from the current matching to be matched with each other.

\subsection{A cutoff characterization of stable matchings}
A benefit of this continuum model is that stable matchings are characterized by a tractable cutoff structure \citep{azevedo2016supply}. Given a vector of \textit{cutoffs} $P = (P_1, \cdots, P_\numcolleges),$ an applicant with type θ can \textit{afford} firm $\college$ if $e^\theta_\college \ge P_\college$. The applicant's \textit{demand} at P is $D^\theta(P)$, the applicant's most preferred firm (according to $\succ^\theta$) among those they can afford. If they cannot afford any firm, $D^\theta(P)=\emptyset$.
The \textit{aggregate demand} of a firm $\college$ at P is
\begin{equation}
D^\college(P) := \int_\RR \Pr[D^{\theta(v)}(P) = \college]\,d\eta(v),
\end{equation}
the measure of applicants who demand it.

A vector of cutoffs P is \textit{market clearing} if and only if
\begin{equation}
D^\college(P) = \frac{\totalsupply}{\numcolleges},\qquad \forall \college\in \numcolleges.\footnote{Again, as in the above footnotes, the standard definition in \cite{azevedo2016supply} would allow for an inequality here if no applicant would prefer to take the empty spots of a firm. This is not possible in our model, so we simplify our definition.}
\end{equation}

[Next verbatim source excerpt]

\subsection{The Lattice Theorem for Stable Matchings}
A powerful result in the stable matching literature shows that stable matchings have a complete lattice structure. In this section, we state this result in our setup (Theorem A.1 in \cite{azevedo2016supply}). Consider the lattice operators $\vee$ and $\wedge$ on cutoffs in $\RR^C$, where for a set Z of cutoffs,
\begin{equation}
\left(\bigvee_{P\in Z} P\right)_\college = \sup_{P\in Z} P_\college
\end{equation}
and
\begin{equation}
\left(\bigwedge_{P\in Z} P\right)_\college = \inf_{P\in Z} P_\college.
\end{equation}

[Next verbatim source excerpt]

2. The Continuum Model
2.1. Definitions. We begin the exposition with the simpler, and novel continuum
model, and examine its connection with the standard discrete Gale and Shapley (1962)
model in Sections 3 and 4. The model follows closely the Gale and Shapley (1962) college
admissions model. The main departure is that a finite set of colleges C = {1, 2, . . . , C}
are matched to a continuum mass of students. A student is described by her type
θ = ([U+001F control character]θ , eθ). [U+001F control character]θ is the student’s strict preference ordering over colleges. The vector
eθ ∈ [0, 1]C describes the colleges’ ordinal preferences for the student. We refer to eθc

as student θ's score at college c. Colleges prefer students with higher scores. That is,
17Specifically, Hatfield et al. (2011b) show that, in a trading network with quasilinear utilities, free

transfers of a numeraire between agents, and substitutable preferences the set of stable outcomes is essentially equivalent to the set of Walrasian equilibria.

[form-feed in source extraction]
SUPPLY AND DEMAND IN MATCHING MARKETS

11

0

college c prefers student θ over θ0 if eθc > eθc .18 To simplify notation we assume that all students and colleges are acceptable.19 Let R be the set of all strict preference orderings over colleges. We denote the set of all student types by Θ = R × [0, 1]C .
A continuum economy is given by E = [η, S], where η is a probability measure20 over Θ and S = (S1 , S2 , . . . , SC) is a vector of strictly positive capacities for each college. We make the following assumption on η, which corresponds to colleges having strict preferences over students in the discrete model.
Assumption 1. (Strict Preferences) Every college's indifference curves have ηmeasure 0. That is, for any college c and real number x we have η({θ : eθc = ↪ x}) = 0.

[Next verbatim source excerpt]

A.3. Lattice Theorem and Rural Hospitals Theorem. Consider the sup (∨) and inf (∧) operators on Rn as lattice operators on cutoffs. That is, given two vectors of cutoffs
(P ∨ P 0)c = sup{Pc , Pc0 }.
C

More generally, given an arbitrary set of cutoffs X ⊆ 2([0,1]) , we define the sup and inf operators analogously. That is
(∨X)c = sup Pc .
P ∈X

We then have that the set of market clearing cutoffs forms a complete lattice with respect to these operators.
Theorem A1. (Lattice Theorem) The set of market clearing cutoffs is a complete lattice under ∨, ∧.
Proof. First note that the set of market clearing cutoffs is nonempty. Now, consider two market clearing cutoffs P and P 0 , and let P + = P ∨ P 0 . Take a college c, and assume without loss of generality that Pc ≤ Pc0 . By the definition of demand, we must have that Dc (P +) ≥ Dc (P 0), as Pc+ = Pc0 and the cutoffs of other colleges are higher under P + . In addition, if Pc0 > 0, then Dc (P +) ≥ Sc ≥ Dc (P). Moreover, if Pc0 = 0,

[form-feed in source extraction]
AZEVEDO AND LESHNO

44

then Pc = Pc0 , and Dc (P +) ≥ Dc (P). Either way, we have that
Dc (P +) ≥ max{Dc (P), Dc (P 0)}.
P
Moreover, the demand for staying unmatched 1 − c∈C Dc (·) must at least as large under P + than under P or P 0 . Because demand for staying unmatched plus for all colleges always sums to 1, we have that, for all colleges, Dc (P +) = Dc (P) = Dc (P 0) . In particular, P + is a market clearing cutoff. The proof for the inf operator is analogous. Moreover, it follows by induction that the sup and inf of any finite set of market clearing cutoffs is a market clearing cutoff. This establishes that the set of market clearing cutoffs is a lattice.
Last, we show that the lattice is a complete lattice. That is, that the sup of any arbitrary, and possibly infinite, set of market clearing cutoffs P ⊆ RC is a market clearing cutoff. Every subset of RC has a sup, so that the vector ∨P is well-defined. Moreover, there exists a sequence of finite sets Pk ⊆ P such that the vectors ∨Pk ∈ RC converge to ∨P as k grows. By the first part of the proof, each ∨Pk is a market clearing cutoff. Since the demand function is continuous,40 the set of market clearing cutoffs is closed. It follows that ∨P = limk→∞ ∨Pk is a market clearing cutoff. The proof for the inf operator is analogous.

Expanded PaperInterface specification

```
∀ {College : Type v} [inst : Fintype College] [Nonempty College]
  (typeLaw : MeasureTheory.Measure (PG23MonocultureMatching.PG23ApplicantType College))
  [MeasureTheory.IsProbabilityMeasure typeLaw],
  PG23MonocultureMatching.pg23SourceScoreLevelNull typeLaw →
    ∀ (S : ℝ),
      0 < S →
        S < 1 →
          (∃ (cutoff : College → ℝ),
             PG23MonocultureMatching.pg23SourceMarketClearing
               (PG23MonocultureMatching.pg23SourceAggregateDemand typeLaw)
               (fun (x : College) => S / ↑(Fintype.card College)) cutoff) ∧
            AppliedModelingLib.Matching.CompleteLatticeOn
              (PG23MonocultureMatching.pg23SourceMarketClearing
                (PG23MonocultureMatching.pg23SourceAggregateDemand typeLaw) fun (x : College) =>
                S / ↑(Fintype.card College))
              PG23MonocultureMatching.pg23RawCutoffLe ∧
             (∀ (Z : Set (College → ℝ)),
                Z.Nonempty →
                  (∀ (P : College → ℝ),
                     Z P →
                       PG23MonocultureMatching.pg23SourceMarketClearing
```

(PG23MonocultureMatching.pg23SourceAggregateDemand typeLaw)
(fun (x : College) => S / ↑(Fintype.card College)) P) →
AppliedModelingLib.Matching.IsLeastUpperBoundOn
(PG23MonocultureMatching.pg23SourceMarketClearing
(PG23MonocultureMatching.pg23SourceAggregateDemand typeLaw) fun (x : College) =>
S / ↑(Fintype.card College))
PG23MonocultureMatching.pg23RawCutoffLe Z (PG23MonocultureMatching.pg23RawCutoffSup Z)) ∧
∀ (Z : Set (College → ℝ)),
Z.Nonempty →
(∀ (P : College → ℝ),
Z P →
PG23MonocultureMatching.pg23SourceMarketClearing
(PG23MonocultureMatching.pg23SourceAggregateDemand typeLaw)
(fun (x : College) => S / ↑(Fintype.card College)) P) →
AppliedModelingLib.Matching.IsGreatestLowerBoundOn
(PG23MonocultureMatching.pg23SourceMarketClearing
(PG23MonocultureMatching.pg23SourceAggregateDemand typeLaw) fun (x : College) =>
S / ↑(Fintype.card College))
PG23MonocultureMatching.pg23RawCutoffLe Z (PG23MonocultureMatching.pg23RawCutoffInf Z)

Retained governing declarations

- AL16SupplyDemandMatching.instMeasurableSpaceAL16Ranking
- AppliedModelingLib.Matching.CompleteLatticeOn
- AppliedModelingLib.Matching.IsGreatestLowerBoundOn
- AppliedModelingLib.Matching.IsLeastUpperBoundOn
- PG23MonocultureMatching.PG23ApplicantType
- PG23MonocultureMatching.PG23Ranking
- PG23MonocultureMatching.pg23RawCutoffInf
- PG23MonocultureMatching.pg23RawCutoffLe
- PG23MonocultureMatching.pg23RawCutoffSup
- PG23MonocultureMatching.pg23SourceAggregateDemand
- PG23MonocultureMatching.pg23SourceMarketClearing
- PG23MonocultureMatching.pg23SourceScoreLevelNull

Lean-elaborated claim roles

1. parameter: Type v
2. parameter: Fintype College
3. assumption: Nonempty College
4. parameter: MeasureTheory.Measure (PG23MonocultureMatching.PG23ApplicantType College)
5. assumption: MeasureTheory.IsProbabilityMeasure typeLaw
6. assumption: PG23MonocultureMatching.pg23SourceScoreLevelNull typeLaw
7. parameter: ℝ
8. assumption: 0 < S
9. assumption: S < 1
10. conclusion: (∃ (cutoff : College → ℝ),
PG23MonocultureMatching.pg23SourceMarketClearing (PG23MonocultureMatching.pg23SourceAggregateDemand typeLaw)
(fun (x : College) => S / ↑(Fintype.card College)) cutoff) ∧
AppliedModelingLib.Matching.CompleteLatticeOn
(PG23MonocultureMatching.pg23SourceMarketClearing (PG23MonocultureMatching.pg23SourceAggregateDemand typeLaw)
fun (x : College) => S / ↑(Fintype.card College))
PG23MonocultureMatching.pg23RawCutoffLe ∧
(∀ (Z : Set (College → ℝ)),
Z.Nonempty →
(∀ (P : College → ℝ),
Z P →
PG23MonocultureMatching.pg23SourceMarketClearing
(PG23MonocultureMatching.pg23SourceAggregateDemand typeLaw)
(fun (x : College) => S / ↑(Fintype.card College)) P) →
AppliedModelingLib.Matching.IsLeastUpperBoundOn
(PG23MonocultureMatching.pg23SourceMarketClearing
(PG23MonocultureMatching.pg23SourceAggregateDemand typeLaw) fun (x : College) =>
S / ↑(Fintype.card College))
PG23MonocultureMatching.pg23RawCutoffLe Z (PG23MonocultureMatching.pg23RawCutoffSup Z)) ∧
∀ (Z : Set (College → ℝ)),
Z.Nonempty →
(∀ (P : College → ℝ),
Z P →
PG23MonocultureMatching.pg23SourceMarketClearing
(PG23MonocultureMatching.pg23SourceAggregateDemand typeLaw)
(fun (x : College) => S / ↑(Fintype.card College)) P) →
AppliedModelingLib.Matching.IsGreatestLowerBoundOn
(PG23MonocultureMatching.pg23SourceMarketClearing
(PG23MonocultureMatching.pg23SourceAggregateDemand typeLaw) fun (x : College) =>
S / ↑(Fintype.card College))
PG23MonocultureMatching.pg23RawCutoffLe Z (PG23MonocultureMatching.pg23RawCutoffInf Z)

Lean proof endpoint (built separately)

PG23MonocultureMatching.PaperInterface.review_proposition_latticeSpec_proof

Recorded source-to-Spec screening **Verdict:** matches

Reason: Under the cited score-level-null condition and the paper’s positive equal capacities, the target asserts a nonempty market-clearing carrier, complete-lattice structure under pointwise cutoff order, and identifies the least upper and greatest lower bounds of every nonempty clearing family with coordinatewise suprema and infima. The abstract complete-lattice fields also supply the empty-family extrema, matching the source lattice theorem.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 8

Source locator: cited publication, lines 324-340

Verbatim source input

```
\begin{lemma}\label{lem:potato}
If $\DD$ is maximum concentrating, the following hold for all $\epsilon, \delta > 0$ when $\numcolleges$ is sufficiently large:
\begin{enumerate}
    \item[(i)] For all $v < P_{\poly} - \EE\left[X^{(\numcolleges)}\right] - \frac{\delta}{3},$
    \begin{equation}
        \Pr[\mu_{\poly}(v)\in \numcolleges] < \epsilon.
    \end{equation}
    \item[(ii)] For all $v > P_{\poly} - \EE\left[X^{(\numcolleges)}\right] + \frac{\delta}{3},$
    \begin{equation}
        \Pr[\mu_{\poly}(v)\in \numcolleges] > 1 - \epsilon.
    \end{equation}
    \item[(iii)]
    \begin{equation}
\left|\left(P_{\poly} - \EE\left[X^{(\numcolleges)}\right]\right) - v_\totalsupply\right| < \frac{2\delta}{3}.
    \end{equation}
\end{enumerate}
\end{lemma}
```

Expanded PaperInterface specification

```
∀ (valueLaw noiseLaw : MeasureTheory.Measure ℝ) [inst : MeasureTheory.IsProbabilityMeasure valueLaw]
  [inst_1 : MeasureTheory.IsProbabilityMeasure noiseLaw],
  PG23MonocultureMatching.pg23ConnectedSupport valueLaw →
    ∀ {polyCutoff : ℕ → ℝ} {vS S : ℝ},
      0 < S ∧ S < 1 →
        AppliedModelingLib.Probability.upperTailMass valueLaw vS = S →
          PG23MonocultureMatching.pg23MaximumConcentratingNoiseLaw noiseLaw →
            (∀ (n : ℕ),
                PG23MonocultureMatching.pg23SourceMarketClearing
                  (PG23MonocultureMatching.pg23SourceAggregateDemand
                    (PG23MonocultureMatching.pg23PolycultureTypeLaw valueLaw noiseLaw))
                  (fun (x : Fin (n + 1)) => S / ↑(Fintype.card (Fin (n + 1)))) fun (x : Fin (n + 1)) => polyCutoff n) →
              (∀ (ε δ : ℝ),
                  0 < ε →
                    0 < δ →
                      ∀ᶠ (n : ℕ) in Filter.atTop,
                        ∀
                          v <
                            polyCutoff n -
                                AppliedModelingLib.Probability.expectedTopOrderStatisticSeq
                                  (fun (k : ℕ) => MeasureTheory.Measure.pi fun (x : Fin (k + 1)) => noiseLaw) n -
                              δ / 3,
                            (AppliedModelingLib.Matching.cutoffWeakCrossingProbability
                                (MeasureTheory.Measure.pi fun (x : Fin (n + 1)) => noiseLaw) Finset.univ v
                                fun (x : Fin (n + 1)) => polyCutoff n) <
                              ε) ∧
                (∀ (ε δ : ℝ),
                    0 < ε →
                      0 < δ →
                        ∀ᶠ (n : ℕ) in Filter.atTop,
                          ∀ (v : ℝ),
                            polyCutoff n -
                                  AppliedModelingLib.Probability.expectedTopOrderStatisticSeq
                                    (fun (k : ℕ) => MeasureTheory.Measure.pi fun (x : Fin (k + 1)) => noiseLaw) n +
                                δ / 3 <
                              v →
                            1 - ε <
                              AppliedModelingLib.Matching.cutoffWeakCrossingProbability
                                (MeasureTheory.Measure.pi fun (x : Fin (n + 1)) => noiseLaw) Finset.univ v
                                fun (x : Fin (n + 1)) => polyCutoff n) ∧
                  Filter.Tendsto
                    (fun (n : ℕ) =>
                      polyCutoff n -
                        AppliedModelingLib.Probability.expectedTopOrderStatisticSeq
                          (fun (k : ℕ) => MeasureTheory.Measure.pi fun (x : Fin (k + 1)) => noiseLaw) n)
                    Filter.atTop (nhds vS)
```

Retained governing declarations

- AppliedModelingLib.Matching.cutoffWeakCrossingProbability
- AppliedModelingLib.Probability.expectedTopOrderStatisticSeq
- AppliedModelingLib.Probability.upperTailMass
- PG23MonocultureMatching.pg23ConnectedSupport
- PG23MonocultureMatching.pg23MaximumConcentratingNoiseLaw
- PG23MonocultureMatching.pg23PolycultureTypeLaw
- PG23MonocultureMatching.pg23SourceAggregateDemand
- PG23MonocultureMatching.pg23SourceMarketClearing

Lean-elaborated claim roles

```
1. parameter: MeasureTheory.Measure ℝ
2. parameter: MeasureTheory.Measure ℝ
3. assumption: MeasureTheory.IsProbabilityMeasure valueLaw
4. assumption: MeasureTheory.IsProbabilityMeasure noiseLaw
5. assumption: PG23MonocultureMatching.pg23ConnectedSupport valueLaw
6. parameter: ℕ → ℝ
7. parameter: ℝ
8. parameter: ℝ
9. assumption: 0 < S ∧ S < 1
10. assumption: AppliedModelingLib.Probability.upperTailMass valueLaw vS = S
11. assumption: PG23MonocultureMatching.pg23MaximumConcentratingNoiseLaw noiseLaw
12. assumption: ∀ (n : ℕ),
  PG23MonocultureMatching.pg23SourceMarketClearing
    (PG23MonocultureMatching.pg23SourceAggregateDemand
      (PG23MonocultureMatching.pg23PolycultureTypeLaw valueLaw noiseLaw))
    (fun (x : Fin (n + 1)) => S / ↑(Fintype.card (Fin (n + 1)))) fun (x : Fin (n + 1)) => polyCutoff n
13. conclusion: (∀ (ε δ : ℝ),
    0 < ε →
      0 < δ →
        ∀ᶠ (n : ℕ) in Filter.atTop,
          ∀
            v <
              polyCutoff n -
                  AppliedModelingLib.Probability.expectedTopOrderStatisticSeq
                    (fun (k : ℕ) => MeasureTheory.Measure.pi fun (x : Fin (k + 1)) => noiseLaw) n -
                δ / 3,
          (AppliedModelingLib.Matching.cutoffWeakCrossingProbability
              (MeasureTheory.Measure.pi fun (x : Fin (n + 1)) => noiseLaw) Finset.univ v fun (x : Fin (n + 1)) =>
              polyCutoff n) <
            ε) ∧
  (∀ (ε δ : ℝ),
      0 < ε →
        0 < δ →
          ∀ᶠ (n : ℕ) in Filter.atTop,
            ∀ (v : ℝ),
              polyCutoff n -
                    AppliedModelingLib.Probability.expectedTopOrderStatisticSeq
                      (fun (k : ℕ) => MeasureTheory.Measure.pi fun (x : Fin (k + 1)) => noiseLaw) n +
                  δ / 3 <
                v →
              1 - ε <
                AppliedModelingLib.Matching.cutoffWeakCrossingProbability
                  (MeasureTheory.Measure.pi fun (x : Fin (n + 1)) => noiseLaw) Finset.univ v fun (x : Fin (n + 1)) =>
                  polyCutoff n) ∧
    Filter.Tendsto
      (fun (n : ℕ) =>
        polyCutoff n -
          AppliedModelingLib.Probability.expectedTopOrderStatisticSeq
            (fun (k : ℕ) => MeasureTheory.Measure.pi fun (x : Fin (k + 1)) => noiseLaw) n)
      Filter.atTop (nhds vS)
```

Lean proof endpoint (built separately)

PG23MonocultureMatching.PaperInterface.review_lemma10_threePartSpec_proof

Recorded source-to-Spec screening **Verdict:** matches

Reason: The target gives the source's two eventual polyculture match-probability bounds using the literal weak-affordability event and states convergence of Ppoly(n) minus the expected finite-sample maximum to vS. That convergence is equivalent to the source's eventual absolute bound for every positive delta, and the displayed clearing, tail-mass, connected-support, and maximum-concentration premises supply the same model context.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 9

Source locator: cited publication, lines 38-43

Verbatim source input

```
\begin{lemma}[Supply and Demand Lemma \citep{azevedo2016supply}]\label{lem:supply-demand}
   A matching $\mu$ is stable if and only if there exists a vector of market-clearing cutoffs $P$ such that
   \begin{equation}
      \mu(\theta) = D^\theta(P).
   \end{equation}
\end{lemma}

[Next verbatim source excerpt]

\section{Model and Preliminaries}
Our model builds on the model of \cite{azevedo2016supply}, in which there is a continuum of applicants and a finite number of firms (analogously,
↪  colleges). An applicant is identified by their true \textit{value}, which is distributed on $\RR$ according to a probability measure $\eta$. Let $\colleges
↪  = \{1, 2, \cdots, \numcolleges\}$ be the set of all firms. Firms have total capacity $\totalsupply < 1$ (so there are fewer positions than applicants)
↪  and each firm has the same capacity $\frac{\totalsupply}{\numcolleges}.$

An applicant with value $v$ is associated with an independent random variable $\theta(v)$, characterizing their preferences over firms and firms'
↪  estimated values for that applicant. The realization of $\theta(v)$ is their \textit{type}, which lies in $\Theta$. $\Theta := \mathcal{R} \times
↪  \RR^\numcolleges$ is the set of applicant types, where $\mathcal{R}$ is the set of preference orderings over the firms. If $\theta(v) = \theta =
↪  (\succ^\theta, e^\theta),$ then $\succ^\theta$ is the preference ordering of $v$ over firms and $e^\theta_\college$ is the \textit{estimated value} of
↪  the applicant at firm $\college$.

In the model, $\theta(v)$ plays a central role. First, it is used to model the process of preference approximation for firms. Firms all prefer applicants with
↪  higher true values, but in practice, rank applicants based on the estimated values given by $\theta(v)$. Second, it connects our model with that of
↪  \cite{azevedo2016supply}. In particular, the distribution $\eta$ over values in $\RR$ together with an appropriate choice of $\theta(v)$ induces a
↪  distribution over applicant types, which, alongside specifications of firm capacities, fully determines the model in
↪  \cite{azevedo2016supply}.\footnote{Formally, consider the probability distribution $\eta'$ over $\Theta$ such that for $A\subseteq \Theta$,
\begin{equation}
   \eta'(A) = \int_\RR \Pr[\theta(v)\in A]\,d\eta(v).
\end{equation}}

\paragraph{}
A \textit{matching} is a function $\mu: \Theta \rightarrow \colleges\cup \{\emptyset\}$ that satisfies the capacity constraints
\begin{equation}
   \int_\RR \Pr[\mu(\theta(v)) = \college]\,d\eta(v) = \frac{\totalsupply}{\numcolleges},\qquad \forall \college\in \colleges,
\end{equation}
as well as two additional technical assumptions: that $\mu^{-1}(\college)$ is measurable and that the set $\{\theta:\mu(\theta)\prec^\theta \college\}$ is
↪  open.\footnote{We remark that matchings are not typically required to fill capacities (i.e., the equality above is typically an inequality), but in our
↪  setting, since there is limited total capacity and applicants all prefer being matched to being unmatched, we are only concerned with matchings that
↪  fill capacity.}

Then $\mu(\theta(v))$ is a random variable indicating where an applicant with value $v$ is matched, with $\mu(\theta(v)) = \emptyset$ indicating that the
↪  applicant is not matched. When it is clear what $\theta$ is, we will abuse notation to write $\mu(v)$ instead of $\mu(\theta(v))$ to denote the random
↪  variable representing where an applicant with value $v$ is matched. Notice that the matching of an applicant depends only on their random type,
↪  reflecting the fact that their matching outcome depends directly on the scores they receive at each firm rather than their true value.

A matching is \textit{stable} if there does not exist a pair $\theta\in \Theta, \college\in \colleges$ such that $\college\succ^\theta \mu(\theta)$, and
↪  $e^\theta_\college > e^{\theta'}_{\college}$ for some $\theta'\in \mu^{-1}(\college)$. In other words, there is no applicant-firm pair that would
↪  jointly prefer to defect from the current matching to be matched with each other.

\subsection{A cutoff characterization of stable matchings}
A benefit of this continuum model is that stable matchings are characterized by a tractable cutoff structure \citep{azevedo2016supply}. Given a vector of
↪  \textit{cutoffs} $P = (P_1, \cdots, P_\numcolleges),$ an applicant with type $\theta$ can \textit{afford} firm $\college$ if $e^\theta_\college \ge
↪  P_\college$. The applicant's \textit{demand} at $P$ is $D^\theta(P)$, the applicant's most preferred firm (according to $\succ^\theta$) among those
↪  they can afford. If they cannot afford any firm, $D^\theta(P)=\emptyset$.
The \textit{aggregate demand} of a firm $\college$ at $P$ is
\begin{equation}
   D^\college(P) := \int_\RR \Pr[D^{\theta(v)}(P) = \college]\,d\eta(v),
\end{equation}
the measure of applicants who demand it.

A vector of cutoffs $P$ is \textit{market clearing} if and only if
\begin{equation}
   D^\college(P) = \frac{\totalsupply}{\numcolleges},\qquad \forall \college\in \numcolleges.\footnote{Again, as in the above footnotes, the standard
   ↪  definition in \cite{azevedo2016supply} would allow for an inequality here if no applicant would prefer to take the empty spots of a firm. This is not
   ↪  possible in our model, so we simplify our definition.}
\end{equation}

[Next verbatim source excerpt]

2. The Continuum Model
2.1. Definitions. We begin the exposition with the simpler, and novel continuum
model, and examine its connection with the standard discrete Gale and Shapley (1962)
model in Sections 3 and 4. The model follows closely the Gale and Shapley (1962) college
admissions model. The main departure is that a finite set of colleges C = {1, 2, . . . , C}
are matched to a continuum mass of students. A student is described by her type
θ = ([U+001F control character]θ , eθ ). [U+001F control character]θ is the student's strict preference ordering over colleges. The vector
eθ ∈ [0, 1]C describes the colleges' ordinal preferences for the student. We refer to eθc
as student θ's score at college c. Colleges prefer students with higher scores. That is,
17Specifically, Hatfield et al. (2011b) show that, in a trading network with quasilinear utilities, free

transfers of a numeraire between agents, and substitutable preferences the set of stable outcomes is
essentially equivalent to the set of Walrasian equilibria.


[form-feed in source extraction]
SUPPLY AND DEMAND IN MATCHING MARKETS

11
```

0

college c prefers student θ over θ0 if eθc > eθc .18 To simplify notation we assume that all students and colleges are acceptable.19 Let R be the set of all strict preference orderings over colleges. We denote the set of all student types by Θ = R × [0, 1]C .
A continuum economy is given by E = [η, S], where η is a probability measure20 over Θ and S = (S1 , S2 , . . . , SC) is a vector of strictly positive capacities for each college. We make the following assumption on η, which corresponds to colleges having strict preferences over students in the discrete model.
Assumption 1. (Strict Preferences) Every college's indifference curves have ηmeasure 0. That is, for any college c and real number x we have η({θ : eθc = ↪ x}) = 0.

[Next verbatim source excerpt]

dependence on E when there is no risk of confusion. A cutoff is a minimal score Pc ∈ [0, 1] required for admission at a college c. We say that a student θ can afford college c if Pc ≤ eθc , that is c would accept θ. A student's demand given a vector of cutoffs is her favorite college among those she can afford. That is,
(2.1)

Dθ (P) = arg max{c|Pc ≤ eθc } ∪ {θ}.
[U+001F control character]θ

Aggregate demand for college c is the mass of students that demand it,
Dc (P) = η({θ : Dθ (P) = c}).
The aggregate demand vector {Dc (P)}c∈C is denoted by D(P).
A market clearing cutoff is a vector of cutoffs that clears supply and demand for colleges.
Definition 2. A vector of cutoffs P is a market clearing cutoff if it satisfies the following market clearing equations.
Dc (P) ≤ Sc
for all c, and Dc (P) = Sc if Pc > 0.
There is a natural one-to-one correspondence between stable matchings and market clearing cutoffs. To describe this correspondence, we define two operators. Given a market clearing cutoff P , we define the associated matching μ = MP using the demand function:
μ(θ) = Dθ (P).
Conversely, given a stable matching μ, we define the associated cutoff P = Pμ by the score of a marginal students matched to each college:
(2.2)

Pc = inf eθc .
θ∈μ(c)

The operators M and P form a bijection between stable matchings and market clearing cutoffs.
Lemma 1. (Supply and Demand Lemma) If μ is stable matching, then Pμ is a market clearing cutoff. If P is a market clearing cutoff, then MP is a stable matching. In addition, the operators P and M are inverses of each other.

Expanded PaperInterface specification

```
∀ {College : Type v} [inst : Fintype College] [Nonempty College]
  (typeLaw : MeasureTheory.Measure (PG23MonocultureMatching.PG23ApplicantType College))
  [MeasureTheory.IsProbabilityMeasure typeLaw] (S : ℝ),
  0 < S ∧ S < 1 →
    ∀ (matching : PG23MonocultureMatching.PG23Matching College),
      PG23MonocultureMatching.pg23SourceStableMatching typeLaw (fun (x : College) => S / ↑(Fintype.card College))
          matching ↔
        ∃ (P : College → ℝ),
          PG23MonocultureMatching.pg23SourceMarketClearing (PG23MonocultureMatching.pg23SourceAggregateDemand typeLaw)
              (fun (x : College) => S / ↑(Fintype.card College)) P ∧
            matching = PG23MonocultureMatching.pg23SourceChoice P
```

Retained governing declarations

- AL16SupplyDemandMatching.instMeasurableSpaceAL16Ranking
- PG23MonocultureMatching.PG23ApplicantType
- PG23MonocultureMatching.PG23Matching
- PG23MonocultureMatching.PG23Ranking
- PG23MonocultureMatching.pg23SourceAggregateDemand
- PG23MonocultureMatching.pg23SourceChoice
- PG23MonocultureMatching.pg23SourceMarketClearing
- PG23MonocultureMatching.pg23SourceStableMatching

Lean-elaborated claim roles

1. parameter: Type v
2. parameter: Fintype College
3. assumption: Nonempty College
4. parameter: MeasureTheory.Measure (PG23MonocultureMatching.PG23ApplicantType College)

5. assumption: MeasureTheory.IsProbabilityMeasure typeLaw
6. parameter: ℝ
7. assumption: 0 < S ∧ S < 1
8. parameter: PG23MonocultureMatching.PG23Matching College
9. conclusion: PG23MonocultureMatching.pg23SourceStableMatching typeLaw (fun (x : College) => S / ↑(Fintype.card College)) matching ↔
∃ (P : College → ℝ),
PG23MonocultureMatching.pg23SourceMarketClearing (PG23MonocultureMatching.pg23SourceAggregateDemand typeLaw)
(fun (x : College) => S / ↑(Fintype.card College)) P ∧
matching = PG23MonocultureMatching.pg23SourceChoice P

Lean proof endpoint (built separately)

PG23MonocultureMatching.PaperInterface.review_lemma1_supplyDemandSpec_proof

Recorded source-to-Spec screening **Verdict:** matches

Reason: On the source-admissible domain, including AL16 Assumption 1's zero mass on every college score level, the target states the same equivalence between source stability and existence of an exact market-clearing cutoff whose favorite-affordable choice equals the matching. The displayed feasibility, blocking, weak-affordability choice, and equal-capacity clearing definitions preserve every source case and conclusion. Quantifying the valid theorem over additional type laws does not weaken or alter its specialization to the paper's restricted domain.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 10

Source locator: cited publication, lines 71-74

Verbatim source input

```
\begin{lemma}[Equal Cutoffs Lemma]\label{prop:equal-cutoffs}
   If $\theta\in \{\theta_{\mono}, \theta_{\poly}\},$ then there is a unique vector of market-clearing cutoffs $P$, and
   $P_1 = P_2 = \cdots = P_\numcolleges.$
\end{lemma}
```

Formalized review target

The archival source above is not asserted equivalent to this formalized target.

For each literal monoculture and polyculture economy with connected-support value and noise laws, zero mass on every induced score level, and 0 < S < 1, there is a unique market-clearing cutoff vector and all its coordinates are equal.

Archival source anchor: cited publication, lines 71-74

Expanded PaperInterface specification

```
∀ {College : Type v} [inst : Fintype College] [Nonempty College],
  (∀ (valueLaw noiseLaw : MeasureTheory.Measure ℝ) [inst_2 : MeasureTheory.IsProbabilityMeasure valueLaw]
     [inst_3 : MeasureTheory.IsProbabilityMeasure noiseLaw],
     PG23MonocultureMatching.pg23ConnectedSupport valueLaw →
       PG23MonocultureMatching.pg23ConnectedSupport noiseLaw →
         PG23MonocultureMatching.pg23SourceScoreLevelNull
           (PG23MonocultureMatching.pg23MonocultureTypeLaw valueLaw noiseLaw) →
           ∀ (S : ℝ),
             0 < S ∧ S < 1 →
               ∃ (P : College → ℝ),
                 PG23MonocultureMatching.pg23SourceMarketClearing
                   (PG23MonocultureMatching.pg23SourceAggregateDemand
                     (PG23MonocultureMatching.pg23MonocultureTypeLaw valueLaw noiseLaw))
                   (fun (x : College) => S / ↑(Fintype.card College)) P ∧
                 (∀ (Q : College → ℝ),
                    PG23MonocultureMatching.pg23SourceMarketClearing
                      (PG23MonocultureMatching.pg23SourceAggregateDemand
                        (PG23MonocultureMatching.pg23MonocultureTypeLaw valueLaw noiseLaw))
                      (fun (x : College) => S / ↑(Fintype.card College)) Q →
                    Q = P) ∧
                 ∃ (p : ℝ), ∀ (c : College), P c = p) ∧
  ∀ (valueLaw noiseLaw : MeasureTheory.Measure ℝ) [inst_2 : MeasureTheory.IsProbabilityMeasure valueLaw]
    [inst_3 : MeasureTheory.IsProbabilityMeasure noiseLaw],
    PG23MonocultureMatching.pg23ConnectedSupport valueLaw →
      PG23MonocultureMatching.pg23ConnectedSupport noiseLaw →
        PG23MonocultureMatching.pg23SourceScoreLevelNull
          (PG23MonocultureMatching.pg23PolycultureTypeLaw valueLaw noiseLaw) →
          ∀ (S : ℝ),
            0 < S ∧ S < 1 →
              ∃ (P : College → ℝ),
                PG23MonocultureMatching.pg23SourceMarketClearing
                  (PG23MonocultureMatching.pg23SourceAggregateDemand
                    (PG23MonocultureMatching.pg23PolycultureTypeLaw valueLaw noiseLaw))
                  (fun (x : College) => S / ↑(Fintype.card College)) P ∧
                (∀ (Q : College → ℝ),
                   PG23MonocultureMatching.pg23SourceMarketClearing
                     (PG23MonocultureMatching.pg23SourceAggregateDemand
                       (PG23MonocultureMatching.pg23PolycultureTypeLaw valueLaw noiseLaw))
                     (fun (x : College) => S / ↑(Fintype.card College)) Q →
                   Q = P) ∧
                ∃ (p : ℝ), ∀ (c : College), P c = p
```

Retained governing declarations

- PG23MonocultureMatching.pg23ConnectedSupport
- PG23MonocultureMatching.pg23MonocultureTypeLaw
- PG23MonocultureMatching.pg23PolycultureTypeLaw
- PG23MonocultureMatching.pg23SourceAggregateDemand
- PG23MonocultureMatching.pg23SourceMarketClearing
- PG23MonocultureMatching.pg23SourceScoreLevelNull

Lean-elaborated claim roles

1. parameter: Type v
2. parameter: Fintype College
3. assumption: Nonempty College
4. conclusion: (∀ (valueLaw noiseLaw : MeasureTheory.Measure ℝ) [inst : MeasureTheory.IsProbabilityMeasure valueLaw]
   [inst_1 : MeasureTheory.IsProbabilityMeasure noiseLaw],
   PG23MonocultureMatching.pg23ConnectedSupport valueLaw →

```
    PG23MonocultureMatching.pg23ConnectedSupport noiseLaw →
      PG23MonocultureMatching.pg23SourceScoreLevelNull
          (PG23MonocultureMatching.pg23MonocultureTypeLaw valueLaw noiseLaw) →
        ∀ (S : ℝ),
          0 < S ∧ S < 1 →
            ∃ (P : College → ℝ),
              PG23MonocultureMatching.pg23SourceMarketClearing
                  (PG23MonocultureMatching.pg23SourceAggregateDemand
                    (PG23MonocultureMatching.pg23MonocultureTypeLaw valueLaw noiseLaw))
                  (fun (x : College) => S / ↑(Fintype.card College)) P ∧
                (∀ (Q : College → ℝ),
                    PG23MonocultureMatching.pg23SourceMarketClearing
                        (PG23MonocultureMatching.pg23SourceAggregateDemand
                          (PG23MonocultureMatching.pg23MonocultureTypeLaw valueLaw noiseLaw))
                        (fun (x : College) => S / ↑(Fintype.card College)) Q →
                      Q = P) ∧
                ∃ (p : ℝ), ∀ (c : College), P c = p) ∧
∀ (valueLaw noiseLaw : MeasureTheory.Measure ℝ) [inst : MeasureTheory.IsProbabilityMeasure valueLaw]
  [inst_1 : MeasureTheory.IsProbabilityMeasure noiseLaw],
  PG23MonocultureMatching.pg23ConnectedSupport valueLaw →
    PG23MonocultureMatching.pg23ConnectedSupport noiseLaw →
      PG23MonocultureMatching.pg23SourceScoreLevelNull
          (PG23MonocultureMatching.pg23PolycultureTypeLaw valueLaw noiseLaw) →
        ∀ (S : ℝ),
          0 < S ∧ S < 1 →
            ∃ (P : College → ℝ),
              PG23MonocultureMatching.pg23SourceMarketClearing
                  (PG23MonocultureMatching.pg23SourceAggregateDemand
                    (PG23MonocultureMatching.pg23PolycultureTypeLaw valueLaw noiseLaw))
                  (fun (x : College) => S / ↑(Fintype.card College)) P ∧
                (∀ (Q : College → ℝ),
                    PG23MonocultureMatching.pg23SourceMarketClearing
                        (PG23MonocultureMatching.pg23SourceAggregateDemand
                          (PG23MonocultureMatching.pg23PolycultureTypeLaw valueLaw noiseLaw))
                        (fun (x : College) => S / ↑(Fintype.card College)) Q →
                      Q = P) ∧
                ∃ (p : ℝ), ∀ (c : College), P c = p
```

Lean proof endpoint (built separately)

PG23MonocultureMatching.PaperInterface.review_lemma2_equalCutoffsSpec_proof

Recorded source-to-Spec screening **Verdict:** matches_approved_corrected_target

Reason: For each literal monoculture and polyculture law, the target assumes connected supports, interior total supply, and the applicable score-level-null condition, then gives a clearing cutoff, uniqueness against every other clearing cutoff, and equality of all coordinates. This exactly implements the approved PG23-CONTINUOUS-REGULARITY-01 correction.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 11

Source locator: cited publication, lines 539-558

Verbatim source input

```
\begin{theorem}[Wisdom of the Crowds in Polyculture but not Monoculture]\label{thm:wisdom}
   Suppose $\DD$ is maximum concentrating. Then:
   \begin{enumerate}
      \item[(i)] Under polyculture, the probability an applicant is matched approaches $0$ or $1$ as $\numcolleges\rightarrow \infty$:
         \begin{equation}
            \lim_{m\rightarrow \infty} \Pr[\mu_{\poly}(v)\in \colleges]
            = \begin{cases}
               0 &\quad v < v_S \\
               1 &\quad v > v_S
            \end{cases}
            \quad,
         \end{equation}
         and firm welfare approaches optimal.
      \item[(ii)] Under monoculture, the probability an applicant is matched
      \begin{equation}
         \Pr[\mu_{\mono}(v)\in \colleges]
      \end{equation}
      is constant in $\numcolleges$, and firm welfare is constant and suboptimal.
   \end{enumerate}
\end{theorem}
```

Formalized review target

The archival source above is not asserted equivalent to this formalized target.

With atomless connected-support value and noise laws, literal score-level-null market clearing, and maximum-concentrating noise, literal weak-event polyculture matching converges to the efficient upper-tail allocation and monoculture matching is invariant. With nondegenerate noise and absolute integrability of value, welfare converges to upper-tail welfare under polyculture and is invariant and strictly lower under monoculture.

Archival source anchor: cited publication, lines 539-558

Expanded PaperInterface specification

```
(∀ (valueLaw noiseLaw : MeasureTheory.Measure ℝ) [inst : MeasureTheory.IsProbabilityMeasure valueLaw]
   [inst_1 : MeasureTheory.IsProbabilityMeasure noiseLaw] [MeasureTheory.NoAtoms valueLaw]
   [MeasureTheory.NoAtoms noiseLaw],
   PG23MonocultureMatching.pg23ConnectedSupport valueLaw →
     PG23MonocultureMatching.pg23ConnectedSupport noiseLaw →
       ∀ {monoCutoff polyCutoff : ℕ → ℝ} {vS S : ℝ},
         0 < S ∧ S < 1 →
           AppliedModelingLib.Probability.upperTailMass valueLaw vS = S →
             PG23MonocultureMatching.pg23MaximumConcentratingNoiseLaw noiseLaw →
               (∀ (n : ℕ),
                  PG23MonocultureMatching.pg23SourceScoreLevelNull
                    (PG23MonocultureMatching.pg23MonocultureTypeLaw valueLaw noiseLaw)) →
                 (∀ (n : ℕ),
                    PG23MonocultureMatching.pg23SourceScoreLevelNull
                      (PG23MonocultureMatching.pg23PolycultureTypeLaw valueLaw noiseLaw)) →
                   (∀ (n : ℕ),
                      PG23MonocultureMatching.pg23SourceMarketClearing
                        (PG23MonocultureMatching.pg23SourceAggregateDemand
                          (PG23MonocultureMatching.pg23MonocultureTypeLaw valueLaw noiseLaw))
                        (fun (x : Fin (n + 1)) => S / ↑(Fintype.card (Fin (n + 1)))) fun (x : Fin (n + 1)) =>
                        monoCutoff n) →
                     (∀ (n : ℕ),
                        PG23MonocultureMatching.pg23SourceMarketClearing
                          (PG23MonocultureMatching.pg23SourceAggregateDemand
                            (PG23MonocultureMatching.pg23PolycultureTypeLaw valueLaw noiseLaw))
                          (fun (x : Fin (n + 1)) => S / ↑(Fintype.card (Fin (n + 1)))) fun (x : Fin (n + 1)) =>
                          polyCutoff n) →
                       (∀ v < vS,
                          Filter.Tendsto
                            (fun (n : ℕ) =>
                              AppliedModelingLib.Matching.cutoffWeakCrossingProbability
                                (MeasureTheory.Measure.pi fun (x : Fin (n + 1)) => noiseLaw) Finset.univ v
                                fun (x : Fin (n + 1)) => polyCutoff n)
                            Filter.atTop (nhds 0)) ∧
                         (∀ (v : ℝ),
                            vS < v →
                              Filter.Tendsto
                                (fun (n : ℕ) =>
                                  AppliedModelingLib.Matching.cutoffWeakCrossingProbability
                                    (MeasureTheory.Measure.pi fun (x : Fin (n + 1)) => noiseLaw) Finset.univ v
                                    fun (x : Fin (n + 1)) => polyCutoff n)
                                Filter.atTop (nhds 1)) ∧
                           ∀ (v : ℝ) (m n : ℕ),
                             noiseLaw.real {noise : ℝ | monoCutoff m ≤ v + noise} =
                               noiseLaw.real {noise : ℝ | monoCutoff n ≤ v + noise}) ∧
 ∀ (valueLaw noiseLaw : MeasureTheory.Measure ℝ) [inst : MeasureTheory.IsProbabilityMeasure valueLaw]
   [inst_1 : MeasureTheory.IsProbabilityMeasure noiseLaw] [MeasureTheory.NoAtoms valueLaw]
   [MeasureTheory.NoAtoms noiseLaw],
   PG23MonocultureMatching.pg23ConnectedSupport valueLaw →
     PG23MonocultureMatching.pg23ConnectedSupport noiseLaw →
       (∃ (x : ℝ) (y : ℝ), x ∈ noiseLaw.support ∧ y ∈ noiseLaw.support ∧ x < y) →
         ∀ {monoCutoff polyCutoff : ℕ → ℝ} {vS S : ℝ},
```

```
0 < S ∧ S < 1 →
  AppliedModelingLib.Probability.upperTailMass valueLaw vS = S →
    PG23MonocultureMatching.pg23MaximumConcentratingNoiseLaw noiseLaw →
      (∀ (n : ℕ),
         PG23MonocultureMatching.pg23SourceScoreLevelNull
           (PG23MonocultureMatching.pg23MonocultureTypeLaw valueLaw noiseLaw)) →
        (∀ (n : ℕ),
           PG23MonocultureMatching.pg23SourceScoreLevelNull
             (PG23MonocultureMatching.pg23PolycultureTypeLaw valueLaw noiseLaw)) →
          (∀ (n : ℕ),
             PG23MonocultureMatching.pg23SourceMarketClearing
               (PG23MonocultureMatching.pg23SourceAggregateDemand
                 (PG23MonocultureMatching.pg23MonocultureTypeLaw valueLaw noiseLaw))
               (fun (x : Fin (n + 1)) => S / ↑(Fintype.card (Fin (n + 1)))) fun (x : Fin (n + 1)) =>
               monoCutoff n) →
            (∀ (n : ℕ),
               PG23MonocultureMatching.pg23SourceMarketClearing
                 (PG23MonocultureMatching.pg23SourceAggregateDemand
                   (PG23MonocultureMatching.pg23PolycultureTypeLaw valueLaw noiseLaw))
                 (fun (x : Fin (n + 1)) => S / ↑(Fintype.card (Fin (n + 1)))) fun (x : Fin (n + 1)) =>
                 polyCutoff n) →
              MeasureTheory.Integrable (fun (v : ℝ) => |v|) valueLaw →
                Filter.Tendsto
                  (fun (n : ℕ) =>
                    ∫ (v : ℝ),
                      v *
                        AppliedModelingLib.Matching.cutoffWeakCrossingProbability
                          (MeasureTheory.Measure.pi fun (x : Fin (n + 1)) => noiseLaw) Finset.univ v
                          fun (x : Fin (n + 1)) => polyCutoff n ∂valueLaw)
                  Filter.atTop (nhds (∫ (v : ℝ), v * if vS < v then 1 else 0 ∂valueLaw)) ∧
                (∀ (m n : ℕ),
                   ∫ (v : ℝ), v * noiseLaw.real {noise : ℝ | monoCutoff m ≤ v + noise} ∂valueLaw =
                     ∫ (v : ℝ), v * noiseLaw.real {noise : ℝ | monoCutoff n ≤ v + noise} ∂valueLaw) ∧
                  ∫ (v : ℝ), v * noiseLaw.real {noise : ℝ | monoCutoff 0 ≤ v + noise} ∂valueLaw <
                    ∫ (v : ℝ), v * if vS < v then 1 else 0 ∂valueLaw
```

Retained governing declarations

- AppliedModelingLib.Matching.cutoffWeakCrossingProbability
- AppliedModelingLib.Probability.upperTailMass
- PG23MonocultureMatching.pg23ConnectedSupport
- PG23MonocultureMatching.pg23MaximumConcentratingNoiseLaw
- PG23MonocultureMatching.pg23MonocultureTypeLaw
- PG23MonocultureMatching.pg23PolycultureTypeLaw
- PG23MonocultureMatching.pg23SourceAggregateDemand
- PG23MonocultureMatching.pg23SourceMarketClearing
- PG23MonocultureMatching.pg23SourceScoreLevelNull

Lean-elaborated claim roles

1. conclusion:
```
(∀ (valueLaw noiseLaw : MeasureTheory.Measure ℝ) [inst : MeasureTheory.IsProbabilityMeasure valueLaw]
  [inst_1 : MeasureTheory.IsProbabilityMeasure noiseLaw] [MeasureTheory.NoAtoms noiseLaw]
  [MeasureTheory.NoAtoms valueLaw]
  PG23MonocultureMatching.pg23ConnectedSupport valueLaw →
    PG23MonocultureMatching.pg23ConnectedSupport noiseLaw →
      ∀ {monoCutoff polyCutoff : ℕ → ℝ} {vS S : ℝ},
        0 < S ∧ S < 1 →
          AppliedModelingLib.Probability.upperTailMass valueLaw vS = S →
            PG23MonocultureMatching.pg23MaximumConcentratingNoiseLaw noiseLaw →
              (∀ (n : ℕ),
                 PG23MonocultureMatching.pg23SourceScoreLevelNull
                   (PG23MonocultureMatching.pg23MonocultureTypeLaw valueLaw noiseLaw)) →
                (∀ (n : ℕ),
                   PG23MonocultureMatching.pg23SourceScoreLevelNull
                     (PG23MonocultureMatching.pg23PolycultureTypeLaw valueLaw noiseLaw)) →
                  (∀ (n : ℕ),
                     PG23MonocultureMatching.pg23SourceMarketClearing
                       (PG23MonocultureMatching.pg23SourceAggregateDemand
                         (PG23MonocultureMatching.pg23MonocultureTypeLaw valueLaw noiseLaw))
                       (fun (x : Fin (n + 1)) => S / ↑(Fintype.card (Fin (n + 1)))) fun (x : Fin (n + 1)) =>
                       monoCutoff n) →
                    (∀ (n : ℕ),
                       PG23MonocultureMatching.pg23SourceMarketClearing
                         (PG23MonocultureMatching.pg23SourceAggregateDemand
                           (PG23MonocultureMatching.pg23PolycultureTypeLaw valueLaw noiseLaw))
                         (fun (x : Fin (n + 1)) => S / ↑(Fintype.card (Fin (n + 1)))) fun (x : Fin (n + 1)) =>
                         polyCutoff n) →
                      (∀ v < vS,
                         Filter.Tendsto
                           (fun (n : ℕ) =>
                             AppliedModelingLib.Matching.cutoffWeakCrossingProbability
                               (MeasureTheory.Measure.pi fun (x : Fin (n + 1)) => noiseLaw) Finset.univ v
                               fun (x : Fin (n + 1)) => polyCutoff n)
```

```
                  Filter.atTop (nhds 0)) ∧
               (∀ (v : ℝ),
                  vS < v →
                    Filter.Tendsto
                      (fun (n : ℕ) =>
                        AppliedModelingLib.Matching.cutoffWeakCrossingProbability
                          (MeasureTheory.Measure.pi fun (x : Fin (n + 1)) => noiseLaw) Finset.univ v
                          fun (x : Fin (n + 1)) => polyCutoff n)
                      Filter.atTop (nhds 1)) ∧
                 ∀ (v : ℝ) (m n : ℕ),
                  noiseLaw.real {noise : ℝ | monoCutoff m ≤ v + noise} =
                    noiseLaw.real {noise : ℝ | monoCutoff n ≤ v + noise}) ∧
∀ (valueLaw noiseLaw : MeasureTheory.Measure ℝ) [inst : MeasureTheory.IsProbabilityMeasure valueLaw]
  [inst_1 : MeasureTheory.IsProbabilityMeasure noiseLaw] [MeasureTheory.NoAtoms valueLaw]
  [MeasureTheory.NoAtoms noiseLaw],
  PG23MonocultureMatching.pg23ConnectedSupport valueLaw →
    PG23MonocultureMatching.pg23ConnectedSupport noiseLaw →
      (∃ (x : ℝ) (y : ℝ), x ∈ noiseLaw.support ∧ y ∈ noiseLaw.support ∧ x < y) →
        ∀ {monoCutoff polyCutoff : ℕ → ℝ} {vS S : ℝ},
          0 < S ∧ S < 1 →
            AppliedModelingLib.Probability.upperTailMass valueLaw vS = S →
              PG23MonocultureMatching.pg23MaximumConcentratingNoiseLaw noiseLaw →
                (∀ (n : ℕ),
                    PG23MonocultureMatching.pg23SourceScoreLevelNull
                      (PG23MonocultureMatching.pg23MonocultureTypeLaw valueLaw noiseLaw)) →
                  (∀ (n : ℕ),
                      PG23MonocultureMatching.pg23SourceScoreLevelNull
                        (PG23MonocultureMatching.pg23PolycultureTypeLaw valueLaw noiseLaw)) →
                    (∀ (n : ℕ),
                        PG23MonocultureMatching.pg23SourceMarketClearing
                          (PG23MonocultureMatching.pg23SourceAggregateDemand
                            (PG23MonocultureMatching.pg23MonocultureTypeLaw valueLaw noiseLaw))
                          (fun (x : Fin (n + 1)) => S / ↑(Fintype.card (Fin (n + 1)))) fun (x : Fin (n + 1)) =>
                          monoCutoff n) →
                      (∀ (n : ℕ),
                          PG23MonocultureMatching.pg23SourceMarketClearing
                            (PG23MonocultureMatching.pg23SourceAggregateDemand
                              (PG23MonocultureMatching.pg23PolycultureTypeLaw valueLaw noiseLaw))
                            (fun (x : Fin (n + 1)) => S / ↑(Fintype.card (Fin (n + 1)))) fun (x : Fin (n + 1)) =>
                            polyCutoff n) →
                        MeasureTheory.Integrable (fun (v : ℝ) => |v|) valueLaw →
                          Filter.Tendsto
                              (fun (n : ℕ) =>
                                ∫ (v : ℝ),
                                  v *
                                    AppliedModelingLib.Matching.cutoffWeakCrossingProbability
                                      (MeasureTheory.Measure.pi fun (x : Fin (n + 1)) => noiseLaw) Finset.univ v
                                      fun (x : Fin (n + 1)) => polyCutoff n ∂valueLaw)
                              Filter.atTop (nhds (∫ (v : ℝ), v * if vS < v then 1 else 0 ∂valueLaw)) ∧
                            (∀ (m n : ℕ),
                                ∫ (v : ℝ), v * noiseLaw.real {noise : ℝ | monoCutoff m ≤ v + noise} ∂valueLaw =
                                  ∫ (v : ℝ), v * noiseLaw.real {noise : ℝ | monoCutoff n ≤ v + noise} ∂valueLaw) ∧
                              ∫ (v : ℝ), v * noiseLaw.real {noise : ℝ | monoCutoff 0 ≤ v + noise} ∂valueLaw <
                                ∫ (v : ℝ), v * if vS < v then 1 else 0 ∂valueLaw
```

Lean proof endpoint (built separately)

PG23MonocultureMatching.PaperInterface.review_theorem1_wisdomSpec_proof

Recorded source-to-Spec screening **Verdict:** matches_approved_corrected_target

Reason: Compared archival Theorem 1, the disclosed PG23-CONTINUOUS-REGULARITY-01 corrected target, the complete expanded proposition, and all displayed prerequisites. The first conjunct assumes probability value/noise laws with atomlessness and connected support, the upper-tail threshold and capacity share, maximum-concentrating noise, literal score-level-null type laws, and literal exact clearing; it concludes the two pointwise polyculture weak-crossing limits and invariance of monoculture weak-crossing probability. The second conjunct adds nondegenerate noise and absolute value-integrability and concludes convergence to upper-tail value welfare, invariance of monoculture welfare, and strict inferiority of monoculture welfare. The displayed common-noise/iid-noise type laws, weak crossing event, upper-tail mass, and expected-maximum concentration definitions preserve those formulas. This matches the expressly non-equivalent corrected target rather than claiming equivalence to the archival theorem without the disclosed regularity.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 12

Source locator: cited publication, lines 609-628

Verbatim source input

```
\begin{theorem}[Likelihood of Matching to Top Choice or At All]\label{thm:top-choice}
The following hold:
\begin{enumerate}
   \item[(i)] For all $v$, the probability an applicant matches to their top choice is at least as high under monoculture as under polyculture:
   \begin{equation}\label{eq:prob-first}
      \Pr[\Rank_v(\mu_{\mono}(v)) = 1] \ge \Pr[\Rank_v(\mu_{\poly}(v)) = 1],
   \end{equation}
   and the inequality is strict for all $v\in (P_{\mono} - X_+, P_{\mono} - X_-),$ a set of positive $\eta$-measure.
   \item[(ii)] For all $v$, if $\mu_{\mono}(v)\in \colleges$,
   \begin{equation}
      \Rank_v(\mu_{\mono}(v)) = 1,
   \end{equation}
   meaning that all applicants who match under monoculture are matched to their top choice, and that the matching attains optimal applicant welfare.
   \item[(iii)]
   When $\DD$ is maximum concentrating, for all $v\in (v_S, P_{\mono} - X_-),$ which is a set of positive $\eta$-measure, and for all $\numcolleges$
   ↪  sufficiently large, an applicant with value $v$ is less likely to match under monoculture than under polyculture:
   \begin{equation}\label{eq:tiger-2}
      \Pr[\mu_{\mono}(v)\in \colleges] < \Pr[\mu_{\poly}(v)\in \colleges].
   \end{equation}
\end{enumerate}
\end{theorem}
```

Formalized review target

The archival source above is not asserted equivalent to this formalized target.

With connected-support laws, atomless nondegenerate noise, literal score-level-null economies, and exact clearing, literal weak-event top-choice
↪ probability is weakly higher under monoculture and strictly higher on the support-interior positive-mass region. Every monoculture match is top
↪ choice. With atomless values and maximum-concentrating noise, polyculture eventually has higher literal weak-event overall match probability on the
↪ positive-mass region where v exceeds the efficient threshold and monoculture matching probability is below one.

Archival source anchor: cited publication, lines 609-628

Expanded PaperInterface specification

```
(∀ {n : ℕ} [NeZero n] (valueLaw noiseLaw : MeasureTheory.Measure ℝ) [inst : MeasureTheory.IsProbabilityMeasure valueLaw]
   [inst_1 : MeasureTheory.IsProbabilityMeasure noiseLaw] [MeasureTheory.NoAtoms noiseLaw],
   1 < n →
     PG23MonocultureMatching.pg23ConnectedSupport valueLaw →
       PG23MonocultureMatching.pg23ConnectedSupport noiseLaw →
         (∃ (x : ℝ) (y : ℝ), x ∈ noiseLaw.support ∧ y ∈ noiseLaw.support ∧ x < y) →
           ∀ {Pmono Ppoly S : ℝ},
             0 < S ∧ S < 1 →
               (∀ (c : Fin n) (x : ℝ),
                  (PG23MonocultureMatching.pg23MonocultureTypeLaw valueLaw noiseLaw)
                     {theta : PG23MonocultureMatching.PG23ApplicantType (Fin n) |
                       PG23MonocultureMatching.pg23SourceScore theta c = x} =
                   0) →
                 (∀ (c : Fin n) (x : ℝ),
                    (PG23MonocultureMatching.pg23PolycultureTypeLaw valueLaw noiseLaw)
                       {theta : PG23MonocultureMatching.PG23ApplicantType (Fin n) |
                         PG23MonocultureMatching.pg23SourceScore theta c = x} =
                     0) →
                   (PG23MonocultureMatching.pg23SourceMarketClearing
                      (PG23MonocultureMatching.pg23SourceAggregateDemand
                        (PG23MonocultureMatching.pg23MonocultureTypeLaw valueLaw noiseLaw))
                      (fun (x : Fin n) => S / ↑(Fintype.card (Fin n))) fun (x : Fin n) => Pmono) →
                     (PG23MonocultureMatching.pg23SourceMarketClearing
                        (PG23MonocultureMatching.pg23SourceAggregateDemand
                          (PG23MonocultureMatching.pg23PolycultureTypeLaw valueLaw noiseLaw))
                        (fun (x : Fin n) => S / ↑(Fintype.card (Fin n))) fun (x : Fin n) => Ppoly) →
                       (∀ (v : ℝ),
                          noiseLaw.real {noise : ℝ | Ppoly ≤ v + noise} ≤
                            noiseLaw.real {noise : ℝ | Pmono ≤ v + noise}) ∧
                         0 < valueLaw (PG23MonocultureMatching.pg23NoiseInteriorCutoffRegion noiseLaw Pmono) ∧
                           ∀ v ∈ PG23MonocultureMatching.pg23NoiseInteriorCutoffRegion noiseLaw Pmono,
                             noiseLaw.real {noise : ℝ | Ppoly ≤ v + noise} <
                               noiseLaw.real {noise : ℝ | Pmono ≤ v + noise}) ∧
  (∀ {College : Type v} [inst : Fintype College] [Nonempty College] (value noise cutoff : ℝ)
     (rank : College → Fin (Fintype.card College)) (hrank : Function.Bijective rank) (c : College),
     PG23MonocultureMatching.pg23SourceChoice (fun (x : College) => cutoff)
         (PG23MonocultureMatching.pg23MonocultureType value noise (Equiv.ofBijective rank hrank)) =
       some c →
       PG23MonocultureMatching.pg23SourceRank
         (PG23MonocultureMatching.pg23MonocultureType value noise (Equiv.ofBijective rank hrank)) c =
       0) ∧
   ∀ (valueLaw noiseLaw : MeasureTheory.Measure ℝ) [inst : MeasureTheory.IsProbabilityMeasure valueLaw]
     [inst_1 : MeasureTheory.IsProbabilityMeasure noiseLaw] [MeasureTheory.NoAtoms valueLaw]
     [MeasureTheory.NoAtoms noiseLaw],
     IsPreconnected valueLaw.support →
       IsPreconnected noiseLaw.support →
         (∃ (x : ℝ) (y : ℝ), x ∈ noiseLaw.support ∧ y ∈ noiseLaw.support ∧ x < y) →
           ∀ {monoCutoff polyCutoff : ℕ → ℝ} {vS S : ℝ},
             0 < S ∧ S < 1 →
               AppliedModelingLib.Probability.upperTailMass valueLaw vS = S →
```

```
PG23MonocultureMatching.pg23MaximumConcentratingNoiseLaw noiseLaw →
  (∀ (n : ℕ),
     PG23MonocultureMatching.pg23SourceScoreLevelNull
       (PG23MonocultureMatching.pg23MonocultureTypeLaw valueLaw noiseLaw)) →
    (∀ (n : ℕ),
       PG23MonocultureMatching.pg23SourceScoreLevelNull
         (PG23MonocultureMatching.pg23PolycultureTypeLaw valueLaw noiseLaw)) →
      (∀ (n : ℕ),
         PG23MonocultureMatching.pg23SourceMarketClearing
           (PG23MonocultureMatching.pg23SourceAggregateDemand
             (PG23MonocultureMatching.pg23MonocultureTypeLaw valueLaw noiseLaw))
           (fun (x : Fin (n + 1)) => S / ↑(Fintype.card (Fin (n + 1)))) fun (x : Fin (n + 1)) =>
           monoCutoff n) →
        (∀ (n : ℕ),
           PG23MonocultureMatching.pg23SourceMarketClearing
             (PG23MonocultureMatching.pg23SourceAggregateDemand
               (PG23MonocultureMatching.pg23PolycultureTypeLaw valueLaw noiseLaw))
             (fun (x : Fin (n + 1)) => S / ↑(Fintype.card (Fin (n + 1)))) fun (x : Fin (n + 1)) =>
             polyCutoff n) →
          0 < valueLaw {v : ℝ | vS < v ∧ noiseLaw.real {noise : ℝ | monoCutoff 0 ≤ v + noise} < 1} ∧
            ∀ v ∈ {w : ℝ | vS < w ∧ noiseLaw.real {noise : ℝ | monoCutoff 0 ≤ w + noise} < 1},
              ∀ᶠ (n : ℕ) in Filter.atTop,
                noiseLaw.real {noise : ℝ | monoCutoff n ≤ v + noise} <
                  AppliedModelingLib.Matching.cutoffWeakCrossingProbability
                    (MeasureTheory.Measure.pi fun (x : Fin (n + 1)) => noiseLaw) Finset.univ v
                    fun (x : Fin (n + 1)) => polyCutoff n
```

Retained governing declarations

- AL16SupplyDemandMatching.instMeasurableSpaceAL16Ranking
- AppliedModelingLib.Matching.cutoffWeakCrossingProbability
- AppliedModelingLib.Probability.upperTailMass
- PG23MonocultureMatching.PG23ApplicantType
- PG23MonocultureMatching.PG23Ranking
- PG23MonocultureMatching.pg23ConnectedSupport
- PG23MonocultureMatching.pg23MaximumConcentratingNoiseLaw
- PG23MonocultureMatching.pg23MonocultureType
- PG23MonocultureMatching.pg23MonocultureTypeLaw
- PG23MonocultureMatching.pg23NoiseInteriorCutoffRegion
- PG23MonocultureMatching.pg23PolycultureTypeLaw
- PG23MonocultureMatching.pg23SourceAggregateDemand
- PG23MonocultureMatching.pg23SourceChoice
- PG23MonocultureMatching.pg23SourceMarketClearing
- PG23MonocultureMatching.pg23SourceRank
- PG23MonocultureMatching.pg23SourceScore
- PG23MonocultureMatching.pg23SourceScoreLevelNull

Lean-elaborated claim roles

1. conclusion: (∀ {n : ℕ} [NeZero n] (valueLaw noiseLaw : MeasureTheory.Measure ℝ) [inst : MeasureTheory.IsProbabilityMeasure valueLaw]
   [inst_1 : MeasureTheory.IsProbabilityMeasure noiseLaw] [MeasureTheory.NoAtoms noiseLaw],
   1 < n →
     PG23MonocultureMatching.pg23ConnectedSupport valueLaw →
       PG23MonocultureMatching.pg23ConnectedSupport noiseLaw →
         (∃ (x : ℝ) (y : ℝ), x ∈ noiseLaw.support ∧ y ∈ noiseLaw.support ∧ x < y) →
           ∀ {Pmono Ppoly S : ℝ},
             0 < S ∧ S < 1 →
               (∀ (c : Fin n) (x : ℝ),
                  (PG23MonocultureMatching.pg23MonocultureTypeLaw valueLaw noiseLaw)
                    {theta : PG23MonocultureMatching.PG23ApplicantType (Fin n) |
                      PG23MonocultureMatching.pg23SourceScore theta c = x} =
                  0) →
                 (∀ (c : Fin n) (x : ℝ),
                    (PG23MonocultureMatching.pg23PolycultureTypeLaw valueLaw noiseLaw)
                      {theta : PG23MonocultureMatching.PG23ApplicantType (Fin n) |
                        PG23MonocultureMatching.pg23SourceScore theta c = x} =
                    0) →
                   (PG23MonocultureMatching.pg23SourceMarketClearing
                      (PG23MonocultureMatching.pg23SourceAggregateDemand
                        (PG23MonocultureMatching.pg23MonocultureTypeLaw valueLaw noiseLaw))
                      (fun (x : Fin n) => S / ↑(Fintype.card (Fin n))) fun (x : Fin n) => Pmono) →
                     (PG23MonocultureMatching.pg23SourceMarketClearing
                        (PG23MonocultureMatching.pg23SourceAggregateDemand

```
                  (PG23MonocultureMatching.pg23PolycultureTypeLaw valueLaw noiseLaw))
                (fun (x : Fin n) => S / ↑(Fintype.card (Fin n))) fun (x : Fin n) => Ppoly) →
              (∀ (v : ℝ),
                 noiseLaw.real {noise : ℝ | Ppoly ≤ v + noise} ≤
                   noiseLaw.real {noise : ℝ | Pmono ≤ v + noise}) ∧
                0 < valueLaw (PG23MonocultureMatching.pg23NoiseInteriorCutoffRegion noiseLaw Pmono) ∧
                  ∀ v ∈ PG23MonocultureMatching.pg23NoiseInteriorCutoffRegion noiseLaw Pmono,
                    noiseLaw.real {noise : ℝ | Ppoly ≤ v + noise} <
                      noiseLaw.real {noise : ℝ | Pmono ≤ v + noise}) ∧
(∀ {College : Type v} [inst : Fintype College] [Nonempty College] (value noise cutoff : ℝ)
   (rank : College → Fin (Fintype.card College)) (hrank : Function.Bijective rank) (c : College),
   PG23MonocultureMatching.pg23SourceChoice (fun (x : College) => cutoff)
       (PG23MonocultureMatching.pg23MonocultureType value noise (Equiv.ofBijective rank hrank)) =
     some c →
    PG23MonocultureMatching.pg23SourceRank
       (PG23MonocultureMatching.pg23MonocultureType value noise (Equiv.ofBijective rank hrank)) c =
     0) ∧
 ∀ (valueLaw noiseLaw : MeasureTheory.Measure ℝ) [inst : MeasureTheory.IsProbabilityMeasure valueLaw]
   [inst_1 : MeasureTheory.IsProbabilityMeasure noiseLaw] [MeasureTheory.NoAtoms valueLaw]
   [MeasureTheory.NoAtoms noiseLaw],
   IsPreconnected valueLaw.support →
     IsPreconnected noiseLaw.support →
       (∃ (x : ℝ) (y : ℝ), x ∈ noiseLaw.support ∧ y ∈ noiseLaw.support ∧ x < y) →
         ∀ {monoCutoff polyCutoff : ℕ → ℝ} {vS S : ℝ},
           0 < S ∧ S < 1 →
             AppliedModelingLib.Probability.upperTailMass valueLaw vS = S →
               PG23MonocultureMatching.pg23MaximumConcentratingNoiseLaw noiseLaw →
                 (∀ (n : ℕ),
                    PG23MonocultureMatching.pg23SourceScoreLevelNull
                      (PG23MonocultureMatching.pg23MonocultureTypeLaw valueLaw noiseLaw)) →
                   (∀ (n : ℕ),
                      PG23MonocultureMatching.pg23SourceScoreLevelNull
                        (PG23MonocultureMatching.pg23PolycultureTypeLaw valueLaw noiseLaw)) →
                     (∀ (n : ℕ),
                        PG23MonocultureMatching.pg23SourceMarketClearing
                          (PG23MonocultureMatching.pg23SourceAggregateDemand
                            (PG23MonocultureMatching.pg23MonocultureTypeLaw valueLaw noiseLaw))
                          (fun (x : Fin (n + 1)) => S / ↑(Fintype.card (Fin (n + 1)))) fun (x : Fin (n + 1)) =>
                          monoCutoff n) →
                       (∀ (n : ℕ),
                          PG23MonocultureMatching.pg23SourceMarketClearing
                            (PG23MonocultureMatching.pg23SourceAggregateDemand
                              (PG23MonocultureMatching.pg23PolycultureTypeLaw valueLaw noiseLaw))
                            (fun (x : Fin (n + 1)) => S / ↑(Fintype.card (Fin (n + 1)))) fun (x : Fin (n + 1)) =>
                            polyCutoff n) →
                         0 < valueLaw {v : ℝ | vS < v ∧ noiseLaw.real {noise : ℝ | monoCutoff 0 ≤ v + noise} < 1} ∧
                           ∀ v ∈ {w : ℝ | vS < w ∧ noiseLaw.real {noise : ℝ | monoCutoff 0 ≤ w + noise} < 1},
                             ∀ᶠ (n : ℕ) in Filter.atTop,
                               noiseLaw.real {noise : ℝ | monoCutoff n ≤ v + noise} <
                                 AppliedModelingLib.Matching.cutoffWeakCrossingProbability
                                   (MeasureTheory.Measure.pi fun (x : Fin (n + 1)) => noiseLaw) Finset.univ v
                                   fun (x : Fin (n + 1)) => polyCutoff n
```

Lean proof endpoint (built separately)

PG23MonocultureMatching.PaperInterface.review_theorem2_topChoiceSpec_proof

Recorded source-to-Spec screening

Verdict: matches_approved_corrected_target
Reason: Compared archival Theorem 2, the disclosed PG23-ENDPOINT-AND-INTERVAL-01 and PG23-CONTINUOUS-REGULARITY-01 corrected target, the complete expanded proposition, and all displayed prerequisites. The first conjunct gives the weak top-choice tail inequality for every value and the strict inequality throughout the translated interior-noise-support region, whose value-law mass is positive, under the stated connected-support, atomless nondegenerate-noise, score-level-null, and exact-clearing conditions. The second conjunct says every college selected under common monoculture noise has the top zero-based Lean rank. The third conjunct, with atomless values and maximum concentration, gives a positive-mass region where value exceeds the upper-tail threshold and monoculture weak-match probability is below one, and eventual strict dominance there by polyculture weak-crossing probability. The displayed ranking, common/iid noise, favorite-affordable choice, clearing, weak-crossing, support-interior, and maximum-concentration definitions preserve the corrected formulas. This matches the expressly non-equivalent correction replacing endpoint arithmetic and exposing the required regularity.
Reviewer check: ☐ Matches formalized target
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

Review row 13

Source locator: cited publication, lines 681-684

Verbatim source input

```
\begin{lemma}[Equal Cutoffs Lemma for Differential Application Access]\label{lem:equal-cutoffs-diff-access}
   If $\theta\in \{\theta_{\mono, \kappa}, \theta_{\poly, \kappa}\},$ then there is a unique vector of market-clearing cutoffs $P$, and
   $P_1 = P_2 = \cdots = P_\numcolleges.$
\end{lemma}
```

Formalized review target

The archival source above is not asserted equivalent to this formalized target.

For each differential-application-access monoculture and polyculture economy with connected-support laws, zero mass on every induced score level, and 0 < S < 1, there is a unique market-clearing cutoff vector and all its coordinates are equal.

Archival source anchor: cited publication, lines 681-684

Expanded PaperInterface specification

```
∀ {College : Type v} [inst : Fintype College] [Nonempty College],
  (∀ {n : ℕ} (accessLaw : MeasureTheory.Measure (Fin n)) [MeasureTheory.IsProbabilityMeasure accessLaw]
     (valueLaw noiseLaw : MeasureTheory.Measure ℝ) [inst_3 : MeasureTheory.IsProbabilityMeasure valueLaw]
     [inst_4 : MeasureTheory.IsProbabilityMeasure noiseLaw],
     PG23MonocultureMatching.pg23ConnectedSupport valueLaw →
       PG23MonocultureMatching.pg23ConnectedSupport noiseLaw →
         PG23MonocultureMatching.pg23SourceScoreLevelNull
             (PG23MonocultureMatching.pg23MonocultureTypeLaw valueLaw noiseLaw) →
           ∀ (S : ℝ),
             0 < S ∧ S < 1 →
               ∃ (P : College → ℝ),
                 PG23MonocultureMatching.pg23DifferentialSourceMarketClearing
                     (PG23MonocultureMatching.pg23DifferentialSourceAggregateDemand
                       (PG23MonocultureMatching.pg23MonocultureDifferentialTypeLaw accessLaw valueLaw noiseLaw))
                     (fun (x : College) => S / ↑(Fintype.card College)) P ∧
                   (∀ (Q : College → ℝ),
                       PG23MonocultureMatching.pg23DifferentialSourceMarketClearing
                           (PG23MonocultureMatching.pg23DifferentialSourceAggregateDemand
                             (PG23MonocultureMatching.pg23MonocultureDifferentialTypeLaw accessLaw valueLaw noiseLaw))
                           (fun (x : College) => S / ↑(Fintype.card College)) Q →
                         Q = P) ∧
                   ∃ (p : ℝ), ∀ (c : College), P c = p) ∧
∀ {n : ℕ} (accessLaw : MeasureTheory.Measure (Fin n)) [MeasureTheory.IsProbabilityMeasure accessLaw]
  (valueLaw noiseLaw : MeasureTheory.Measure ℝ) [inst_3 : MeasureTheory.IsProbabilityMeasure valueLaw]
  [inst_4 : MeasureTheory.IsProbabilityMeasure noiseLaw],
  PG23MonocultureMatching.pg23ConnectedSupport valueLaw →
    PG23MonocultureMatching.pg23ConnectedSupport noiseLaw →
      PG23MonocultureMatching.pg23SourceScoreLevelNull
          (PG23MonocultureMatching.pg23PolycultureTypeLaw valueLaw noiseLaw) →
        ∀ (S : ℝ),
          0 < S ∧ S < 1 →
            ∃ (P : College → ℝ),
              PG23MonocultureMatching.pg23DifferentialSourceMarketClearing
                  (PG23MonocultureMatching.pg23DifferentialSourceAggregateDemand
                    (PG23MonocultureMatching.pg23PolycultureDifferentialTypeLaw accessLaw valueLaw noiseLaw))
                  (fun (x : College) => S / ↑(Fintype.card College)) P ∧
                (∀ (Q : College → ℝ),
                    PG23MonocultureMatching.pg23DifferentialSourceMarketClearing
                        (PG23MonocultureMatching.pg23DifferentialSourceAggregateDemand
                          (PG23MonocultureMatching.pg23PolycultureDifferentialTypeLaw accessLaw valueLaw noiseLaw))
                        (fun (x : College) => S / ↑(Fintype.card College)) Q →
                      Q = P) ∧
                ∃ (p : ℝ), ∀ (c : College), P c = p
```

Retained governing declarations

- PG23MonocultureMatching.pg23ConnectedSupport
- PG23MonocultureMatching.pg23DifferentialSourceAggregateDemand
- PG23MonocultureMatching.pg23DifferentialSourceMarketClearing
- PG23MonocultureMatching.pg23MonocultureDifferentialTypeLaw
- PG23MonocultureMatching.pg23MonocultureTypeLaw
- PG23MonocultureMatching.pg23PolycultureDifferentialTypeLaw
- PG23MonocultureMatching.pg23PolycultureTypeLaw
- PG23MonocultureMatching.pg23SourceScoreLevelNull

Lean-elaborated claim roles

```
1. parameter: Type v
2. parameter: Fintype College
3. assumption: Nonempty College
4. conclusion: (∀ {n : ℕ} (accessLaw : MeasureTheory.Measure (Fin n)) [MeasureTheory.IsProbabilityMeasure accessLaw]
    (valueLaw noiseLaw : MeasureTheory.Measure ℝ) [inst : MeasureTheory.IsProbabilityMeasure valueLaw]
    [inst_1 : MeasureTheory.IsProbabilityMeasure noiseLaw],
    PG23MonocultureMatching.pg23ConnectedSupport valueLaw →
      PG23MonocultureMatching.pg23ConnectedSupport noiseLaw →
        PG23MonocultureMatching.pg23SourceScoreLevelNull
            (PG23MonocultureMatching.pg23MonocultureTypeLaw valueLaw noiseLaw) →
          ∀ (S : ℝ),
            0 < S ∧ S < 1 →
              ∃ (P : College → ℝ),
                PG23MonocultureMatching.pg23DifferentialSourceMarketClearing
                    (PG23MonocultureMatching.pg23DifferentialSourceAggregateDemand
                      (PG23MonocultureMatching.pg23MonocultureDifferentialTypeLaw accessLaw valueLaw noiseLaw))
                    (fun (x : College) => S / ↑(Fintype.card College)) P ∧
                  (∀ (Q : College → ℝ),
                      PG23MonocultureMatching.pg23DifferentialSourceMarketClearing
                          (PG23MonocultureMatching.pg23DifferentialSourceAggregateDemand
                            (PG23MonocultureMatching.pg23MonocultureDifferentialTypeLaw accessLaw valueLaw noiseLaw))
                          (fun (x : College) => S / ↑(Fintype.card College)) Q →
                        Q = P) ∧
                    ∃ (p : ℝ), ∀ (c : College), P c = p) ∧
  ∀ {n : ℕ} (accessLaw : MeasureTheory.Measure (Fin n)) [MeasureTheory.IsProbabilityMeasure accessLaw]
    (valueLaw noiseLaw : MeasureTheory.Measure ℝ) [inst : MeasureTheory.IsProbabilityMeasure valueLaw]
    [inst_1 : MeasureTheory.IsProbabilityMeasure noiseLaw],
    PG23MonocultureMatching.pg23ConnectedSupport valueLaw →
      PG23MonocultureMatching.pg23ConnectedSupport noiseLaw →
        PG23MonocultureMatching.pg23SourceScoreLevelNull
            (PG23MonocultureMatching.pg23PolycultureTypeLaw valueLaw noiseLaw) →
          ∀ (S : ℝ),
            0 < S ∧ S < 1 →
              ∃ (P : College → ℝ),
                PG23MonocultureMatching.pg23DifferentialSourceMarketClearing
                    (PG23MonocultureMatching.pg23DifferentialSourceAggregateDemand
                      (PG23MonocultureMatching.pg23PolycultureDifferentialTypeLaw accessLaw valueLaw noiseLaw))
                    (fun (x : College) => S / ↑(Fintype.card College)) P ∧
                  (∀ (Q : College → ℝ),
                      PG23MonocultureMatching.pg23DifferentialSourceMarketClearing
                          (PG23MonocultureMatching.pg23DifferentialSourceAggregateDemand
                            (PG23MonocultureMatching.pg23PolycultureDifferentialTypeLaw accessLaw valueLaw noiseLaw))
                          (fun (x : College) => S / ↑(Fintype.card College)) Q →
                        Q = P) ∧
                    ∃ (p : ℝ), ∀ (c : College), P c = p
```

Lean proof endpoint (built separately)

PG23MonocultureMatching.PaperInterface.review_lemma_equalCutoffs_differentialAccessSpec_proof

Recorded source-to-Spec screening **Verdict:** matches_approved_corrected_target

Reason: The two target conjuncts cover the monoculture and polyculture differential type laws for an independent access law, assume connected supports, interior supply, and the relevant base score-level-null condition, and conclude existence, uniqueness, and a common cutoff. This is exactly the approved differential-access correction under PG23-CONTINUOUS-REGULARITY-01.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation